

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Show Choices	\$gameMessage.setChoices(choices , defaultIndex , cancelIndex); \$gameMessage.setChoiceBackground(bgType); \$gameMessage.setChoicePositionType(posType); \$gameMessage.setChoiceCallback(callback); interpreter.setWaitMode('message');	// Set choices to A, B, C; default C, no cancel \$gameMessage.setChoices(['A', 'B', 'C'], 2, -1); // Set background: Dim \$gameMessage.setChoiceBackground(1); // Set position: Center \$gameMessage.setChoicePositionType(1); // On choice, set variable #1 equal to the selected index \$gameMessage.setChoiceCallback(n => { \$gameVariables.setValue(1, n); }); // Wait for a choice to be selected this.setWaitMode('message');	String[] choices • Array of choice texts to show, in order. Number defaultIndex • Starting choice index (0 for choice #1). Number cancelIndex • Cancel selects this index (-1 for no cancel). • Use -2 for "Branch" cancel type. Number bgType • 0 (Window), 1 (Dim), or 2 (Transparent). Number posType • 0 (Left), 1 (Center), or 2 (Right). Function callback • 1 argument: selected choice index. • Gets called when a choice is made. • Runs in Game_Interpreter context. Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	You can use as many choices as you like: the window will scroll if necessary. Remember that choice #1 is index 0, #2 is index 1, etc. setWaitMode is important here.
✓	✓	Input Number	\$gameMessage.setNumberInput(variableId , maxDigits); interpreter.setWaitMode('message');	// Input a 5-digit number; store result in variable #1 \$gameMessage.setNumberInput(1, 5); // Wait for input to be confirmed this.setWaitMode('message');	Number variableId • ID of the variable used to store input. Number maxDigits • Number of digits to show. Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	setWaitMode is important here.
✓	✓	Select Item	\$gameMessage.setItemChoice(variableId , itemType); interpreter.setWaitMode('message');	// Select a Key item; store that item's ID in variable #1 \$gameMessage.setItemChoice(1, 2); // Wait for an item to be selected this.setWaitMode('message');	Number variableId • ID of the variable to store selected item's ID. Number itemType • 1 (Normal), 2 (Key), 3 (Hidden A), or 4 (Hidden B). Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	setWaitMode is important here.
✓	✓	Show Scrolling Text	\$gameMessage.setScroll(scrollSpeed , noFast); \$gameMessage.add(text); interpreter.setWaitMode('message');	// Write out the story text var text = ['People have been coming to the wise man', 'complaining about the same problems every time.\n', 'One day he told them a joke and everyone roared', 'in laughter.\n', 'After a couple of minutes, he told them the same joke', 'and only a few of them smiled.\n', 'When he told the same joke for the third time no one', 'laughed anymore.\n', 'The wise man smiled and said:', '""You can't laugh at the same joke over and over.', 'So why are you always crying about the same problem?""].join('\n'); // Set the message to scroll mode, speed 3, can fast-forward \$gameMessage.setScroll(3, false); // Add the story to the message \$gameMessage.add(text); // Wait for scrolling text to end this.setWaitMode('message');	Number scrollSpeed • Normally 1-8. Boolean noFast • If true, the player cannot fast-forward it. String text • Message text! Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	See the setWaitMode note for Show Text. You can use "\n" for a new line. The example uses the <i>Array#join</i> method to merge a list of text lines into a single, multi-line string. More information here: Array#join .
		Party				
✓	✓	Change Gold (Add)	\$gameParty.gainGold(amount);	// Give the party +1000 gold \$gameParty.gainGold(1000);	Number amount • Quantity to add/remove (can be negative).	-
✓	✓	Change Gold (Remove)	\$gameParty.loseGold(amount);	// Remove 500 gold from the party \$gameParty.loseGold(500);		
✓	✓	Change Items (Add)	\$gameParty.gainItem(item , amount);	// Give the party 5x Item ID 1 \$gameParty.gainItem(\$dataItems[1], 5);	\$dataItem item • Database item reference	-
✓	✓	Change Items (Remove)	\$gameParty.loseItem(item , amount);	// Remove up to 1 of Item 12 from the party inventory \$gameParty.loseItem(\$dataItems[12], 1);	Number amount • Quantity to add/remove (can be negative).	
✓	✓	Change Weapons (Add)	\$gameParty.gainItem(weapon , amount);	// Give the party 1x Weapon ID 1 \$gameParty.gainItem(\$dataWeapons[1], 1);	\$dataWeapon weapon • Database weapon reference. Number amount	
✓	✓	Change Weapons (Remove)	\$gameParty.loseItem(weapon , amount , includeEquip);	// Remove up to 99 of Weapon 2, including equipped items \$gameParty.loseItem(\$dataWeapons[2], 99, true);	• Quantity to add/remove (can be negative). Boolean includeEquip • If true, equipped weapons can be removed.	Equipped weapons/armors can only be removed if there are not enough of that item in inventory.
✓	✓	Change Armors (Add)	\$gameParty.gainItem(armor , amount);	// Give the party 2x Armor ID 1 \$gameParty.gainItem(\$dataArmors[1], 2);	\$dataArmor armor • Database armor reference. Number amount	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Change Armors (Remove)	\$gameParty.loseItem(armor, amount, includeEquip);	// Remove up to 5 of Armor 2, ignoring equipped items \$gameParty.loseItem(\$dataArmors[2], 5);	• Quantity to add/remove (can be negative). Boolean includeEquip • If true, equipped armors can be removed.	
✓	✓	Change Party Member (Add)	\$gameParty.addActor(actorId);	// Add actor ID 4 to the party \$gameParty.addActor(4);	Number actorId • ID of the actor to add/remove.	If used in battle you may need to refresh the battle status window to match the new party setup.
✓	✓	Change Party Member (Remove)	\$gameParty.removeActor(actorId);	// Remove actor ID 1 from the party \$gameParty.removeActor(1);		
		Actor				
✓	✓	Change HP	target.setHp(value);	// Set the HP of party member #2 (first follower) to 50% var actor = \$gameParty.members()[1]; actor.setHp(actor.mhp / 2);		-
✓	✓		target.gainHp(value);	// Heal 50 HP for actor ID 1 \$gameActors.actor(1).gainHp(10);	Game_Interpreter interpreter • Interpreter reference (this in a Script command). Game_Battler battler • The actor or enemy to gain/lose HP. Number value • Amount of HP to gain (can be negative). Boolean allowKO • If true, the target can die.	No effect if the target is dead.
✓	✓		interpreter.changeHp(target, value, allowKO);	// In a Script command: 100 damage to party leader, allow KO this.changeHp(\$gameParty.members()[0], -100, true);		The collapse effect will start if the target dies in battle.
✓	✓		Game_Interpreter.prototype.changeHp.call(this, target, value, allowKO);	// Heal 50 HP for actor ID 1 Game_Interpreter.prototype.changeHp.call(this, \$gameActors.actor(1), 50, false);		Works for actors or enemies.
✓	✓	Change MP	battler.gainMp(value);	// Make actor ID 2 lose 20 MP \$gameActors.actor(2).gainMp(-20); // Restore +10% MP for the party leader var actor = \$gameParty.members()[0]; actor.gainMp(0.1 * actor.mmp);	Game_Battler battler • The actor or enemy to gain/lose MP. Number value • Amount of MP to gain (can be negative).	Works for actors or enemies.
✓	✓	Change TP	battler.gainTp(value);	// Make party member #3 (index 2) gain 100 TP \$gameParty.members()[2].gainTp(100); // Make the party leader lose 10 TP \$gameParty.members()[0].gainTp(-10);	Game_Battler battler • The actor or enemy to gain/lose TP. Number value • Amount of TP to gain (can be negative).	Works for actors or enemies.
✓	✓	Change State (Add)	battler.addState(stateId); var target = battler; var isDead = target.isDead(); target.addState(stateId); if (!isDead && target.isDead()) target.performCollapse();	// Add state ID 1 to the party leader and check for collapse var target = \$gameParty.members()[0]; var isDead = target.isDead(); target.addState(1); if (!isDead && target.isDead()) target.performCollapse();	Game_Battler battler • The actor or enemy to gain/lose the state. Number stateId • ID of the state to add/remove.	In some situations, adding/removing a state can cause the target to die/revive, e.g. • Add state 1; • Remove a state with a "Resist State 1" trait at 0 HP. In these cases it is recommended to use the "check for collapse" version to ensure the collapse effect plays at the correct time.
✓	✓		Change State (Remove)	battler.removeState(stateId); var target = battler; var isDead = target.isDead(); target.removeState(stateId); if (!isDead && target.isDead()) target.performCollapse();		
✓	✓	Recover ALL (One)	battler.recoverAll();	// Make actor ID 1 fully recover \$gameActors.actor(1).recoverAll();	Game_Battler battler • The actor or enemy to be fully recovered.	Removes all states and sets HP & MP to maximum.
✓	✓	Recover ALL (Entire Party)	\$gameParty.members().forEach(function(actor) { actor.recoverAll(); });	-	-	If used in battle, this will only affect the battle members.
✓	✓	Change EXP	actor.gainExp(value);	// Give +1000 EXP to the party leader \$gameParty.members()[0].gainExp(1000); // Remove 500 EXP from actor ID 2 \$gameActors.actor(2).gainExp(-500);	Game_Actor actor • The actor to gain/lose EXP. Number value • Amount of EXP to gain (can be negative).	Level-up messages will be shown if appropriate.
✓	✓	Change Level	actor.changeLevel(level, show);	// Set the party leader to level 5 \$gameParty.members()[0].changeLevel(5, true); // Silently increase the level of actor ID 2 by +1 var actor = \$gameActors.actor(2); actor.changeLevel(actor.level + 1, false);	Game_Actor actor • The actor to change level. Number level • New level. Boolean show • If true, show level-up message(s).	-
✓	✓		Change Parameters	battler.addParam(paramId, value);	// Give the party leader a permanent +10 ATK bonus \$gameParty.members()[0].addParam(2, 10);	Game_Battler battler • The one to gain/lose the basic param bonus. Number paramId

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Change Parameters	<code>battler.ador.aram(paramId, value);</code>	<code>// Give actor ID 5 a permanent -20 LUK \$gameActors.actor(5).addParam(7, -20);</code>	- Basic parameter ID (see notes ->). Number value - Bonus to add (can be negative).	Basic parameter IDs: • 0 = MHP • 1 = MMP • 2 = ATK • 3 = DEF • 4 = MAT • 5 = MDF • 6 = AGI • 7 = LUK
✓	✓	Change Skill	<code>actor.learnSkill(skillId);</code>	<code>// Make the party leader learn skill ID 25 \$gameParty.members()[0].learnSkill(25);</code>	Game_Actor actor The one to learn/forget the skill.	Works for actors or enemies.
✓	✓		<code>actor.forgetSkill(skillId);</code>	<code>// Make actor ID 2 forget skill 7 \$gameActors.actor(2).forgetSkill(7);</code>	Number skillId New skill ID.	-
✓	✓	Change Equipment	<code>actor.changeEquipById(etypeId, itemId);</code>	<code>// Equip weapon/armor ID 5 in the party leader's first equip slot \$gameParty.members()[0].changeEquipById(1, 5);</code>	Game_Actor actor - The one to (un)equip the item. Number etypeId - Equipment Type ID: 1 = first slot. Number itemId - ID of the item to equip.	Equipment Types are defined on the Types tab in the database.
✓	✓	Change Name	<code>actor.setName(name);</code>	<code>// Rename the party leader to Leader \$gameParty.members()[0].setName('Leader');</code> <code>// Set the name of actor 2 equal to the name of actor 1 \$gameActors.actor(2).setName(\$gameActors.actor(1).name());</code>	Game_Actor actor - The one to rename. String name - Their new name.	-
✓	✓	Change Class	<code>actor.changeClass(classId, keepExp);</code>	<code>// Change the party leader to class ID 3, keeping previous EXP \$gameParty.members()[0].changeClass(3, true);</code> <code>// Make actor 1 change to class 1 with 0 EXP \$gameActors.actor(1).changeClass(1, false);</code>	Game_Actor actor - The one to change class. Number classId - New class ID Boolean keepExp - If false, reset EXP to 0.	-
✓	✓	Change Nickname	<code>actor.setNickname(nickname);</code>	<code>// Set the party leader's nickname to Superhero \$gameParty.members()[0].setNickname('Superhero');</code>	Game_Actor actor - The one to rename. String nickname - New nickname.	-
✓	✓	Change Profile	<code>actor.setProfile(profile);</code>	<code>// Set the party leader's profile to Here is some profile text. \$gameParty.leader.setProfile('Here is some profile text.');</code> <code>// Add to the end of actor 2's profile var actor = \$gameActors.actor(2); var text = actor.profile() + '\n\nSome text.'; actor.setProfile(text);</code>	Game_Actor actor - The one to change profile. String profile - New profile.	-
		Game Progression				
✓	✓	Control Switches	<code>\$gameSwitches.setValue(switchId, value);</code>	<code>// Turn on switch ID 1 \$gameSwitches.setValue(1, true);</code> <code>// Turn off switch ID 5 \$gameSwitches.setValue(5, false);</code>	Number switchId - ID of switch to change. Boolean value - true (ON) or false (OFF).	-
✓	✓	Control Variables	<code>\$gameVariables.setValue(variableId, value);</code>	<code>// Set variable ID 1 equal to 5 \$gameVariables.setValue(1, 5);</code>	Number variableId - ID of variable to change. Any value - Operand value (can be negative).	Variables can store any value, including: • String • Array Numerical results will be rounded down to the nearest integer. Non-stringifiable data will be lost on save/load, more info here: JSON.stringify .
✓	✓	Control Variables (Add)	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) + value);</code>	<code>// +10 to variable ID 1 \$gameVariables.setValue(1, \$gameVariables.value(1) + 10);</code>	Number variableId - ID of variable to change. Number value - Operand (can be negative)	Also works for strings, e.g. • "cheese" + "cake" = "cheesecake"
✓	✓	Control Variables (Sub)	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) - value);</code>	<code>// -10 to variable ID 1 \$gameVariables.setValue(1, \$gameVariables.value(1) - 10);</code>		-
✓	✓	Control Variables (Mul)	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) * value);</code>	<code>// Multiply variable ID 1 by 5 \$gameVariables.setValue(1, \$gameVariables.value(1) * 5);</code>		-
✓	✓	Control Variables (Div)	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) / value);</code>	<code>// Divide variable ID 1 by 2 \$gameVariables.setValue(1, \$gameVariables.value(1) / 2);</code>		In JavaScript, div by 0 returns the special value <i>Infinity</i> .
✓	✓	Control Variables (Mod)	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId).mod(value));</code>	<code>// Store the +remainder after dividing variable ID 1 by 12 \$gameVariables.setValue(1, \$gameVariables.value(1).mod(12));</code>		For modular arithmetic. Common uses: • Get digits of a number, e.g. mod 10 to get units digit • Cyclic calculations, e.g. time of day
✓	✓	Control Switches Control Variables (Range)	<code>for (var n = id1; n <= id2; n++) { operation }</code>	<code>// Turn on all switches from 1~10 for (var n = 1; n <= 10; n++) { \$gameSwitches.setValue(n, true); }</code> <code>// Store the value 1 in all variables from 5~15 for (var n = 5; n <= 15; n++) { \$gameVariables.setValue(n, 1); }</code>	Number id1 - Lower end of ID range. Number id2 - Upper end of ID range. operation - Script to run for each ID value.	More info about for loops: for loop .

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Control Self Switch	\$gameSelfSwitches.setValue([mapId, eventId, switchId], value);	// In a Script command: turn on self-switch D for "This Event" \$gameSelfSwitches.setValue([this._mapId, this.eventId()], 'D'], true }; // Turn off self-switch A for event 5 on map 1 \$gameSelfSwitches.setValue([1, 5, 'A'], false);	Number mapId • ID of the event's map. Number eventId • ID of the event. String switchId • "A", "B", "C", or "D" Boolean value • true (ON) or false (OFF)	For the current map, use: \$gameMap.mapId() For the map of "This Event", use: this._mapId (Only in an interpreter context, e.g. a Script command.)
✓	✓	Control Timer	\$gameTimer.start(frames); \$gameTimer.stop();	// Start the timer with a duration of 10 seconds \$gameTimer.start(10 * 60);	Number frames • Starting timer duration, in frames.	-
		Flow Control				
✓	✓	Conditional Branch	-	-	-	Pure-script equivalent: if...else .
✓	✓	Loop	-	-	-	Pure-script equivalents: while , for .
✓	✓	Break Loop	interpreter.command113();	// In a Script command: break out of the current Loop block this.command113();	-	Pure-script equivalent: break .
✓	✓	Exit Event Processing	interpreter.command115();	// In a Script command: end the current event this.command115(); // Anywhere: end the current non-Parallel map event \$gameMap._interpreter.command115();	Game Interpreter interpreter • Target interpreter (this in a Script command).	-
☐	✓	Common Event	interpreter.command117([commonEventId]);	// MZ, anywhere: insert common event 5 into the map interpreter \$gameMap._interpreter.command117([5]);	Game Interpreter interpreter • Target interpreter (this in a Script command). Number commonEventId • ID of common event to insert.	Adds the common event's commands to the target interpreter. The common event's commands can refer to properties of "This Event".
✓	☐		interpreter._params = [commonEventId]; interpreter.command117();	// MV, in a Script command: insert CE with ID = variable 1's value this._params = [\$gameVariables.value(1)]; this.command117();		
✓	✓	Label	-	-	-	Pure-script equivalent: label .
☐	✓	Jump to Label	interpreter.command119([label]);	// MZ, in a Script command: jump to label Winner this.command119(['Winner']);	Game Interpreter interpreter • Target interpreter (this in a Script command). String label • Destination label name.	Pure-script equivalent: break , continue .
✓	☐		interpreter._params = [label]; interpreter.command119();	// MV, anywhere: make the map interpreter jump to label Secret var self = \$gameMap._interpreter; self._params = ["Secret"]; self.command119();		
✓	✓	Comment	-	-	-	Pure-script equivalent: comments .
Page 2		Movement, Character, Picture, Timing, Screen, Audio & Video				▲ Return to top ▲
		Movement				
✓	✓	Transfer Player	\$gamePlayer.reserveTransfer(mapId, x, y, direction, fadeType);	// Transfer to (5, 10) on map 4, no direction change, fade Black \$gamePlayer.reserveTransfer(4, 5, 10, 0, 0); // Transfer to (4, 15) on this map, face up, fade type None \$gamePlayer.reserveTransfer(\$gameMap.mapId(), 4, 15, 8, 2);	Number mapId • Destination map ID. Number x, y • Destination map coordinates. Number direction • 0 (no change), 2 (down), 4 (left), 6 (right), or 8 (up). Number fadeType • 0 (Black), 1 (White), or 2 (No Fade).	For variable coordinates, see the Set Vehicle Location airship example below.
✓	✓	Set Vehicle Location	vehicle.setLocation(mapId, x, y);	// Place boat at (7, 5) on this map \$gameMap.boat().setLocation(\$gameMap.mapId(), 7, 5); // Place ship at (20, 8) on map 2 \$gameMap.ship().setLocation(2, 20, 8); // Place airship at coordinates (map, x, y) = variables (1, 2, 3) var mapId = \$gameVariables.value(1); var x = \$gameVariables.value(2); var y = \$gameVariables.value(3); \$gameMap.airship().setLocation(mapId, x, y);	Game Vehicle vehicle • The vehicle to relocate. Number mapId • Destination map ID. Number x, y • Destination map coordinates.	There is only one copy of each vehicle.
✓	✓	Set Event Location (X, Y)	character.locate(x, y);	// Place map event 2 at (3, 4) \$gameMap.event(2).locate(3, 4); // In a Script command: place "This Event" at (5, 6) this.character(0).locate(5, 6); // In a move route: place "this character" at (7, 8) this.locate(7, 8);	Game Character character • The one to relocate. Number x, y • Destination map coordinates.	Works for all characters, not just events.
✓	✓	Set Event Location (Swap)	character1.swap(character2);	// Swap the locations of the player and map event 7 \$gamePlayer.swap(\$gameMap.event(7));	Game Character character1, character2 • The ones to swap locations.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Scroll Map	\$gameMap.startScroll(direction, distance, speed);	// Scroll down by 3 tiles at speed 4 \$gameMap.startScroll(2, 3, 4);	Number direction • 2 (down), 4 (left), 6 (right), or 8 (up). Number distance • Distance to scroll, in tiles. Number speed • A character move speed.	The default player walking speed is 4.
✓	✓	Set Movement Route (No Wait)	character.forceMoveRoute(moveRoute);	// MV, in a Script command // Fade out "This Event" by -10 opacity/frame, wait for completion var list = []; var char = this._character = this.character(0); var cOpa = Game_Character.ROUTE_CHANGE_OPACITY; for (var p = char.opacity(); p > 0; p -= 10) { list.push({ code: cOpa, parameters: [p] }); }	Game_Interpreter interpreter • Interpreter to pause (this in a Script command). Number characterId • -1 (player), 0 (This Event), or 1+ (map event ID). Game_Character character • The one to move. Object moveRoute • An object with the following properties: ◦ list: an array of move command objects (required). ◦ repeat: if true, the route will repeat. ◦ skippable: if true, skip failed move commands. • A move command has these properties: ◦ code: a unique move command code (required). ◦ parameters: an array of values for this command.	Use this one if you don't need to wait for the movement to finish before continuing. The last command in a move route should always have code ROUTE_END.
□	✓	Set Movement Route (Wait)	// MZ const self = interpreter; const char = self.character(self.characterId = characterId); char.forceMoveRoute(moveRoute); self.setWaitMode(route);	list.push({ code: cOpa, parameters: [p] }); list.push({ code: Game_Character.ROUTE_END }); var route = { list: list, repeat: false, skippable: false }; char.forceMoveRoute(route); this.setWaitMode(route);	Game_Character character • The one to move.	A character can only have one active move route at a time. Starting a new route will immediately end the current one. Events may have an auto move route to resume when their active route ends.
✓	□	Set Movement Route (Wait)	// MV var self = interpreter; var char = self._character = character; char.forceMoveRoute(moveRoute); self.setWaitMode(route);			Move routes process up to one command per frame. A Script move command can be used to perform multiple actions in a single command.
✓	✓	Get on/off Vehicle	\$gamePlayer.getOnOffVehicle();	-	-	See the <i>Move Commands</i> section for details.
		(Move commands)				
✓	✓	(End Route)	character.processRouteEnd();	// End the player's current move route \$gamePlayer.processRouteEnd();	Game_Character character • The target of the move route.	The last move command in any move route. Route code: • Game_Character.ROUTE_END
✓	✓	Move Down	character.moveStraight(direction);	// Make the player move 1 tile to the left \$gamePlayer.moveStraight(4);	Game_Character character • The one to move. Number direction • 2 (down), 4 (left), 6 (right), or 8 (up).	Route codes: • Game_Character.ROUTE_MOVE_DOWN • Game_Character.ROUTE_MOVE_LEFT • Game_Character.ROUTE_MOVE_RIGHT • Game_Character.ROUTE_MOVE_UP
✓	✓	Move Left				
✓	✓	Move Right				
✓	✓	Move Up				
✓	✓	Move Lower Left	character.moveDiagonally(horz, vert);	// Make map event 3 move 1 tile to the upper-left \$gameMap.event(3).moveDiagonally(4, 8);	Game_Character character • The one to move. Number horz • Horizontal move direction: 4 (left) or 6 (right). Number vert • Vertical move direction: 2 (down) or 8 (up).	Route codes: • Game_Character.ROUTE_MOVE_LOWER_L • Game_Character.ROUTE_MOVE_LOWER_R • Game_Character.ROUTE_MOVE_UPPER_L • Game_Character.ROUTE_MOVE_UPPER_R
✓	✓	Move Lower Right				
✓	✓	Move Upper Left				
✓	✓	Move Upper Right				
✓	✓	Move at Random	character.moveRandom();	// Make the player move 1 tile in a random available direction \$gamePlayer.moveRandom();	Game_Character character • The one to move.	Route code: • Game_Character.ROUTE_MOVE_RANDOM
✓	✓	Move toward Player	character.moveTowardPlayer();	// Make map event 5 move 1 tile toward the player \$gameMap.event(5).moveTowardPlayer();	Game_Character character • The one to move.	These commands do not use pathfinding!
✓	✓	Move away from Player	character.moveAwayFromPlayer();	// Make map event 5 move 1 tile away from the player \$gameMap.event(5).moveAwayFromPlayer();	Game_Character character • The one to move.	Route codes: • Game_Character.ROUTE_MOVE_TOWARD • Game_Character.ROUTE_MOVE_AWAY
✓	✓	1 Step Forward	character.moveForward();	// Make map event 12 move 1 tile in the direction it is facing \$gameMap.event(12).moveForward();	Game_Character character • The one to move.	Move backward will lock the character's facing direction for that move.
✓	✓	1 Step Backward	character.moveBackward();	// Make the player move 1 tile backwards \$gamePlayer.moveBackward();	Game_Character character • The one to move.	Route codes: • Game_Character.ROUTE_MOVE_FORWARD • Game_Character.ROUTE_MOVE_BACKWARD
✓	✓	Jump	character.jump(xPlus, yPlus);	// Make the player jump 5 tiles right \$gamePlayer.jump(5, 0); // Make map event 1 jump 2 tiles up \$gameMap.event(1).jump(0, -2);	Game_Character character • The one to move. Number xPlus • Horizontal jump distance, use negative to go left. Number yPlus • Vertical jump distance, use negative to go up.	Route code: • Game_Character.ROUTE_JUMP ◦ Parameters: [xPlus, yPlus]
✓	✓	Wait	character._waitCount = frames - 1;	// Make map event 20 wait for 60 frames \$gameMap.event(20)._waitCount = 59;	Game_Character character • The one to affect. Number frames • Wait duration.	Route code: • Game_Character.ROUTE_WAIT ◦ Parameters: [frames]
✓	✓	Turn Down	character.setDirection(direction);	// Make the player face right \$gamePlayer.setDirection(6);	Game_Character character • The one to move. Number direction • 2 (down), 4 (left), 6 (right), or 8 (up).	Route codes: • Game_Character.ROUTE_TURN_DOWN • Game_Character.ROUTE_TURN_LEFT • Game_Character.ROUTE_TURN_RIGHT • Game_Character.ROUTE_TURN_UP
✓	✓	Turn Left				
✓	✓	Turn Right				
✓	✓	Turn Up				
✓	✓	Turn 90° Right	character.turnRight90();	// Make the player turn 90° left \$gamePlayer.turnLeft90();	Game_Character character	Route codes: • Game_Character.ROUTE_TURN_90D_R • Game_Character.ROUTE_TURN_90D_L
✓	✓	Turn 90° Left	character.turnLeft90();			

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Turn 180°	<code>character.turn180();</code>	// Make map event 2 turn 90° left or right (random) <code>SgameMap.event(2).turnRightOrLeft90();</code>	The one to move.	<code>Game_Character.ROUTE_TURN_180_L</code> <code>Game_Character.ROUTE_TURN_180D</code> <code>Game_Character.ROUTE_TURN_90D_R_L</code>
✓	✓	Turn 90° Right or Left	<code>character.turnRightOrLeft90();</code>			
✓	✓	Turn at Random	<code>character.turnRandom();</code>	// Make the player face a random direction <code>SgamePlayer.turnRandom();</code>	Game_Character character The one to move.	Route code: <code>Game_Character.ROUTE_TURN_RANDOM</code>
✓	✓	Turn toward Player	<code>character.turnTowardPlayer();</code>	// Make map event 15 face the player <code>SgameMap.event(15).turnTowardPlayer();</code>	Game_Character character The one to move.	Route codes: <code>Game_Character.ROUTE_TURN_TOWARD</code> <code>Game_Character.ROUTE_TURN_AWAY</code>
✓	✓	Turn away from Player	<code>character.turnAwayFromPlayer();</code>	// Make map event 15 face away from the player <code>SgameMap.event(15).turnAwayFromPlayer();</code>		
✓	✓	Switch ON	<code>SgameSwitches.setValue(switchId, true);</code>	// Turn on switch ID 1 <code>SgameSwitches.setValue(1, true);</code>	Number switchId ID of switch to change.	Route code: <code>Game_Character.ROUTE_SWITCH_ON</code> - Parameters: [switchId]
✓	✓	Switch OFF	<code>SgameSwitches.setValue(switchId, false);</code>	// Turn off switch ID 5 <code>SgameSwitches.setValue(5, false);</code>		Route code: <code>Game_Character.ROUTE_SWITCH_OFF</code> - Parameters: [switchId]
✓	✓	Change Speed	<code>character.setMoveSpeed(speed);</code>	// Set the player's walking move speed to 3.5 <code>SgamePlayer.setMoveSpeed(3.5);</code>	Game_Character character - The one to affect. Number speed - New move speed; can be a decimal, e.g. 3.5.	When dashing, the player moves at walking speed +1. Route code: <code>Game_Character.ROUTE_CHANGE_SPEED</code> - Parameters: [speed]
✓	✓	Change Frequency	<code>character.setMoveFrequency(frequency);</code>	// Set the move frequency of map event 4 to Highest (5) <code>SgameMap.event(4).setMoveFrequency(5);</code>	Game_Character character - The one to affect. Number frequency - New frequency value; typically an integer 1–5.	Frequency affects time between moves for map events. Route code: <code>Game_Character.ROUTE_CHANGE_FREQ</code> - Parameters: [frequency]
✓	✓	Walking Animation ON	<code>character.setWalkAnime(true);</code>	// Disable walking animation for the player <code>SgamePlayer.setWalkAnime(false);</code>		Walk/step animation rate is based on move speed. Stepping animation happens even if you're not moving. Walking animates +50% faster than stepping.
✓	✓	Walking Animation OFF	<code>character.setWalkAnime(false);</code>			
✓	✓	Stepping Animation ON	<code>character.setStepAnime(true);</code>	// Enable stepping animation for the player <code>SgamePlayer.setStepAnime(true);</code>		Route codes: <code>Game_Character.ROUTE_WALK_ANIME_ON</code> <code>Game_Character.ROUTE_WALK_ANIME_OFF</code> <code>Game_Character.ROUTE_STEP_ANIME_ON</code> <code>Game_Character.ROUTE_STEP_ANIME_OFF</code>
✓	✓	Stepping Animation OFF	<code>character.setStepAnime(false);</code>			Route code: <code>Game_Character.ROUTE_DIR_FIX_ON</code>
✓	✓	Direction Fix ON	<code>character.setDirectionFix(true);</code>	// Lock the player's direction until further notice <code>SgamePlayer.setDirectionFix(true);</code>	Game_Character character The one to affect.	Route code: <code>Game_Character.ROUTE_DIR_FIX_OFF</code>
✓	✓	Direction Fix OFF	<code>character.setDirectionFix(false);</code>	// Unlock the player's direction <code>SgamePlayer.setDirectionFix(false);</code>		Route code: <code>Game_Character.ROUTE_THROUGH_ON</code>
✓	✓	Through ON	<code>character.setThrough(true);</code>	// Enable through for map event 10, i.e. ignore collisions <code>SgameMap.event(10).setThrough(true);</code>		Route code: <code>Game_Character.ROUTE_THROUGH_OFF</code>
✓	✓	Through OFF	<code>character.setThrough(false);</code>	// Disable through for map event 10 <code>SgameMap.event(10).setThrough(false);</code>		This will not change the character's opacity value.
✓	✓	Transparent ON	<code>character.setTransparent(true);</code>	// Make map event 4 invisible until further notice <code>SgameMap.event(4).setTransparent(true);</code>		Route codes: <code>Game_Character.ROUTE_TRANSPARENT_ON</code> <code>Game_Character.ROUTE_TRANSPARENT_OFF</code>
✓	✓	Transparent OFF	<code>character.setTransparent(false);</code>	// Make map event 4 visible again <code>SgameMap.event(4).setTransparent(false);</code>		
✓	✓	Change Image	<code>character.setImage(name, index);</code>	// Change map event 7 to use index 0 of sheet Actor1_1.png <code>SgameMap.event(7).setImage('Actor1_1', 0);</code>	Game_Character character - The one to affect. String name - Name of the spritesheet to use. Number index - Index to use (0 for top-left).	Route code: <code>Game_Character.ROUTE_CHANGE_IMAGE</code> - Parameters: [name, index]
✓	✓	Change Opacity	<code>character.setOpacity(opacity);</code>	// Set the player to full opacity (0% transparent) <code>SgamePlayer.setOpacity(255);</code> // Set event 5 to 128 opacity (~50% transparent) <code>SgameMap.event(5).setOpacity(128);</code>	Game_Character character - The one to affect. Number opacity 0 to 255, where 0 is fully transparent.	Route code: <code>Game_Character.ROUTE_CHANGE_OPACITY</code> - Parameters: [opacity]
✓	✓	Change Blend Mode	<code>character.setBlendMode(blendMode);</code>	// Set map event 11 to use Additive blend mode <code>SgameMap.event(11).setBlendMode(1);</code>	Game_Character character - The one to affect. Number blendMode 0 (Normal), 1 (Additive), 2 (Multiply), or 3 (Screen).	Route code: <code>Game_Character.ROUTE_CHANGE_BLEND_MODE</code> - Parameters: [blendMode]
✓	✓	Play SE	<code>AudioManager.playSe(se);</code>	// Play SE Attack1 at 20% volume, 100% pitch, and 0 panning <code>AudioManager.playSe({ name: 'Attack1', volume: 20, pitch: 100 });</code>	Object se - An object with properties: name: audio filename, without extension. volume: use 100 for 100%. pitch: use 100 for 100%. pan: -100 (left) to 100 (right).	The pan property is optional (0 if omitted). Route code: <code>Game_Character.ROUTE_PLAY_SE</code> - Parameters: [se]

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Script	-	-	<u>String script</u> A string to evaluate as code, e.g. 'this.turnTowardCharacter(\$gameMap.event(5));'	This move command can be used to include script calls in a script-constructed move route. Runs in the context of the character being moved. Route code: - Game_Character.ROUTE_SCRIPT - Parameters: [script]
		Character				
✓	✓	Change Transparency	\$gamePlayer.setTransparent(value);	// Make the player invisible until further notice \$gamePlayer.setTransparent(true);	<u>Boolean value</u> true (invisible) or false (visible).	This will not change the player's opacity value.
✓	✓	Change Player Followers (Show)	\$gamePlayer.showFollowers(); \$gamePlayer.refresh();			Makes followers invisible. Works independently of <i>setTransparent</i> .
✓	✓	Change Player Followers (Hide)	\$gamePlayer.hideFollowers(); \$gamePlayer.refresh();	-	-	Each frame, followers update properties to match \$gamePlayer: - move speed - opacity - blend mode - walk/step animation on/off - direction fix on/off - transparent on/off
✓	✓	Gather Followers (Wait)	\$gamePlayer.gatherFollowers(); interpreter .setWaitMode('gather');	// In a Script command \$gamePlayer.gatherFollowers(); this.setWaitMode('gather');	<u>Game_Interpreter interpreter</u> Interpreter to pause (<i>this</i> in a Script command).	Gathering will stop when all followers are on the same tile as the player.
✓	✓	Gather Followers (No Wait)	\$gamePlayer.gatherFollowers();	-	-	
☐	✓	Show Animation (No Wait)	\$gameTemp.requestAnimation([character], animationId , mirror);	// Show animation 15 on map event 20, no waiting \$gameTemp.requestAnimation([\$gameMap.event(20)], 15);	<u>Game_Interpreter interpreter</u> - Interpreter to pause (<i>this</i> in a Script command). <u>Game_Character character</u> - Animation target. <u>Number characterId</u> -1 (player), 0 (This Event), or 1+ (map event ID). <u>Number animationId</u> - ID of the animation to show. <u>Boolean mirror</u> - If true, mirror the animation.	
✓	☐		character .requestAnimation(animationId);	// Show animation 15 on map event 20, no waiting \$gameMap.event(20).requestAnimation(15);		
☐	✓	Show Animation (Wait)	const self = interpreter ; const char = self.character(self.characterId = characterId); \$gameTemp.requestAnimation([char], animationId , mirror); self.setWaitMode('animation');	// In a Script command: show animation 5 on the player and wait const char = this.character(this.characterId = -1); \$gameTemp.requestAnimation([char], 5); this.setWaitMode('animation');		
✓	☐		var self = interpreter ; self.character = character ; self.character.requestAnimation(animationId); self.setWaitMode('animation');	// In a Script command: show animation 5 on the player and wait this._character = \$gamePlayer; this._character.requestAnimation(5); this.setWaitMode('animation');		
☐	✓	Show Balloon (No Wait)	\$gameTemp.requestBalloon(character , balloonId);	// Show Music Note balloon on the player, no waiting \$gameTemp.requestBalloon(\$gamePlayer, 3);	<u>Game_Interpreter interpreter</u> - Interpreter to pause (<i>this</i> in a Script command). <u>Game_Character character</u> - Balloon target. <u>Number characterId</u> -1 (player), 0 (This Event), or 1+ (map event ID). <u>Number balloonId</u> - ID of balloon to show, see notes ->	Balloon IDs: 1 = Exclamation 2 = Question 3 = Music Note 4 = Heart 5 = Anger 6 = Sweat 7 = Frustration 8 = Silence 9 = Light Bulb 10 = Zzz 11~15 = User Defined
✓	☐		character .requestBalloon(balloonId);	// Show Music Note balloon on the player, no waiting \$gamePlayer.requestBalloon(3);		
☐	✓	Show Balloon (Wait)	const self = interpreter ; const char = self.character(self.characterId = characterId); \$gameTemp.requestBalloon(char, balloonId); self.setWaitMode('balloon');	// In a Script command: show Zzz balloon on map event 1 and wait const char = this.character(this.characterId = 1); \$gameTemp.requestBalloon(char, 10); this.setWaitMode('balloon');		
✓	☐		var self = interpreter ; self.character = character ; self.character.requestBalloon(balloonId); self.setWaitMode('balloon');	// In a Script command: show Zzz balloon on map event 1 and wait this._character = \$gameMap.event(1); this._character.requestBalloon(10); this.setWaitMode('balloon');		
✓	✓	Erase Event	\$gameMap.eraseEvent(eventId);	// Erase map event 8	<u>Number eventId</u> - ID of the event to erase. <u>Game_Event event</u> - Event to erase.	An erased event is always on page -1 (inactive).
✓	✓		event .erase();	\$gameMap.eraseEvent(8);		Erased events will return after going to a different map.
		Picture				
✓	✓	Show Picture	\$gameScreen.showPicture(pictureId , name , origin , x , y , scaleX , scaleY , opacity , blendMode);	// Show picture 1, <i>pic.png</i> , at 100% scale centred on (50, 50) \$gameScreen.showPicture(1, 'pic', 1, 50, 50, 100, 100, 255, 0);	<u>Number pictureId</u> - ID of the picture 1~100. <u>Game_Picture picture</u> - Picture object to change. <u>String name</u> - Filename, without extension. <u>Number origin</u> - Anchor type: 0 (Upper Left) or 1 (Center).	Map and battle pictures are stored separately: picture ID 1 is one picture on the map, and a different picture in battle.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Show Picture				
✓	✓		<code>picture.show(name, origin, x, y, scaleX, scaleY, opacity, blendMode);</code>	<code>// Show picture 2, dog.png, 50% scale in the top-left of the screen \$gameScreen.picture(2).show('dog', 0, 0, 0, 50, 50, 255, 0);</code>	Number x, y - Anchor screen coordinates (px). Number scaleX, scaleY - Scale multipliers: use 100 for 100% (original size). Number opacity 0 to 255, where 0 is 100% transparent. Number blendMode 0 (Normal), 1 (Additive), 2 (Multiply), or 3 (Screen).	All battle pictures are erased at the start of each battle.
✓	✓	Move Picture	<code>\$gameScreen.movePicture(pictureId, origin, x, y, scaleX, scaleY, opacity, blendMode, frames, easing);</code>	<code>// Move picture 1 to center on (200, 150), 0 opacity, over 60 frames \$gameScreen.movePicture(1, 1, 200, 150, 100, 100, 0, 0, 60);</code>	Number pictureId - ID of the picture 1~100. Game_Picture picture - Picture object to move. Number origin - New anchor type: 0 (Upper Left) or 1 (Center). Number x, y - New anchor screen coordinates (px). Number scaleX, scaleY - New scale multipliers: use 100 for 100%. Number opacity 0 to 255, where 0 is 100% transparent. Number blendMode 0 (Normal), 1 (Additive), 2 (Multiply), or 3 (Screen).	origin, blendMode, and easing will apply as soon as the movement starts. Other changes will be applied smoothly over the duration.
✓	✓		<code>picture.move(origin, x, y, scaleX, scaleY, opacity, blendMode, frames, easing);</code>	<code>// Move picture 2 instantly to the top-left of screen at 100% scale \$gameScreen.picture(2).move(0, 0, 0, 100, 100, 255, 0, 1);</code>	Number frames - Movement duration, in frames. Number easing [MZ only] (optional) 0 (Normal), 1 (Linear), 2 (Smooth), or 3 (Instant).	Cross-fade can be achieved with multiple pictures on top of one another.
✓	✓	Rotate Picture	<code>\$gameScreen.rotatePicture(pictureId, speed);</code>	<code>// Make picture 1 start rotating at speed 3 \$gameScreen.rotatePicture(1, 3);</code>	Number pictureId - ID of the picture 1~100. Number speed - Rotation speed 4 is +2° clockwise per frame.	Rotation will occur about the picture's origin/anchor.
✓	✓		<code>picture.rotate(speed);</code>	<code>// Make picture 2 start rotating at speed -2.5 \$gameScreen.picture(2).rotate(-4.5);</code>	Game_Picture picture - Picture object to rotate. Number pictureId - ID of the picture 1~100.	
✓	✓	Tint Picture	<code>\$gameScreen.tintPicture(pictureId, [red, green, blue, grey], frames);</code>	<code>// Tint picture 1 fully grey over 300 frames \$gameScreen.tintPicture(1, [0, 0, 0, 255], 300);</code>	Game_Picture picture - Picture object to tint. Number red, green, blue - Values from -255 to 255.	-
✓	✓		<code>picture.tint([red, green, blue, grey], frames);</code>	<code>// Tint picture 2 yellow (red + green) over 60 frames \$gameScreen.picture(2).tint([80, 80, 0, 0], 60);</code>	Number grey 0 (no change) to 255 (greyscale). Number frames - Tint will gradually be applied over this duration.	-
✓	✓	Erase Picture	<code>\$gameScreen.erasePicture(pictureId);</code>	<code>// Erase picture 1 \$gameScreen.erasePicture(1);</code>	Number pictureId ID of the picture 1~100.	-
		Timing				
✓	✓	Wait	<code>interpreter.wait(frames);</code>	<code>// In a Script command: wait for 60~300 frames (random) this.wait(60 + Math.randomInt(241));</code>	Game_Interpreter interpreter - Interpreter to pause (<i>this</i> in a Script command). Number frames - Wait duration, in frames.	This wait time applies between event commands. If you want to wait in the middle of a Script command, split it into 2+ commands so the wait time can apply.
		Screen				
✓	✓	Fadeout Screen	<code>\$gameScreen.startFadeOut(frames); interpreter.wait(frames);</code>	<code>// In a Script command: fade out over 60 frames \$gameScreen.startFadeOut(60); this.wait(60);</code>	Number frames How long it takes to fade in/out, in frames.	Default fade time is 24 frames.
✓	✓	Fadein Screen	<code>\$gameScreen.startFadeIn(frames); interpreter.wait(frames);</code>	<code>// In a Script command: make the screen fade in over 60 frames \$gameScreen.startFadeIn(60); this.wait(60);</code>	Game_Interpreter interpreter Interpreter to pause (<i>this</i> in a Script command).	
✓	✓	Tint Screen	<code>\$gameScreen.startTint([red, green, blue, grey], frames);</code>	<code>// Tint the screen magenta (red + blue) over 200 frames \$gameScreen.startTint([90, 0, 90, 0], 180);</code>	Number red, green, blue - Values from -255 to 255. Number grey 0 (no change) to 255 (full greyscale). Number frames - Tint will gradually be applied over this duration.	-
✓	✓	Flash Screen	<code>\$gameScreen.startFlash([red, green, blue, intensity], frames);</code>	<code>// Apply a 120-frame black screen flash at full intensity \$gameScreen.startFlash([0, 0, 0, 255], 120);</code>	Number red, green, blue - Values from 0 to 255. Number intensity - Starting flash opacity: 0 (invisible) to 255 (opaque). Number frames - Flash duration, in frames.	-

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Shake Screen	\$gameScreen.startShake(power , speed , frames);	// Apply a 60-frame shake at speed 3, power 5 \$gameScreen.startShake(5, 3, 60);	Number power • Shake power, typically 1~9. Number speed • Shake speed, typically 1~9. Number frames • Shake duration, in frames.	-
✓	✓	Set Weather Effect	\$gameScreen.changeWeather(type , power , frames);	// Change the weather to rain, power 4, over 300 frames \$gameScreen.changeWeather('rain', 4, 300); // Instantly disable weather effects \$gameScreen.changeWeather('none', 0, 1);	String type • "none", "rain", "storm", or "snow". Number power • Weather intensity, typically 1~9. Number frames • Weather will gradually change over this duration.	-
		Audio & Video				
✓	✓	(Audio Object)	{ name: filename , volume: volume , pitch: pitch , pan: pan }	(see below)	String filename • Audio filename, without extension. Number volume • Volume multiplier, use 100 for 100%. Number pitch • Pitch multiplier, use 100 for 100%. Number pan (optional) • Left/right panning, from -100 to 100.	pan is optional; if omitted, it will default to 0.
✓	✓	Play BGM	AudioManager.playBgm(bgm , pos);	// Play BGM <i>bgm1.ogg</i> with 20% volume, 100% pitch, 0 pan AudioManager.playBgm({ name: 'bgm1', volume: 20, pitch: 100 });	(Audio Object) bgm • Background music object. Number pos (optional) • Start time, in seconds.	pos is optional; if omitted, it will default to 0.
✓	✓	Fadeout BGM	AudioManager.fadeOutBgm(seconds);	// Fade out the current BGM over 5 seconds AudioManager.fadeOutBgm(5);	Number seconds • Fadeout duration, in seconds.	-
✓	✓	Save BGM	\$gameSystem.saveBgm();	-	-	Saves the current BGM. Pairs with <i>Replay BGM</i> .
✓	✓	Replay BGM	\$gameSystem.replayBgm();	-	-	Replays the saved BGM. Pairs with <i>Save BGM</i> .
✓	✓	Play BGS	AudioManager.playBgs(bgs);	// Play BGS <i>raindrops.ogg</i> with 25% volume, 80% pitch, 0 pan AudioManager.playBgs({ name: 'raindrops', volume: 25, pitch: 100 });	(Audio Object) bgs • Background sound object. Number pos (optional) • Start time, in seconds from the start of the track.	pos is optional; if omitted, it will default to 0.
✓	✓	Fadeout BGS	AudioManager.fadeOutBgs(seconds);	// Fade out the current BGS over 3 seconds AudioManager.fadeOutBgs(3);	Number seconds • Fadeout duration, in seconds.	-
✓	✓	Play ME	AudioManager.playMe(me);	// Play ME <i>fanfare.ogg</i> with 20% volume, 120% pitch, 0 pan AudioManager.playMe({ name: 'fanfare', volume: 20, pitch: 120 });	(Audio Object) me • Music effect sound object.	-
✓	✓	Play SE	AudioManager.playSe(se , pos);	// Play SE <i>beep.ogg</i> at 5% volume, 100% pitch, panned 50% left AudioManager.playSe({ name: 'beep', volume: 5, pitch: 100, pan: -50 });	(Audio Object) se • Sound effect object. Number pos (optional) • Start time, in seconds from the start of the track.	-
✓	✓	Stop SE	AudioManager.stopSe();	-	-	Stops all currently-playing SE audio channels.
□	✓	Play Movie	interpreter.command261([movieName]);	// In a Script command: play <i>sample.webm</i> this.command261(['sample']);	Game_Interpreter interpreter • Interpreter to pause (<i>this</i> in a Script command). String movieName • Video filename, without extension.	-
✓	□		interpreter._params = [movieName]; interpreter.command261();	// In a Script command: play <i>sample.webm</i> this._params = ['sample']; this.command261();		
Page 3		Scene Control, System Settings, Map, Battle, Advanced				▲ Return to top ▲
		Scene Control	<i>Note: the core scripts are not designed to allow scene changes during battle.</i>			
✓	✓	Battle Processing	BattleManager.setup(troopId , canEscape , canLose); \$gamePlayer.makeEncounterCount(); SceneManager.push(Scene_Battle);	// Start battle with troop ID 20, can escape, lose = game over BattleManager.setup(20, true, false); \$gamePlayer.makeEncounterCount(); SceneManager.push(Scene_Battle);	Number troopId • ID of the troop to fight. Boolean canEscape • If true, the party can escape	If canLose is false, a defeat will result in a game over.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Battle Processing (Result Handlers)	BattleManager.setup(troopId , canEscape , canLose); BattleManager.setEventCallback(callback); SceneManager.makeEncounterCount(); SceneManager.push(Scene_Battle);	// Start battle with troop 3, no escape; on lose, turn on switch 5 BattleManager.setup(3, false, true); BattleManager.setEventCallback(function(result) { if (result === 2) \$gameSwitches.setValue(5, true); }); SceneManager.makeEncounterCount(); SceneManager.push(Scene_Battle);	• If true, the party can escape. Boolean canLose • If true, the party can continue after losing. Function callback • Runs when the battle ends. 1 argument: result . Number result • 0 (victory), 1 (escape/abort), or 2 (defeat).	A result of 1 is: • Escape if <code>BattleManager._escaped</code> is true; • Abort otherwise.
✓	✓	Shop Processing	SceneManager.push(Scene_Shop); SceneManager.prepareNextScene(goods , purchaseOnly);	// Open a shop selling armors 1, 2, 3 at standard price (buy only) SceneManager.push(Scene_Shop); SceneManager.prepareNextScene([[2, 1, 0], [2, 2, 0], [2, 3, 0]], true);	Number[][] goods • An array of [itemType , itemId , priceType , price]: ◦ itemType : 0 (Item), 1 (Weapon), or 2 (Armor). ◦ itemId : ID of the item. ◦ priceType : 0 (Standard) or 1 (Specific). ◦ price : item price; ignored if priceType = 0. Boolean purchaseOnly • If true, items cannot be sold to this shop.	-
✓	✓	Name Input Processing	SceneManager.push(Scene_Name); SceneManager.prepareNextScene(actorId , maxLength);	// Get the player to rename the party leader, up to 12 characters SceneManager.push(Scene_Name); SceneManager.prepareNextScene(\$gameParty.members()[0].actorId(), 12);	Number actorId • ID of the actor to rename. Number maxLength • Maximum character count for the new name.	-
✓	✓	Open Menu Screen	SceneManager.push(Scene_Menu); Window_MenuCommand.initCommandPosition();	-	-	Open the pause menu.
✓	✓	Open Save Screen	SceneManager.push(Scene_Save);	-	-	Open the save screen.
✓	✓	Game Over	SceneManager.goto(Scene_Gameover);	-	-	Cause a game over.
✓	✓	Return to Title Screen	SceneManager.goto(Scene_Title);	-	-	Return to the title screen.
		System Settings				
✓	✓	(Audio Object)	{ name: filename , volume: volume , pitch: pitch , pan: pan }	(see below)	String filename • Audio filename, without extension. Number volume • Volume multiplier, use 100 for 100%. Number pitch • Pitch multiplier, use 100 for 100%. Number pan (optional) • Left/right panning, from -100 to 100.	pan is optional; if omitted, it will default to 0.
✓	✓	Change Battle BGM	\$gameSystem.setBattleBgm(bgm);	// Set the battle BGM to <i>Battle1.ogg</i> at 20% volume, 100% pitch \$gameSystem.setBattleBgm({ name: 'Battle1', volume: 20, pitch: 100 });	(Audio Object) bgm • Background music object.	-
✓	✓	Change Victory ME	\$gameSystem.setVictoryMe(me);	// Set the victory ME to <i>Victory1</i> at 20% volume, 100% pitch \$gameSystem.setVictoryMe({ name: 'Victory1', volume: 20, pitch: 100 });	(Audio Object) me • Music effect object.	-
✓	✓	Change Defeat ME	\$gameSystem.setDefeatMe(me);	// Set the defeat ME to <i>Defeat1</i> at 20% volume, 100% pitch \$gameSystem.setDefeatMe({ name: 'Defeat1', volume: 20, pitch: 100 });		-
✓	✓	Change Vehicle BGM	vehicle .setBgm(bgm);	// Set the boat BGM to <i>Boat1</i> at 20% volume, 90% pitch, 0 pan \$gameMap.boat().setBgm({ name: 'Boat1', volume: 20, pitch: 90 });	Game_Vehicle vehicle • The vehicle to affect. (Audio Object) bgm • Background music object.	-
✓	✓	Change Save Access (Enable)	\$gameSystem.enableSave();	-	-	When disabled, the Save option on the pause menu will be locked. It also disables MZ's autosave.
✓	✓	Change Save Access (Disable)	\$gameSystem.disableSave();	-	-	The player can still save via Open Save Screen etc.
✓	✓	Change Menu Access (Enable)	\$gameSystem.enableMenu();	-	-	When disabled, pressing the menu button on the map will not

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Change Menu Access (Disable)	\$gameSystem.disableMenu();	-	-	open the pause menu.
✓	✓	Change Encounter (Enable)	\$gameSystem.enableEncounter(); \$gamePlayer.makeEncounterCount();	-	-	When disabled, random encounters will not occur.
✓	✓	Change Encounter (Disable)	\$gameSystem.disableEncounter(); \$gamePlayer.makeEncounterCount();	-	-	<i>makeEncounterCount</i> resets the step counter for random encounters.
✓	✓	Change Formation Access (Enable)	\$gameSystem.enableFormation();	-	-	When disabled, the Formation option on the pause menu will be locked.
✓	✓	Change Formation Access (Disable)	\$gameSystem.disableFormation();	-	-	
✓	✓	Change Window Color	\$gameSystem.setWindowTone([red, green, blue]);	// Set the game window tone to a green/blue colour \$gameSystem.setWindowTone([-50, 120, 120]);	Number red, green, blue • Values from -255 to 255.	Changes the tone of the window skin in-game.
✓	✓	Change Actor Images (Map Character)	actor.setCharacterImage(filename, index); \$gamePlayer.refresh();	// Change the party leader's images: // Map character: img/characters/Hero.png, index 0 (top-left) // Face: img/faces/Hero.png, index 3 (face #4)	Game_Actor actor • The one to affect. String filename • Spritesheet filename, without extension.	For walking on the map.
✓	✓	Change Actor Images (Face)	actor.setFacelImage(filename, index); \$gamePlayer.refresh();	// Battler: img/sv_actors/Hero.png var actor = \$gameParty.members()[0]; actor.setCharacterImage('Hero', 0);	Number index • Position of the image in its file, typically 0~7.	For menus etc.
✓	✓	Change Actor Images (SV Battler)	actor.setBattlerImage(filename); \$gamePlayer.refresh();	actor.setFacelImage('Hero', 3); actor.setBattlerImage('Hero'); \$gamePlayer.refresh();		For side-view battles (unnecessary if using front-view).
✓	✓	Change Vehicle Images	vehicle.setImage(filename, index);	// Set the airship to use character #3 (i.e. index 2) from Bird.png \$gameMap.airship().setImage('Bird', 2);	Game_Vehicle vehicle • The vehicle to affect. String filename • Spritesheet filename, without extension. Number index • Position of the character in its file, 0~7.	-
		Map				
✓	✓	Change Map Name Display (Enable)	\$gameMap.enableNameDisplay();	-	-	Changes whether the map's display name is shown when arriving on a new map.
✓	✓	Change Map Name Display (Disable)	\$gameMap.disableNameDisplay();	-	-	
✓	✓	Change Tileset	\$gameMap.changeTileset(tilesetId);	// Change to tileset ID 2 \$gameMap.changeTileset(2);	Number tilesetId • ID of the new tileset.	This will not wait for the tileset images to load first.
✓	✓	Change Battle Background	\$gameMap.changeBattleback(back1, back2);	// Change this map's battle BG to Wood2.png and Bridge.png \$gameMap.changeBattleback('Wood2', 'Bridge'); // Remove this map's specific battle background \$gameMap.changeBackground(null, null);	String back1, back2 • Battleback image filenames, without extension.	This change will be forgotten when the player transfers to a different map.
✓	✓	Change Parallax	\$gameMap.changeParallax(name, loopX, loopY, speedX, speedY);	// Set the parallax to Stars.png, X & Y loop, auto-scroll X speed 1 \$gameMap.changeParallax('Stars', true, true, 1, 0);	String name • Parallax image filename, without extension. Boolean loopX, loopY • If true, the parallax will loop in that direction. Number speedX, speedY • Auto-scroll speeds.	This change will be forgotten when the player transfers to a different map. At 48 px tile size, auto-scroll speed 1 takes 96 frames to move 1 tile.
✓	✓	Get Location Info (Terrain Tag)	\$gameMap.terrainTag(x, y)	// Store the terrain tag for tile (2, 3) in variable ID 1 \$gameVariables.setValue(1, \$gameMap.terrainTag(2, 3));		
✓	✓	Get Location Info (Event ID)	\$gameMap.eventIdXy(x, y)	// Get the lowest event ID of events at x = variable 3, y = variable 4 \$gameMap.eventIdXy(\$gameVariables.value(3), \$gameVariables.value(4))	Number x, y • Tile coordinates to check.	
✓	✓	Get Location Info (Tile ID)	\$gameMap.tileId(x, y, z)	// Get the ID of the layer #1 (z = 0) tile at the player's coordinates \$gameMap.tileId(\$gamePlayer.x, \$gamePlayer.y, 0)	Number z • Tile layer to check, 0~3.	There are shortcuts for getting the region ID or terrain tag at a character's current position: • <i>character.regionId()</i> • <i>character.terrainTag()</i>
✓	✓	Get Location Info (Region ID)	\$gameMap.regionId(x, y)	// Store the region ID of the tile in front of the player in variable 5 var d = \$gamePlayer.direction(); var x = \$gameMap.roundXWithDirection(\$gamePlayer.x, d); var y = \$gameMap.roundYWithDirection(\$gamePlayer.y, d); \$gameVariables.setValue(5, \$gameMap.regionId(x, y));		
		Battle				
✓	✓	Change Enemy HP	interpreter.changeHp(target, value, allowKO);	// Deal 100 damage to enemy #1 (index 0), allow KO this.changeHp(\$gameTroop.members()[0], -100, true);	Game_Interpreter interpreter • Interpreter reference (this in a Script command). Game_Battler battler • The one to gain/lose HP.	Has no effect if the target is dead. The collapse effect will start if the target dies in battle.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Change Enemy HP	<code>Game_Interpreter.prototype.changeHp.call(this, target, value, allowKO);</code>	<code>// Heal 50 HP for enemy #2 (index 1) Game_Interpreter.prototype.changeHp.call(this, \$gameTroop.members()[1], 50, false);</code>	Number value - Amount of HP to gain (can be negative). Boolean allowKO - If true, the target can die.	The collapse effect will start if the target dies in battle. Works for actors or enemies.
✓	✓	Change Enemy MP	<code>battler.gainMp(value);</code>	<code>// Make the enemy troop leader (index 0) lose 20 MP \$gameTroop.members()[0].gainMp(-20); // Restore +15 MP for enemy #2 (index 1) \$gameTroop.members()[1].gainMp(15);</code>	Game_Battler battler - The one to gain/lose MP. Number value - Amount of MP to gain (can be negative).	Works for actors or enemies.
✓	✓		Change Enemy TP	<code>battler.gainTp(value);</code>	<code>// Make enemy #2 (index 1) gain 100 TP \$gameTroop.members()[1].gainTp(100); // Make enemy #1 (index 0) lose 10 TP \$gameTroop.members()[0].gainTp(-10);</code>	Game_Battler battler - The one to gain/lose TP. Number value - Amount of TP to gain (can be negative).
✓	✓	Change Enemy State (Add)		<code>battler.addState(stateId); var target = battler; var isDead = target.isDead(); target.addState(stateId); if (!isDead && target.isDead()) target.performCollapse();</code>	<code>// Add state ID 1 to enemy #1 (index 0) and check for collapse var target = \$gameTroop.members()[0]; var isDead = target.isDead(); target.addState(1); if (!isDead && target.isDead()) target.performCollapse();</code>	Game_Battler battler - The one to gain/lose the state. Number stateId - ID of the state to add/remove.
✓	✓		Change Enemy State (Remove)	<code>battler.removeState(stateId); var target = battler; var isDead = target.isDead(); target.removeState(stateId); if (!isDead && target.isDead()) target.performCollapse();</code>	<code>// Remove state ID 6 from enemy #2 (index 1), no collapse check \$gameTroop.members()[1].removeState(6);</code>	
✓	✓	Enemy Recover All (One)	<code>battler.recoverAll();</code>	<code>// Make enemy #1 (index 0) fully recover \$gameTroop.members()[0].recoverAll();</code>	Game_Battler battler The one to be fully recovered.	
✓	✓	Enemy Recover All (Entire Troop)	<code>\$gameTroop.members().forEach(function(enemy) { enemy.recoverAll(); });</code>	-	-	Removes all states and sets HP & MP to maximum.
✓	✓	Enemy Appear	<code>battler.appear(); \$gameTroop.makeUniqueNames();</code>	<code>// Make enemy #2 (index 1) appear \$gameTroop.members()[1].appear(); \$gameTroop.makeUniqueNames();</code>	Game_Battler battler The one to appear.	This is typically used for enemies marked <i>Appear Mid-Battle</i> in their troop. It can also be used to bring back a battler who escaped earlier this battle.
✓	✓	Enemy Transform	<code>enemy.transform(newEnemyId); \$gameTroop.makeUniqueNames();</code>	<code>// Transform enemy #1 (index 0) into enemy ID 8 \$gameTroop.members()[0].transform(8); \$gameTroop.makeUniqueNames();</code>	Game_Enemy enemy - The one to transform. Number newEnemyId - Database ID of the transformed enemy.	Transformation will cap current HP/MP to the new maximum values.
☐	✓	Show Battle Animation	<code>\$gameTemp.requestAnimation([battler, animationId, mirror];</code>	<code>// MZ, show animation 9 on enemy #2 (index 1), not mirrored \$gameTemp.requestAnimation([\$gameTroop.members()[1], 9];</code>	Game_Battler battler - Animation target. Number animationId - ID of animation to show.	Can have multiple targets. This is typically for animations with <i>Display Type: Center of all targets</i> .
✓	☐		<code>battler.startAnimation(animationId, mirror, delay);</code>	<code>// MV, show animation 9 on enemy #2 (index 1) \$gameTroop.members()[1].startAnimation(9);</code>	Boolean mirror - If true, mirror the animation. Number delay (optional) - Frames before the animation starts.	-
✓	✓	Force Action	<code>var subject = battler; subject.forceAction(skillId, targetIndex); BattleManager.forceAction(subject); interpreter.setWaitMode('action');</code>	<code>// In a Script command: make enemy #1 use skill 18 on index 0 var subject = \$gameTroop.members()[0]; subject.forceAction(18, 0); BattleManager.forceAction(subject); this.setWaitMode('action');</code>	Game_Battler battler - The action user. Number skillId - ID of the skill to use. Number targetIndex - Determines the action target: -2: user's previous target; -1: random from target unit; 0+: specific index from target unit. Game_Interpreter interpreter - Interpreter to pause (<i>this</i> in a Script command).	Force action will clear the user's current action list.
✓	✓	Abort Battle	<code>BattleManager.abort();</code>	-	-	Aborts the battle. Mechanically similar to escape, but will not be counted in the <i>Battles escaped</i> system statistic.
		Advanced				
✓	✓	Script	-	-	-	Evaluates some JavaScript in the context of the <i>Game_Interpreter</i> processing the event. Technical info on eval here: eval .

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
☐	☒	Plugin Command	PluginManager.callCommand(interpreter , pluginName , command , args);	// In a Script command: a plugin command from <i>TemplateEvent.js</i> PluginManager.callCommand(this, 'TemplateEvent', 'SET_SELF_VARIABLE', { index: '1', type: '0', operand: '20' });	Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command). String pluginName • Plugin filename, without extension. String command • Command name. Object args • Has one property per command argument.	All arguments are passed in JSON/stringified format.
☒	☐		interpreter.pluginCommand(command, args);	// In a Script command: plugin command <i>ItemBook</i> add weapon 3 this.pluginCommand('ItemBook', ['add', 'weapon', '3']);	Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command). String command • Command name. String[] args • Ordered array of argument values.	
General Script Calls						
Message		<i>Message, choices, number input, select item</i>				▲ Return to top ▲
☐	☒	Show Text	\$gameMessage.setFacelImage(faceName , faceIndex); \$gameMessage.setBackground(bgType); \$gameMessage.setPositionType(posType); \$gameMessage.setSpeakerName(speakerName); \$gameMessage.add(text); interpreter.setWaitMode ('message');	// Set face: Actor1.png, index 0 \$gameMessage.setFacelImage('Actor1', 0); // Set background: Window \$gameMessage.setBackground(0); // Set position: Bottom \$gameMessage.setPositionType(2); // [MZ] Set speaker name: lc[6]\n[1] \$gameMessage.setSpeakerName("\lc[6]\n[1]"); // Show text: I am lc[6]\n[1]lc[0]! \$gameMessage.add('I am \lc[6]\n[1]\lc[0]!'); // Wait for message to be dismissed this.setWaitMode('message');	String faceName • Name of the faceset to use. Number faceIndex • Index of face. Use 0 for face #1. Number bgType • 0 (Window), 1 (Dim), or 2 (Transparent). Number posType • 0 (Top), 1 (Middle), or 2 (Bottom). String speakerName [MZ only] • Sets the name box text. String text • Message text! Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	In JavaScript, \backslash is used for escape notation, e.g. "\n" for a new line: String escape sequences . For an actual backslash, use "\\ " like in the example. The final line with <i>setWaitMode</i> pauses the interpreter until the message is dismissed. If you have follow-up commands (e.g. Show Choices or Input Number) then you can put <i>setWaitMode</i> after those instead.
☒	☐		\$gameMessage.setFacelImage(faceName , faceIndex); \$gameMessage.setBackground(bgType); \$gameMessage.setPositionType(posType); \$gameMessage.add(text); interpreter.setWaitMode ('message');			
☒	☒	Close Message	SceneManager._scene._messageWindow.terminateMessage();	-	-	Closes the message window. If related inputs (choices, number input, select item) are active, then use their OK/Cancel option instead!
☒	☒	Show Choices	\$gameMessage.setChoices(choices , defaultIndex , cancelIndex); \$gameMessage.setChoiceBackground(bgType); \$gameMessage.setChoicePositionType(posType); \$gameMessage.setChoiceCallback(callback); interpreter.setWaitMode ('message');	// Set choices to A, B, C; default C, no cancel \$gameMessage.setChoices(['A', 'B', 'C'], 2, -1); // Set background: Dim \$gameMessage.setChoiceBackground(1); // Set position: Center \$gameMessage.setChoicePositionType(1); // On choice, set variable #1 equal to the selected index \$gameMessage.setChoiceCallback(n => { \$gameVariables.setValue(1, n); }); // Wait for a choice to be selected this.setWaitMode('message');	String[] choices • Array of choice texts to show, in order. Number defaultIndex • Starting choice index (0 for choice #1). Number cancelIndex • Cancel selects this index (-1 for no cancel). • Use -2 for "Branch" cancel type. Number bgType • 0 (Window), 1 (Dim), or 2 (Transparent). Number posType • 0 (Left), 1 (Center), or 2 (Right). Function callback • 1 argument: selected choice index. • Gets called when a choice is made. • Runs in <i>Game_Interpreter</i> context. Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	You can use as many choices as you like: the window will scroll if necessary. Remember that choice #1 is index 0, #2 is index 1, etc. <i>setWaitMode</i> is important here.
☐	☒	Get Choice Window	SceneManager._scene._choiceListWindow	-	Returns Window_ChoiceList	Gets the choice window.
☒	☐		SceneManager._scene._messageWindow._choiceWindow	-	The choice window for the current scene.	For use in the following script calls~
☒	☒	Select Choice	choiceWindow.select(index);	// MZ: select choice #3 (index 2) in the active choice window SceneManager._scene._choiceListWindow.select(2);	Window_ChoiceList choiceWindow	Highlights the corresponding choice.
☒	☒	Choice OK	choiceWindow.processOk();	// MV: OK the selected choice SceneManager._scene._messageWindow._choiceWindow. .processOk();	Number index • Index of choice to select; use 0 for choice #1.	Chooses the highlighted choice.
☒	☒	Choice Cancel	choiceWindow.processCancel();			Cancels choices. Works even if cancel is disabled.
☒	☒	Input Number	\$gameMessage.setNumberInput(variableId , maxDigits); interpreter.setWaitMode ('message');	// Input a 5-digit number; store result in variable #1 \$gameMessage.setNumberInput(1, 5); // Wait for input to be confirmed this.setWaitMode('message');	Number variableId • ID of the variable used to store input. Number maxDigits • Number of digits to show. Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	<i>setWaitMode</i> is important here.
☐	☒	Get Input Number Window	SceneManager._scene._numberInputWindow	-	Returns Window_NumberInput	Gets the number input window.
☒	☐		SceneManager._scene._messageWindow._numberWindow	-	The number input window for the current scene.	For use in the following script call~
☒	☒	Number OK	numberInputWindow.processOk();	// MZ: OK the current number SceneManager._scene._numberInputWindow.processOk();	Window_NumberInput numberInputWindow • The number input window to affect.	Accepts the current input value.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Select Item	\$gameMessage.setItemChoice(variableId , itemType); interpreter .setWaitMode('message');	// Select a Key item; store that item's ID in variable #1 \$gameMessage.setItemChoice(1, 2); // Wait for an item to be selected this.setWaitMode('message');	Number variableId • ID of the variable to store selected item's ID. Number itemType • 1 (Normal), 2 (Key), 3 (Hidden A), or 4 (Hidden B). Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	setWaitMode is important here. On cancel, selected item ID is set to 0.
☐	✓	Get Select Item Window	SceneManager._scene._eventItemWindow SceneManager._scene._messageWindow._itemWindow	-	Returns Window_EventItem • The select item window for the current scene.	Gets the select item window. For use in the following script calls~
✓	✓	Select Item OK	eventItemWindow .processOk();	// MZ: OK the selected item SceneManager._scene._eventItemWindow.processOk();	Window_EventItem eventItemWindow • The select item window to affect.	Accepts the highlighted item.
✓	✓	Select Item Cancel	eventItemWindow .processCancel();			Cancels item selection.
✓	✓	Show Scrolling Text	\$gameMessage.setScroll(scrollSpeed , noFast); \$gameMessage.add(text); interpreter .setWaitMode('message');	// Write out the story text var text = ['People have been coming to the wise man,', 'complaining about the same problems every time.\n', 'One day he told them a joke and everyone roared', 'in laughter.\n', 'After a couple of minutes, he told them the same joke', 'and only a few of them smiled.\n', 'When he told the same joke for the third time no one', 'laughed anymore.\n', 'The wise man smiled and said:', 'You can't laugh at the same joke over and over.', 'So why are you always crying about the same problem?']].join("\n"); // Set the message to scroll mode, speed 3, can fast-forward \$gameMessage.setScroll(3, false); // Add the story to the message \$gameMessage.add(text); // Wait for scrolling text to end this.setWaitMode('message');	Number scrollSpeed • Normally 1~8. Boolean noFast • If true, the player cannot fast-forward it. String text • Message text! Game_Interpreter interpreter • Interpreter reference (<i>this</i> in a Script command).	See the setWaitMode note for Show Text. You can use "\n" for a new line. The example uses the <i>Array#join</i> method to merge a list of text lines into a single, multi-line string. More information here: Array#join .
✓	✓	End Scrolling Text	SceneManager._scene._scrollTextWindow.terminateMessage();	-	-	Closes the scrolling text window.
		Inventory	<i>Inventory: gold and items</i>	See also: Equipment		▲ Return to top ▲
✓	✓	Gold	\$gameParty.gold()	-	Returns Number • The party's current gold/money.	-
✓	✓	Add Gold	\$gameParty.gainGold(amount);	// Give the party +1000 gold \$gameParty.gainGold(1000);	Number amount • Quantity to add/remove (can be negative).	-
✓	✓	Remove Gold	\$gameParty.loseGold(amount);	// Remove 500 gold from the party \$gameParty.loseGold(500);		
✓	✓	Add Items	\$gameParty.gainItem(item , amount);	// Give the party 5x Item ID 1 \$gameParty.gainItem(\$dataItems[1], 5);	\$dataItem item • Database item reference.	-
✓	✓	Remove Items	\$gameParty.loseItem(item , amount);	// Remove up to 1 of Item 12 from the party inventory \$gameParty.loseItem(\$dataItems[12], 1);	Number amount • Quantity to add/remove (can be negative).	
✓	✓	Add Weapons	\$gameParty.gainItem(weapon , amount);	// Give the party 1x Weapon ID 1 \$gameParty.gainItem(\$dataWeapons[1], 1);	\$dataWeapon weapon • Database weapon reference. Number amount • Quantity to add/remove (can be negative).	
✓	✓	Remove Weapons	\$gameParty.loseItem(weapon , amount , includeEquip);	// Remove up to 99 of Weapon 2, including equipped items \$gameParty.loseItem(\$dataWeapons[2], 99, true);	Boolean includeEquip • If true, equipped weapons can be removed.	Equipped weapons/armors can only be removed if there are not enough of that item in inventory.
✓	✓	Add Armors	\$gameParty.gainItem(armor , amount);	// Give the party 2x Armor ID 1 \$gameParty.gainItem(\$dataArmors[1], 2);	\$dataArmor armor • Database armor reference.	
✓	✓	Remove Armors	\$gameParty.loseItem(armor , amount , includeEquip);	// Remove up to 5 of Armor 2, ignoring equipped items \$gameParty.loseItem(\$dataArmors[2], 5);	Number amount • Quantity to add/remove (can be negative). Boolean includeEquip • If true, equipped armors can be removed.	
✓	✓	Items in Inventory	\$gameParty.items()			Items only.
✓	✓	Weapons in Inventory	\$gameParty.weapons()		Returns (item)[] • Array of database item references from: <i>\$dataItems</i> , <i>\$dataWeapons</i> , and/or <i>\$dataArmors</i> .	Unequipped Weapons only.
✓	✓	Armors in Inventory	\$gameParty.armors()	-		Unequipped Armors only.
✓	✓	Equipment in Inventory	\$gameParty.equipItems()			Unequipped Weapons and Armors.
✓	✓	Everything in Inventory	\$gameParty.allItems()			Unequipped Items, Weapons, and Armors.
✓	✓	Inventory Item Data	\$gameParty._items			-
✓	✓	Inventory Weapon Data	\$gameParty._weapons	-	Returns Number[] • Array of item quantities, indexed by ID.	Equipped weapons and armors are stored separately.
✓	✓	Inventory Armor Data	\$gameParty._armors			
✓	✓			// Get the number of Item 1 in the party inventory \$gameParty.numItems(\$dataItems[1])	(item) item • Database item reference from:	

Script Calls		Last updated 2023-12-14 (01:50 UTC)					
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes	
✓	✓	Item Count	\$gameParty.numItems(item)	// Get the number of Weapon 2 in the party inventory \$gameParty.numItems(\$dataWeapons[2])	• Database item reference from: <i>\$dataItems</i> , <i>\$dataWeapons</i> , or <i>\$dataArmors</i> . Returns Number • Amount of that item in the party inventory.	Does not check equipped items.	
✓	✓			// Get the number of Armor 3 in the party inventory \$gameParty.numItems(\$dataArmors[3])			
✓	✓			// Does the party have 1+ of Item 1? \$gameParty.hasItem(\$dataItems[1])			
✓	✓						// Does the party have 1+ of Weapon 3, including equipment? \$gameParty.hasItem(\$dataWeapons[3], true)
✓	✓						
✓	✓	Party has Item?	\$gameParty.hasItem(item , includeEquip)		(item) item • Database item reference. Boolean includeEquip • If true, equipped items will be checked. Returns Boolean • true iff the item is present.	-	
✓	✓	Clear Inventory	\$gameParty.initAllItems(); \$gameMap.requestRefresh();	-	-	Removes all Items, Weapons, and Armors from the party inventory.	
✓	✓	Memorise Inventory [Remember Inventory] [Store Inventory]	\$gameVariables.setValue(variableId , { Armors: Object.assign({}, \$gameParty._armors), Items: Object.assign({}, \$gameParty._items), Weapons: Object.assign({}, \$gameParty._weapons) });	// Store the party inventory data in variable ID 1 \$gameVariables.setValue(1, { Armors: Object.assign({}, \$gameParty._armors), Items: Object.assign({}, \$gameParty._items), Weapons: Object.assign({}, \$gameParty._weapons) });	• Number variableId • ID of variable to store data	Stores current inventory data in a variable. Object#assign is used here to shallow-copy each object rather than just copy a reference to the current one.	
✓	✓	Restore Inventory	var data = \$gameVariables.value(variableId); for (var k in data) { var items = window["\$data" + k]; for (var n in data[k]) { \$gameParty.gainItem(items[n], data[k][n]); }; }; \$gameMap.requestRefresh();	// Restore party inventory from variable ID 1 var data = \$gameVariables.value(1); for (var k in data) { var items = window["\$data" + k]; for (var n in data[k]) { \$gameParty.gainItem(items[n], data[k][n]); }; }; \$gameMap.requestRefresh();		Add all stored items to inventory. Pairs with Memorise Inventory . for...in iterates over an object's property identifiers.	
✓	✓	Memorise Inventory Category	\$gameVariables.setValue(variableId , Object.assign({}, \$gameParty._armors));	// Store the party inventory armor data in variable ID 2 \$gameVariables.setValue(2, Object.assign({}, \$gameParty._armors);		Like Memorise Inventory , except for only a single category (Armors, Weapons, or Items).	
✓	✓		\$gameVariables.setValue(variableId , Object.assign({}, \$gameParty._weapons));				
✓	✓		\$gameVariables.setValue(variableId , Object.assign({}, \$gameParty._items));				
✓	✓	Restore Inventory Category	var mem = \$gameVariables.value(variableId); for (var n in mem) \$gameParty.gainItem(\$dataArmors[n], mem[n]);	// Restore party armors from variable ID 2 var mem = \$gameVariables.value(2); for (var n in mem) \$gameParty.gainItem(\$dataArmors[n], mem[n]);	Adds all stored items to inventory. Pairs with Memorise Inventory Category .		
✓	✓		var mem = \$gameVariables.value(variableId); for (var n in mem) \$gameParty.gainItem(\$dataWeapons[n], mem[n]);				
✓	✓		var mem = \$gameVariables.value(variableId); for (var n in mem) \$gameParty.gainItem(\$dataItems[n], mem[n]);				
✓	✓	Add All Items	\$dataItems.forEach(function(item) { \$gameParty.gainItem(item, 99); });	-	-	Gives the party 99x every Item in the database. For weapons, replace <i>\$dataItems</i> with <i>\$dataWeapons</i> . For armors, replace <i>\$dataItems</i> with <i>\$dataArmors</i> .	
✓	✓	Add All Named Items	\$dataItems.forEach(function(item) { if (item && item.name) \$gameParty.gainItem(item, 99); });	-	-	Gives the party 99x every named Item in the database. For weapons, replace <i>\$dataItems</i> with <i>\$dataWeapons</i> . For armors, replace <i>\$dataItems</i> with <i>\$dataArmors</i> .	
Actors & Enemies			Get/set references to battlers (actors/enemies) and their units (party/troop)			▲ Return to top ▲	
✓	✓	Actor (by ID)	\$gameActors.actor(actorId)	// Get the data for actor ID 1 \$gameActors.actor(1)	Number actorId • ID of the actor. Returns Game_Actor • Actor with that ID.	-	
✓	✓	Actor (by Position)	\$gameParty.members()[index]	// Get the data for party member 0 (i.e. the leader) \$gameParty.members()[0]	Number index • Position in party: 0 = leader, 1 = first follower, etc. Returns Game_Actor • Actor in that position.	You can use <i>battleMembers</i> or <i>allMembers</i> instead of <i>members</i> . See below for details.	
✓	✓	Party Leader	\$gameParty.leader()	-	Returns Game_Actor • Actor in the first position in the party.	A shortcut for <i>\$gameParty.members()[0]</i> .	

Script Calls		Last updated 2023-12-14 (01:50 UTC)					
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes	
✓	✓	Enemy (by Position)	<code>\$gameTroop.members()[index]</code>	<code>// Get enemy #1 (index 0) \$gameTroop.members()[0]</code>	<u>Number index</u> - Position in troop: use 0 for the leader. Returns Game_Enemy - Enemy in that position.	<i>\$gameTroop</i> is only reset on new game, game boot, and at the start of each battle. It is not included in save data, so be careful if referencing it outside of battle.	
✓	✓	Troop Members	<code>\$gameTroop.members()</code>	-	Returns Game_Enemy[] Enemies in the current troop.	Enemies who have not yet appeared, or have escaped, will not count as alive or dead. However, they still count as members of the troop.	
✓	✓	Alive Enemies	<code>\$gameTroop.aliveMembers()</code>		Returns Game_Enemy[] Alive/dead enemies in the current troop.		
✓	✓	Dead Enemies	<code>\$gameTroop.deadMembers()</code>		Returns Game_Actor[] Actors in the party.	In battle, this returns <i>\$gameParty.battleMembers()</i> . Otherwise, it returns <i>\$gameParty.allMembers()</i> .	
✓	✓	Party Members	<code>\$gameParty.members()</code>			Returns only those actors in the active/battle party.	
✓	✓	Active Party Members	<code>\$gameParty.battleMembers()</code>			Includes reserve members.	
✓	✓	All Party Members	<code>\$gameParty.allMembers()</code>			Returns Game_Actor[] Alive/dead actors in the current party.	Checks <i>\$gameParty.members()</i> . Escaped or otherwise hidden members will not count as alive or dead.
✓	✓	Alive Party Members	<code>\$gameParty.aliveMembers()</code>				
✓	✓	Dead Party Members	<code>\$gameParty.deadMembers()</code>				
✓	✓	Add Actor to Party	<code>\$gameParty.addActor(actorId);</code>	<code>// Add actor ID 4 to the party \$gameParty.addActor(4);</code>	<u>Number actorId</u> ID of the actor to add/remove.	If used in battle you may need to refresh the battle status window to match the new party setup.	
✓	✓	Remove Actor from Party	<code>\$gameParty.removeActor(actorId);</code>	<code>// Remove actor ID 1 from the party \$gameParty.removeActor(1);</code>			
✓	✓	Swap Party Order	<code>\$gameParty.swapOrder(index1, index2);</code>	<code>// Swap members #1 and #4 (indices 0 and 3) \$gameParty.swapOrder(0, 3);</code>	<u>Number index1</u> - First position, use 0 for leader. <u>Number index2</u> - Second position.		
✓	✓	Position in Unit	<code>battler.index()</code>	<code>// Get the position of actor ID 1 in the party \$gameActors.actor(1).index()</code>	<u>Game_Battler battler</u> - Battler to check. Returns Number - Position in their unit (0+) or -1 if absent.	Actors check <i>\$gameParty</i> ; enemies check <i>\$gameTroop</i> .	
✓	✓	Party Size	<code>\$gameParty.size()</code>	-	Returns Number Number of actors in the party.	Returns the number of actors in <i>\$gameParty.members()</i> .	
✓	✓	Is the Party Empty?	<code>\$gameParty.isEmpty()</code>	-	Returns Boolean true iff the party has no members.	-	
✓	✓	Clear Party	<code>\$gameParty.members().forEach(function(actor) { \$gameParty.removeActor(actor.actorId()); });</code>	-	-	Removes all party members.	
✓	✓	Memorise Party Members [Remember Party] [Store Party]	<code>\$gameVariables.setValue(variableId, \$gameParty.members().map(function(actor) { return actor.actorId(); }));</code>	<code>// Store current party member IDs in variable 1 \$gameVariables.setValue(1, \$gameParty.members().map(function(actor) { return actor.actorId(); }));</code>	<u>Number variableId</u> ID of variable to store data.	Stores an array of actor IDs (corresponding to the current party members) in a variable.	
✓	✓	Restore Party Members	<code>\$gameVariables.value(variableId).forEach(function(actorId) { \$gameParty.addActor(actorId); });</code>	<code>// Restore saved party members from variable 1 \$gameVariables.value(1).forEach(function(actorId) { \$gameParty.addActor(actorId); });</code>		Adds all stored actor IDs to the party. Pairs with Memorise Party Members .	
✓	✓	Is Battler an Actor?	<code>battler.isActor()</code>	<code>// In a skill/item formula: is the user an actor? a.isActor()</code>	<u>Game_Battler battler</u> The one to check.	Useful in skill/item formulae. Can also be used elsewhere.	
✓	✓	Is Battler an Enemy?	<code>battler.isEnemy()</code>	<code>// In a skill/item formula: get the target's enemy ID, or 0 if an actor b.isEnemy() ? b.enemyId() : 0</code>	Returns Boolean true iff the battler is of that type.	Useful for avoiding errors when getting actor/enemy ID.	
✓	✓	Actor ID	<code>actor.actorId()</code>	<code>// Get the leader's actor ID \$gameParty.members()[0].actorId()</code>	<u>Game_Actor actor</u> - The one to check. Returns Number - That actor's database ID.	-	
✓	✓	Enemy ID	<code>enemy.enemyId()</code>	<code>// Get the ID of enemy #1 (index 0) \$gameTroop.members()[0].enemyId()</code>	<u>Game_Enemy enemy</u> - The one to check. Returns Number - That enemy's database ID.		
✓	✓	Actor Database Record	<code>actor.actor()</code>	<code>// Get the database record for the party leader \$gameParty.members()[0].actor()</code>	<u>Game_Actor actor</u> - The one to check. Returns \$dataActor - That actor's database record.	Helpful for looking up static values that are not directly included in <i>Game_Actor</i> / <i>Game_Enemy</i> instances.	
✓	✓	Enemy Database Record	<code>enemy.enemy()</code>	<code>// Does the enemy troop leader have a <flying> notetag? \$gameTroop.members()[0].enemy().meta.flying</code>	<u>Game_Enemy enemy</u> - The one to check. Returns \$dataEnemy - That enemy's database record.	See the Database sheet of the MZ Script Call Reference for details on Actor/Enemy properties.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Troop Database Record	troop.troop()	// Does the current troop's name include "goblin"? \$gameTroop.troop().name.indexOf("goblin") >= 0	Game_Troop troop • The one to check. Returns \$dataTroop • That troop's database record.	Typically there is only one troop instance, \$gameTroop. See the Database sheet of the MZ Script Call Reference for details on Troop properties.
Actor/Enemy Parameters			Get/set battler parameter values			▲ Return to top ▲
✓	✓	Battlers	Actors and enemies are both "battlers". Many things can be applied to any battler, e.g. Change HP or Add State.			
✓	✓	Change HP	target.setHp(value);	// Set the HP of party member #2 (first follower) to 50% var actor = \$gameParty.members()[1]; actor.setHp(actor.mhp / 2);	Game_Interpreter interpreter • Interpreter reference (this in a Script command). Game_Battler battler • The one to gain/lose HP. Number value • Amount of HP to gain (can be negative).	-
✓	✓		target.gainHp(value);	// Heal 50 HP for actor ID 1 \$gameActors.actor(1).gainHp(10);	Number value • Amount of HP to gain (can be negative).	No effect if the target is dead.
✓	✓		interpreter.changeHp(target, value, allowKO);	// In a Script command: deal 100 damage to party leader, allow KO this.changeHp(\$gameParty.members()[0], -100, true);	Boolean allowKO • If true, the target can die.	Collapse effect will start if the target dies in battle.
✓	✓		Game_Interpreter.prototype.changeHp.call(this, target, value, allowKO);	// Deal 200 damage to actor ID 1, cannot KO Game_Interpreter.prototype.changeHp.call(this, \$gameActors.actor(1), 200, false);		
✓	✓	Change MP	battler.gainMp(value);	// Make actor ID 2 lose 20 MP \$gameActors.actor(2).gainMp(-20);	Game_Battler battler • The one to gain/lose MP. Number value • Amount of MP to gain (can be negative).	-
✓	✓			// Restore +10% MP for the party leader var actor = \$gameParty.members()[0]; actor.gainMp(0.1 * actor.mmp);		
✓	✓	Change TP	battler.gainTp(value);	// Make party member #3 (index 2) gain 100 TP \$gameParty.members()[2].gainTp(100);	Game_Battler battler • The one to gain/lose TP. Number value • Amount of TP to gain (can be negative).	-
✓	✓			// Make the party leader lose 10 TP \$gameParty.members()[0].gainTp(-10);		
✓	✓	HP	battler.hp	// Get the current HP of the party leader \$gameParty.members()[0].hp	Game_Battler battler • The one to check. Returns Number • The current HP/MP/TP value.	These are read-only values. To change HP, MP, or TP, use the appropriate functions (above).
✓	✓	MP	battler.mp	// Get the current MP of enemy #3 (i.e. index 2) \$gameTroop.members()[2].mp		
✓	✓	TP	battler.tp	// Get the current TP of actor ID 1 \$gameActors.actor(1).tp		
✓	✓	HP%	battler.hpRate()	// Get the current HP% of the party leader \$gameParty.members()[0].hpRate()	Game_Battler battler • The one to check. Returns Number • 0 to 1, where 1 = 100% (i.e. full HP/MP/TP).	-
✓	✓	MP%	battler.mpRate()	// Get the current MP% of enemy #1 (i.e. index 0) \$gameTroop.members()[0].mpRate()		
✓	✓	TP%	battler.tpRate()	// Get the current TP% of actor ID 1 \$gameActors.actor(1).tpRate()		
		Basic Params	Basic parameters			
✓	✓	Basic Parameter	battler.param(paramId)			You can use one of the shortcuts below instead.
✓	✓	Max HP	battler.mhp			paramId = 0
✓	✓	Max MP	battler.mmp		Game_Battler battler • The one to check. Number paramId • ID of the basic parameter. Returns Number • The value of that basic parameter.	paramId = 1
✓	✓	ATK	battler.atk	// Get the max HP (MHP) of actor ID 1 \$gameActors.actor(1).mhp		paramId = 2
✓	✓	DEF	battler.def			paramId = 3
✓	✓	MAT	battler.mat			paramId = 4
✓	✓	MDF	battler.mdf	// Get the luck (LUK) of enemy #1 (i.e. index 0) \$gameTroop.members()[0].luk		paramId = 5
✓	✓	AGI	battler.agi			paramId = 6
✓	✓	LUK	battler.luk			paramId = 7
✓	✓	Max TP	battler.maxTp()	// Get the party leader's maximum TP \$gameParty.members()[0].maxTp()	Game_Battler battler • The one to check. Returns Number • Maximum TP value for that battler.	By default this is 100 for everyone. This should be used to get the maximum, though, just in case a plugin is used that changes the value.
✓	✓	Add to Basic Parameter (Change Parameters)	battler.addParam(paramId, value);	// Give the party leader a permanent +10 ATK bonus \$gameParty.members()[0].addParam(2, 10);	Game_Battler battler • The one to gain/lose the basic param bonus. Number paramId • Basic parameter ID. Number value • Bonus to add (can be negative).	Basic parameters are calculated as: (Base + Plus) * TraitRate * BuffRate
				// Give actor ID 5 a permanent -20 LUK malus \$gameActors.actor(5).addParam(7, -20);		addParam increases/decreases the Plus value.
✓	✓	Basic parameter (Base value)	battler.paramBase(paramId)	// Get the party leader's base ATK value (from class) \$gameParty.members()[0].paramBase(2)		- Base is the starting value (from class for actors)

Script Calls			Last updated 2023-12-14 (01:50 UTC)			
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Basic parameter (Bonus)	battler._paramPlus[paramId]	// Get the party leader's ATK bonus (from addParam) \$gameParty.members()[0]._paramPlus[2]	Game_Battler battler	• Base is the starting value (from class for actors). • Plus is from addParam and equipped items. • TraitRate is from traits. • BuffRate is from buffs. Basic parameter IDs: • 0 = MHP • 2 = ATK • 4 = MAT • 6 = AGI • 1 = MMP • 3 = DEF • 5 = MDF • 7 = LUK
✓	✓	Basic parameter (Plus value)	battler.paramPlus(paramId)	// Get the party leader's total ATK bonus, including equipment \$gameParty.members()[0].paramPlus(2)	• The one to check. Number paramId	
□	✓	Basic parameter (Base + Plus)	battler.paramBasePlus(paramId)	// Get the party leader's ATK without traits/buffs \$gameParty.members()[0].paramBasePlus(2)	• Basic parameter ID (see above). Returns Number	
✓	✓	Basic parameter (Trait rate)	battler.paramRate(paramId)	// Get actor 2's current LUK rate from traits \$gameActors.actor(2).paramRate(7)	• Value of that parameter or rate.	
✓	✓	Basic parameter (Buff rate)	battler.paramBuffRate(paramId)	// Get actor 2's current LUK rate from (de)buffs \$gameActors.actor(2).paramBuffRate(7)	• Rates are percentages, i.e. 1 = 100%.	
		Ex-Params	Extra parameters			
✓	✓	Extra parameter	battler.xparam(xparamId)			You can use one of the shortcuts below instead. xparamId = 0 xparamId = 1 xparamId = 2 xparamId = 3 xparamId = 4 xparamId = 5 xparamId = 6 xparamId = 7 xparamId = 8 xparamId = 9
✓	✓	Hit Rate	battler.hit			
✓	✓	Evasion Rate	battler.eva	// Get the hit rate (HIT) of the party leader \$gameParty.members()[0].hit	Game_Battler battler	
✓	✓	Critical Rate	battler.cri		• The one to check. Number xparamId	
✓	✓	Critical Evasion	battler.cev		• ID of the extra parameter.	
✓	✓	Magic Evasion	battler.mev	// Get the magic reflect rate (MRF) of actor ID 5 \$gameActors.actor(5).mrf	Returns Number	
✓	✓	Magic Reflection	battler.mrf		• The value of that extra parameter (1 = 100%).	
✓	✓	Counter Rate	battler.cnt	// Get the HP Regen rate (HRG) of enemy #2 (i.e. index 1) \$gameTroop.members()[1].hrg		
✓	✓	HP Regeneration	battler.hrg			
✓	✓	MP Regeneration	battler.mrg			
✓	✓	TP Regeneration	battler.trg			
		Sp-Params	Special parameters			
✓	✓	Special parameter	battler.sparam(sparamId)			You can use one of the shortcuts below instead. sparamId = 0 sparamId = 1 sparamId = 2 sparamId = 3 sparamId = 4 sparamId = 5 sparamId = 6 sparamId = 7 sparamId = 8 sparamId = 9
✓	✓	Target Rate	battler.tgr			
✓	✓	Guard Effect Rate	battler.grd	// Get the recovery rate (REC) of the party leader \$gameParty.members()[0].rec	Game_Battler battler	
✓	✓	Recovery Rate	battler.rec		• The one to check. Number sparamId	
✓	✓	Pharmacology	battler.pha		• ID of the special parameter.	
✓	✓	Mana Cost Rate	battler.mcr	// Get the guard rate (GRD) of actor ID 1 \$gameActors.actor(1).grd	Returns Number	
✓	✓	TP Charge Rate	battler.tcr		• The value of that special parameter (1 = 100%).	
✓	✓	Physical Damage Rate	battler.pdr	// Get the target rate (TGR) of enemy #1 (i.e. index 0) \$gameTroop.members()[0].tgr		
✓	✓	Magical Damage Rate	battler.mdr			
✓	✓	Floor Damage Rate	battler.fdr			
✓	✓	Experience Rate	battler.exr			
		Other	Other parameters			
✓	✓	Level	actor.level	// Get the current level of actor ID 1 \$gameActors.actor(1).level	Game_Actor actor	This is a read-only value. Use Change Level to change its value.
					• The one to check. Returns Number	
✓	✓	Change Level	actor.changeLevel(level, show);	// Set the party leader to level 5 \$gameParty.members()[0].changeLevel(5, true); // Silently increase the level of actor ID 2 by +1 var actor = \$gameActors.actor(2); actor.changeLevel(actor.level + 1, false);	Game_Actor actor	-
✓	✓	EXP	actor.currentExp()	// Get the party leader's current EXP \$gameParty.members()[0].currentExp()	Game_Actor actor	Gets current total experience.
✓	✓	EXP for level	actor.expForLevel(level)	// Get the minimum total EXP for the leader to be level 5+ \$gameParty.members()[0].expForLevel(5)	• The one to change level. Number level	Gets experience threshold for given level.
✓	✓	EXP until next level	actor.nextRequiredExp()	// Get the minimum EXP required for the party leader to level up \$gameParty.members()[0].nextRequiredExp()	• The level to check.	Gets experience required to level up.
✓	✓	Change EXP	actor.gainExp(value);	// Give +1000 EXP to the party leader \$gameParty.members()[0].gainExp(1000); // Remove 500 EXP from actor ID 2 \$gameActors.actor(2).gainExp(-500);	Returns Number	Level-up messages will be shown if appropriate.
					• Experience total.	
✓	✓	Name	battler.name()	// Get the name of the party leader \$gameParty.members()[0].name() // Check if actor 5's name in uppercase is PASSWORD \$gameActors.actor(5).name().toUpperCase() === 'PASSWORD'	Game_Battler battler	-
					• The one to check. Returns String	
					• That battler's name.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Original Enemy Name	<code>enemy.originalName()</code>	<code>// Get the name of enemy #1 as it is defined in the database \$gameTroop.members()[0].originalName()</code>	<u>Game_Enemy</u> enemy • The one to check. Returns String • That enemy's database name.	Duplicate enemies are named Enemy A, Enemy B, etc. Use this to get the name as it is in the database.
✓	✓	Change Name	<code>actor.setName(name);</code>	<code>// Rename the party leader to Leader \$gameParty.members()[0].setName("Leader");</code>	<u>Game_Actor</u> actor • The one to rename. String name • Their new name.	-
✓	✓			<code>// Set the name of actor 2 equal to the name of actor 1 \$gameActors.actor(2).setName(\$gameActors.actor(1).name());</code>		
✓	✓	Nickname	<code>actor.nickname()</code>	<code>// Get the nickname of the party leader (member index 0) \$gameParty.members()[0].nickname()</code>	<u>Game_Actor</u> actor • The one to check. Returns String • That actor's nickname.	-
✓	✓	Change Nickname	<code>actor.setNickname(nickname);</code>	<code>// Set the nickname of actor ID 1 to Superhero \$gameActors.actor(1).setNickname("Superhero");</code>	<u>Game_Actor</u> actor • The one to rename. String nickname • New nickname.	-
✓	✓	Profile	<code>actor.profile()</code>	<code>// Get the profile text of the party leader \$gameParty.members()[0].profile()</code>	<u>Game_Actor</u> actor • The one to check. Returns String • That actor's profile status text.	-
✓	✓	Change Profile	<code>actor.setProfile(profile);</code>	<code>// Set the party leader's profile to Here is some profile text. \$gameParty.members()[0].setProfile("Here is some profile text."); // Add to the end of actor 2's profile var actor = \$gameActors.actor(2); var text = actor.profile() + "\n\nSome text."; actor.setProfile(text);</code>	<u>Game_Actor</u> actor • The one to change profile. String profile • New profile.	The profile displays on the status screen.
✓	✓	Set Actor Map Character	<code>actor.setCharacterImage(filename, index); \$gamePlayer.refresh();</code>	<code>// Change the party leader's images: // Map character: img/characters/Hero.png, index 0 (top-left) // Face: img/faces/Hero.png, index 3 (face #4)</code>	<u>Game_Actor</u> actor • The one to affect.	For walking on the map.
✓	✓	Set Actor Face Image	<code>actor.setFacelImage(filename, index); \$gamePlayer.refresh();</code>	<code>// Battler: img/sv_actors/Hero.png var actor = \$gameParty.members()[0]; actor.setCharacterImage("Hero", 0); actor.setFacelImage("Hero", 3);</code>	String filename • Spritesheet filename, without extension.	For menus etc.
✓	✓	Set Actor SV Battler	<code>actor.setBattlerImage(filename); \$gamePlayer.refresh();</code>	<code>actor.setBattlerImage("Hero", 3); actor.setBattlerImage("Hero"); \$gamePlayer.refresh();</code>	Number index • Position of the image in its file, typically 0–7.	For side-view battles (unnecessary if using front-view).
✓	✓	Actor Map Character Name	<code>actor.characterName()</code>	<code>// Get the index of the party leader's map sprite in their spritesheet \$gameParty.members()[0].characterIndex()</code>	<u>Game_Actor</u> actor • The one to check.	Gets the name of this character's map spritesheet.
✓	✓	Actor Map Character Index	<code>actor.characterIndex()</code>		Returns String Number	Gets the index of this character's map sprite.
✓	✓	Actor Face Image Name	<code>actor.faceName()</code>	<code>// Is the SV battler image for actor ID 1 named HeavyArmor.png? \$gameActors.actor(1).battlerName() === "HeavyArmor"</code>	• The filename (no extension) or sprite index.	Gets the name of this character's faceset.
✓	✓	Actor Face Image Index	<code>actor.faceIndex()</code>			Gets the index of this character's menu face.
✓	✓	Actor SV Battler Name	<code>actor.battlerName()</code>			Gets the name of this character's SV battler sheet.
		Classes & Skills	Check/change actor classes & skills			▲ Return to top ▲
✓	✓	(Class/Skill details)	See the Database sheet of the MZ Script Call Reference for details on \$dataClass and \$dataSkill properties.			
✓	✓	Class	<code>actor.currentClass()</code>	<code>// Get the current class of the party leader \$gameParty.members()[0].currentClass()</code> <code>// Get the name of the party leader's current class \$gameParty.members()[0].currentClass().name</code>	<u>Game_Actor</u> actor • The one to check. Returns \$dataClass • Class data for that actor's current class.	-
✓	✓	Change Class	<code>actor.changeClass(classId, keepExp);</code>	<code>// Change the party leader to class ID 3, keeping previous EXP \$gameParty.members()[0].changeClass(3, true);</code>	<u>Game_Actor</u> actor • The one to change class. Number classId • New class ID	-
✓	✓			<code>// Make actor 1 change to class 1 with 0 EXP \$gameActors.actor(1).changeClass(1, false);</code>	Boolean keepExp • If false, reset EXP to 0.	
✓	✓	Check Class	<code>actor.isClass(class)</code>	<code>// Is the party leader class ID 4 right now? \$gameParty.members()[0].isClass(\$dataClasses[4])</code>	<u>Game_Actor</u> actor • The one to check. <u>\$dataClass</u> class • Class to check against (database reference). Returns Boolean • true iff that actor is that class.	-
✓	✓	Learn Skill	<code>actor.learnSkill(skillId);</code>	<code>// Make the party leader learn skill ID 25 \$gameParty.members()[0].learnSkill(25);</code>	<u>Game_Actor</u> actor • The one to learn/forget the skill.	-
✓	✓	Forget Skill	<code>actor.forgetSkill(skillId);</code>	<code>// Make actor ID 2 forget skill 7 \$gameActors.actor(2).forgetSkill(7);</code>	Number skillId • New skill ID.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Skill is Learned?	<code>actor.isLearnedSkill(skillId)</code>	// Has the party leader learned skill ID 42? <code>\$gameParty.members()[0].isLearnedSkill(42)</code>	Game_Actor actor • The one to check. Number skillId • ID of the skill to check. Returns Boolean • true iff that actor has learned that skill.	This does not include skills granted by Add Skill traits.
✓	✓	Skills	<code>actor.skills()</code>	// Get all skills known by actor ID 1 <code>\$gameActors.actor(1).skills()</code>	Game_Actor actor • The one to check.	-
✓	✓	Usable Skills	<code>actor.usableSkills()</code>	// Get all skills that actor 2 can use right now <code>\$gameActors.actor(2).usableSkills()</code>	Returns \$dataSkill[] • Array of skills.	Does not include skills that cannot be used right now.
✓	✓	Skill is Known?	<code>actor.hasSkill(skillId)</code>	// Is skill ID 42 in the party leader's skill list? <code>\$gameParty.members()[0].hasSkill(42)</code>	Game_Actor actor • The one to check. Number skillId • ID of the skill to check Returns Boolean • true iff that actor knows that skill.	Matches against <code>actor.skills()</code> .
✓	✓	Add All Skills (1 Actor)	<code>\$dataSkills.forEach(function(skill) { if (skill) actor.learnSkill(skillId); });</code>	// Make the party leader learn every skill in the database <code>\$dataSkills.forEach(function(skill){ if (skill) \$gameParty.members()[0].learnSkill(skillId); });</code>	Game_Actor actor • The one to learn all skills.	
✓	✓	Add All Skills (Party)	<code>\$gameParty.members().forEach(function(actor) { \$dataSkills.forEach(function(skill) { if (skill) actor.learnSkill(skillId); }); });</code>	-	-	Teaches all skills to target actor(s).
		Equipment	Check/change actor equipment	See also: Inventory		▲ Return to top ▲
✓	✓	(Armor/Weapon details)	See the Database sheet of the MZ Script Call Reference for details on \$dataWeapon and \$dataArmor properties.			
✓	✓	Equipped Weapons	<code>actor.weapons()</code>	// Get all weapons equipped by actor ID 1 <code>\$gameActors.actor(1).weapons()</code>	Game_Actor actor • The one to check.	Equipped Weapons.
✓	✓	Equipped Armors	<code>actor.armors()</code>	// Get all armors equipped by party member #3 (i.e. index 2) <code>\$gameParty.members()[2].armors()</code>	Returns (equip)[] • Array of database item references from: \$dataWeapons and/or \$dataArmors.	Equipped Armors.
✓	✓	All Equipment	<code>actor.equips()</code>	// Get all weapons and armors equipped by the party leader <code>\$gameParty.members()[0].equips()</code>		Equipped Armors and Weapons.
✓	✓	Weapon is Equipped?	<code>actor.hasWeapon(weapon)</code>	// Does the party leader have weapon ID 8 equipped? <code>\$gameParty.members()[0].hasWeapon(\$dataWeapons[8])</code>	Game_Actor actor • The one to check. (equip) weapon, armor • Reference from \$dataWeapons or \$dataArmors. Returns Boolean • true iff that actor has that item equipped.	-
✓	✓	Armor is Equipped?	<code>actor.hasArmor(armor)</code>	// Does actor 5 have armor 13 equipped? <code>\$gameActors.actor(5).hasArmor(\$dataArmors[13])</code>		
✓	✓	Change Equipment	<code>actor.changeEquipById(etypeId, itemId);</code>	// Equip weapon/armor ID 5 in the party leader's first equip slot <code>\$gameParty.members()[0].changeEquipById(1, 5);</code>	Game_Actor actor • The one to (un)equip the item. Number etypeId • Equipment Type ID: 1 = first slot. Number itemId • ID of the item to equip (0 to empty that slot).	Equipment Types are defined on the Types tab in the database. The item must be present in the party inventory for this to work.
✓	✓	Force Change Equipment	<code>actor.forceChangeEquip(slotId, item);</code>	// Replace the item in the leader's 3rd slot (index 2) with Armor 5 <code>\$gameParty.members()[0].forceChangeEquip(2, \$dataArmors[5]);</code>	Game_Actor actor • Change this actor's equipment. Number slotId • Equip slot ID: 0 = first slot! \$dataArmor \$dataWeapon item • The item to put in that slot (null to empty it).	Replaces the item currently in the slot with the new item. The current item will be discarded. The new item does not have to be in the party inventory. If the new item cannot be held in that slot, it will be released to the inventory.
✓	✓	Unequip All Items (1 Actor)	<code>actor.clearEquipments();</code>	// Make actor ID 1 unequip everything <code>\$gameActors.actor(1).clearEquipments();</code>	Game_Actor actor • The one to unequip.	Unequipped Armors and Weapons will be moved to the inventory.
✓	✓	Unequip All Items (Party)	<code>\$gameParty.members().forEach(function(actor) { actor.clearEquipments(); });</code>	// Make party member #1 (index 0) unequip everything <code>\$gameParty.members()[0].clearEquipments();</code>	-	
✓	✓	Discard Equipped Item	<code>actor.discardEquip(item);</code>	// If the party leader has weapon 1 equipped, discard it <code>\$gameParty.members()[0].discardEquip(\$dataWeapons[1]);</code>	Game_Actor actor • Discard from this actor's equipment. (equip) item • The Armor or Weapon to discard. • Reference from \$dataWeapons or \$dataArmors.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Discard All Equipment (1 Actor)	var self = actor ; self.equips().forEach(function(item) { self.discardEquip(item); });	// Discard all of actor 1's equipped items var self = \$gameActors.actor(1); self.equips().forEach(function(item) { self.discardEquip(item); });	Game_Actor actor • Discard this actor's equipment.	Discarded Armors and Weapons will be removed from the party.
✓	✓	Discard All Equipment (Party)	\$gameParty.members().forEach(function(actor) { actor.equips().forEach(function(item) { actor.discardEquip(item); }); });	-	-	
✓	✓	Memorise Equipment	\$gameVariables.setValue(variableId , JsonEx.makeDeepCopy(actor ._equips));	// Store the party leader's current equipment in variable ID 5 \$gameVariables.setValue(5, JsonEx.makeDeepCopy(\$gameParty.members()[0]._equips));	Number variableId • ID of variable to store data.	Stores current equipment data (<i>Game_Items</i>) in a variable.
✓	✓	Restore Equipment	\$gameVariables.value(variableId).forEach(function(item, slotId) { actor .forceChangeEquip(slotId, item.object()); });	// Restore the party leader's equipment from variable ID 5 \$gameVariables.value(5).forEach(function(item, slotId) { \$gameParty.members()[0].forceChangeEquip(slotId, item.object()); });	Game_Actor actor • Store this actor's equipment data.	Restores equipment from data stored in variable. <i>forceChangeEquip</i> means the equipment will not be sourced from the party inventory. It is <i>item.object()</i> here because <i>item</i> is a <i>Game_Item</i> , not a <i>\$dataItem</i> .
		States, Buffs, etc	States, buffs, and misc trait objects			▲ Return to top ▲
✓	✓	(State details)	See the Database sheet of the MZ Script Call Reference for details on \$dataState properties.			
✓	✓	Add State	battler .addState(stateld);	// Add state ID 4 to enemy #1 (index 0); no collapse check \$gameTroop.members()[0].addState(4);	Game_Battler battler • The one to gain/lose the state. Number stateld • ID of the state to add/remove.	In some situations, adding/removing a state can cause the target to die/revive, e.g. • Add state 1: • Remove a state with a "Resist State 1" trait at 0 HP. In these cases it is recommended to use the "check for collapse" version to ensure the collapse effect plays at the correct time.
✓	✓		var target = battler ; var isDead = target.isDead(); target.addState(stateld); if (!isDead && target.isDead()) target.performCollapse();	// Add state ID 1 to the party leader; check for collapse var target = \$gameParty.members()[0]; var isDead = target.isDead(); target.addState(1); if (!isDead && target.isDead()) target.performCollapse();		
✓	✓	Remove State	battler .removeState(stateld);	// Remove state ID 6 from the party leader; no collapse check \$gameActors.actor(2).removeState(6);		
✓	✓		var target = battler ; var isDead = target.isDead(); target.removeState(stateld); if (!isDead && target.isDead()) target.performCollapse();	// Remove state ID 12 from actor ID 2; check for collapse var target = \$gameActors.actor(2); var isDead = target.isDead(); target.addState(1); if (!isDead && target.isDead()) target.performCollapse();		
✓	✓	States	battler .states()	// Get all states affecting the party leader \$gameParty.members()[0].states()	Game_Battler battler • The one to check. Returns \$dataState[] • Array of states currently active on that battler.	See the Database sheet of the MZ Script Call Reference for details on \$dataState properties.
✓	✓	Has State?	battler .isStateAffected(stateld)	// Is the party leader affected by state ID 5? \$gameParty.members()[0].isStateAffected(5)	Game_Battler battler • The one to check. Number stateld • ID of state to check for. Returns Boolean • true iff they are affected by that state.	-
✓	✓	State Duration	battler ._stateTurns[stateld]	// Check the turns remaining for state 5 on the party leader \$gameParty.members()[0]._stateTurns[5]	Game_Battler battler • The one to check. Number stateld • ID of state to check for. Returns Number • Turns remaining for that state.	Will be <i>undefined</i> if the state is not present. States with <i>Auto-Removal Timing: None</i> still update their state turns down to 0, but the number is not used for anything.
✓	✓	Add Buff	battler .addBuff(paramId , turns);	// Add a 3-turn ATK buff to the party leader \$gameParty.members()[0].addBuff(2, 3);	Game_Battler battler • The one to (de)buff. Number paramId • ID of basic parameter to (de)buff. Number turns • Duration of (de)buff.	See above for basic parameter IDs. A buff gives +1 buff level to a parameter. A debuff gives -1 buff level.
✓	✓	Remove Buff	battler .removeBuff(paramId , turns);	// Add a 2-turn AGI debuff to enemy #1 (index 0) \$gameTroop.members()[0].addDebuff(6, 2);	Game_Battler battler • The one to check. Number paramId • ID of basic parameter to check.	By default the min/max buff level is ±2. Buff level 0 means no buffs or debuffs are active.
✓	✓	Buff Level	battler .buff(paramId)	// Check the MHP buff level of the party leader \$gameParty.members()[0].buff(0)	Game_Battler battler • The one to check. Number paramId • ID of basic parameter to check.	Duration of (de)buffs will use the higher value when a buff is applied, e.g. if current duration is 5 turns and the new buff lasts 2 turns (or vice versa), the new duration will be 5 turns.
✓	✓	Buff Duration	battler ._buffTurns[paramId]	// Check the turns remaining for the party leader's MHP buff \$gameParty.members()[0]._buffTurns[0]	Returns Number • Buff level/duration for that battler & parameter.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Recover All (1 Battler)	<code>battler.recoverAll();</code>	// Make actor ID 1 fully recover <code>\$gameActors.actor(1).recoverAll();</code>	Game_Battler battler • The one to be fully recovered.	Removes all states and sets HP & MP to maximum.
✓	✓	Recover All (Party)	<code>\$gameParty.members().forEach(function(actor) { actor.recoverAll(); });</code>	-	-	You can use <code>battleMembers</code> or <code>allMembers</code> instead of <code>members</code> .
✓	✓	Recover All (Troop)	<code>\$gameTroop.members().forEach(function(enemy) { enemy.recoverAll(); });</code>	-	-	-
✓	✓	Check if Alive	<code>battler.isAlive()</code>	// Check if enemy #1 (index 0) is still alive & not hidden <code>\$gameTroop.members()[0].isAlive()</code>	Game_Battler battler • The one to check. Returns Boolean • true iff the battler is alive/dead respectively.	If the battler is hidden, these checks will return false. Battlers become hidden when they escape, or (for enemies) at the start of battle if marked <i>Appear Midway</i> .
✓	✓	Check if Dead	<code>battler.isDead()</code>	// Check if the party leader is dead or hidden <code>\$gameParty.members()[0].isDead()</code>	Game_Battler battler • The one to check. Returns Object[] • Array of trait-bearing database objects, e.g. actor, class, state, weapon, armor.	Handy for looking up notetag data. Details of properties available on various database objects can be found on the Database sheet of the MZ Script Call Reference .
✓	✓	Trait Objects	<code>battler.traitObjects()</code>	// Get all trait-bearing objects for the party leader <code>\$gameParty.members()[0].traitObjects()</code>		
Switches & Variables			Check/change switches & variables			▲ Return to top ▲
✓	✓	Get Switch	<code>\$gameSwitches.value(switchId)</code>	// Is switch ID 6 on? <code>\$gameSwitches.value(6)</code>	Number switchId • ID of switch to check. Returns Boolean • true iff the switch is ON.	-
✓	✓	Turn On/Off Switch	<code>\$gameSwitches.setValue(switchId, value);</code>	// Turn on switch ID 1 <code>\$gameSwitches.setValue(1, true);</code>	Number switchId • ID of switch to change.	-
✓	✓			// Turn off switch ID 5 <code>\$gameSwitches.setValue(5, false);</code>	Boolean value • true (ON) or false (OFF).	
✓	✓	Toggle Switch (On ↔ Off)	<code>var id = switchId; \$gameSwitches.setValue(id, !\$gameSwitches.value(id));</code>	// Turn switch 1 on if it is off, off if it is on <code>\$gameSwitches.setValue(1, !\$gameSwitches.value(1));</code>	Number switchId • ID of switch to toggle.	
✓	✓	Get Variable	<code>\$gameVariables.value(variableId)</code>	// Get the value of variable ID 3 <code>\$gameVariables.value(3)</code>	Number variableId • ID of variable to check. Returns Number Any • The current value stored in this variable.	Without scripting, game variables only contain numbers. Scripting can store any value in a variable, but numeric values will be rounded down to an integer by default.
✓	✓	Set Variable	<code>\$gameVariables.setValue(variableId, value);</code>	// Set variable ID 1 equal to 5 <code>\$gameVariables.setValue(1, 5);</code>	Number variableId • ID of variable to change. Any value • Operand value (can be negative).	Variables can store any value, including: • String • Array Numerical results will be rounded down to the nearest integer.
✓	✓			// Set variable ID 12 equal to a random integer 0, 1, 2, or 3 <code>\$gameVariables.setValue(12, Math.randomInt(4));</code>		
✓	✓			// Set variable ID 2 equal to an array of Japanese characters <code>\$gameVariables.setValue(3, ['あ', 'い', 'う', 'え', 'お']);</code>		
✓	✓			// Set variable ID 3 equal to a random value from var 2's array <code>var list = \$gameVariables.value(2); var index = Math.randomInt(list.length); \$gameVariables.setValue(3, list[index]);</code>		
✓	✓	Add to Variable	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) + value);</code>	// +10 to variable ID 1 <code>\$gameVariables.setValue(1, \$gameVariables.value(1) + 10);</code>	Number variableId • ID of variable to change. Number value • Operand (can be negative)	Also works for strings, e.g. • "cheese" + "cake" = "cheesecake"
✓	✓	Subtract from Variable	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) - value);</code>	// -10 to variable ID 1 <code>\$gameVariables.setValue(1, \$gameVariables.value(1) - 10);</code>		-
✓	✓	Multiply Variable	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) * value);</code>	// Multiply variable ID 1 by 5 <code>\$gameVariables.setValue(1, \$gameVariables.value(1) * 5);</code>		-
✓	✓	Divide Variable	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId) / value);</code>	// Divide variable ID 1 by 2 <code>\$gameVariables.setValue(1, \$gameVariables.value(1) / 2);</code>		In JavaScript, div by 0 returns the special value <i>Infinity</i> .
✓	✓	Modulo Variable	<code>\$gameVariables.setValue(variableId, \$gameVariables.value(variableId).mod(value));</code>	// Store the +remainder after dividing variable ID 1 by 12 <code>\$gameVariables.setValue(1, \$gameVariables.value(1).mod(12));</code>		For cyclic/modular arithmetic. Common uses: • Get digits of a number, e.g. mod 10 to get units digit; • Time calculations, e.g. 8 hours after 11 pm.
✓	✓	Set Switch/Variable (Range)	for (var n = id1; n <= id2; n++) { operation }	// Turn on all switches from 1~10 inclusive for (var n = 1; n <= 10; n++) { \$gameSwitches.setValue(n, true); }	Number id1 • Lower end of ID range.	More info about for loops: for loop
				// Store the value 1 in all variables from 5~15 inclusive for (var n = 5; n <= 15; n++) { \$gameVariables.setValue(n, 1); }	Number id2 • Upper end of ID range. operation • Script to run for each ID value.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Get Self Switch	<code>\$gameSelfSwitches.value([mapId, eventId, switchId]);</code>	<code>// Is self-switch B for event 1 on map 2 on? \$gameSelfSwitches.value([2, 1, 'B']);</code>	Number <code>mapId</code> - ID of the event's map. Number <code>eventId</code> - ID of the event. String <code>switchId</code> "A", "B", "C", or "D". Returns Boolean <code>true</code> (ON) or <code>false</code> (OFF).	For the current map, you can use: <code>\$gameMap.mapId()</code>
		Turn On/Off Self Switch	<code>\$gameSelfSwitches.setValue([mapId, eventId, switchId], value);</code>	<code>// In a Script command: turn on self-switch D for "This Event" \$gameSelfSwitches.setValue([this._mapId, this.eventId(), 'D'], true);</code> <code>// Turn off self-switch A for event 5 on map 1 \$gameSelfSwitches.setValue([1, 5, 'A'], false);</code>	Number <code>mapId</code> - ID of the event's map. Number <code>eventId</code> - ID of the event. String <code>switchId</code> "A", "B", "C", or "D". Boolean value <code>true</code> (ON) or <code>false</code> (OFF).	For the map of "This Event", you can use <code>this._mapId</code> (Only in an interpreter context, e.g. a Script command.)
		Turn On/Off Self Switches (Range)	<code>eventIds.forEach(function(eventId) { switchIds.forEach(function(switchId) { \$gameSelfSwitches.setValue([mapId, eventId, switchId], value); }); });</code>	<code>// Turn off all self-switches for events 10–18 on the current map var mapId = this._mapId; var eventIds = [10, 11, 12, 13, 14, 15, 16, 17, 18]; var switchIds = ['A', 'B', 'C', 'D']; eventIds.forEach(function(eventId) { switchIds.forEach(function(switchId) { \$gameSelfSwitches.setValue([mapId, eventId, switchId], false); }); });</code>	Number <code>mapId</code> - ID of the relevant map. Number[] <code>eventIds</code> - Array of event IDs on that map. String[] <code>switchIds</code> - Array of self-switch IDs ("A", "B", "C", and/or "D"). Boolean value <code>true</code> (ON) or <code>false</code> (OFF).	To get the current map ID: <code>\$gameMap.mapId()</code> - Gives the ID of the current map. <code>interpreter._mapId</code> - Gives the ID of the map where the current event is. In a Script command, <code>this</code> = current interpreter.
		Reset All Switches	<code>\$gameSwitches.clear();</code>	-	-	Turns off all switches.
		Reset All Self Switches	<code>\$gameSelfSwitches.clear();</code>	-	-	Turns off all self-switches.
		Reset All Variables	<code>\$gameVariables.clear();</code>	-	-	Resets all variables to 0 (or, technically, <i>undefined</i>).
		Events & Interpreters	<i>Call/trigger events, interact with event interpreters</i>			▲ Return to top ▲
		Wait (Interpreter)	<code>interpreter.wait(frames);</code>	<code>// In a Script command: wait for 60–300 frames (random) this.wait(60 + Math.randomInt(241));</code>	Game Interpreter <code>interpreter</code> - Interpreter to pause (<i>this</i> in a Script command). Number <code>frames</code> - Wait duration, in frames.	This wait time applies between event commands. If you want to wait in the middle of a Script command, split it into 2+ commands so the wait time can apply.
		This Event's ID	<code>interpreter.eventId()</code>	<code>// In a Script command: get the ID of This Event this.eventId()</code>	Game Interpreter <code>interpreter</code> - Interpreter reference (<i>this</i> in a Script command). Returns Number - The corresponding ID.	-
		This Event's Map	<code>interpreter._mapId</code>	<code>// In a Script command: get the map ID of This Event this._mapId</code>		This might be different from <code>\$gameMap.mapId()</code> , e.g. a non-Parallel event that has transferred the player to another map part way through.
		Common Event (Insert)	<code>interpreter.command117([commonEventId]);</code>	<code>// MZ, anywhere: insert common event 5 into the map interpreter \$gameMap._interpreter.command117([5]);</code>	Game Interpreter <code>interpreter</code> - Target interpreter (<i>this</i> in a Script command). Number <code>commonEventId</code> - ID of common event to insert.	Adds the common event's commands to the target interpreter. The common event's contents can refer to properties of "This Event".
			<code>interpreter._params = [commonEventId]; interpreter.command117();</code>	<code>// MV, in a Script command: insert CE with ID = variable 1's value this._params = [\$gameVariables.value(1)]; this.command117();</code>		
		Common Event (Reserve)	<code>\$gameTemp.reserveCommonEvent(id);</code>	<code>// Queue common event 3 to run at the next opportunity \$gameTemp.reserveCommonEvent(3);</code>	Number <code>commonEventId</code> ID of common event to reserve.	MV can only reserve one common event at a time. Only the most recent reservation will be retained. MZ runs reserved events in the order they were queued. The reservation queue is not included when the player saves the game: be careful about queuing events on scenes with no interpreter (e.g. menu).
		Clear Reserved Common Event(s)	<code>\$gameTemp._commonEventQueue = [];</code>	-	-	This will not affect any common events that have already started.
			<code>\$gameTemp.clearCommonEvent();</code>	-	-	
		Trigger Map Event	<code>event.start();</code>	<code>// Remote-trigger map event 7 \$gameMap.event(7).start();</code>	Game Event <code>event</code> The event to trigger.	The event will start at the next opportunity.
		Call Map Event (Current Page)	<code>var self = interpreter; var list = event.page().list; if (list) { var id = self.isOnCurrentMap() ? self.eventId() : 0; self.setupChild(list, id); }</code>	<code>// In a Script call: insert map event 7's current page commands var list = \$gameMap.event(7).page().list; var id = this.isOnCurrentMap() ? this.eventId() : 0; this.setupChild(list, id);</code>	Game Interpreter <code>interpreter</code> - Interpreter reference (<i>this</i> in a Script command) Game Event <code>event</code> - Event to call.	These are basically just map event versions of the Common Event command.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Call Map Event (Specific Page)	<pre>var self = interpreter; var page = event.event().pages[pageIndex]; if (page && page.list) { var id = self.isOnCurrentMap() ? self.eventId() : 0; self.setupChild(page.list, id); }</pre>	<pre>// In a Script call: add commands from page 2 of map event 7 var page = \$gameMap.event(7).event().pages[1]; if (page) { var id = this.isOnCurrentMap() ? this.eventId() : 0; this.setupChild(page.list, id); }</pre>	Game Interpreter interpreter - Interpreter reference (this in a Script command). Game Event event - Event to call. Number pageIndex - Index of event page to call, use 0 for page #1.	To call a map event that is on a different map, consider a common event or a plugin instead.
		Call Event Command	interpreter.commandCode(parameters);	<pre>// In a Script command: invoke command "Common Event: 4" this.command117([4]);</pre>	Game Interpreter interpreter - Interpreter reference (this in a Script command). Function commandCode - The method for the appropriate command. - Replace Code with the 3-digit command code. Any[] parameters - Array of command parameters.	This is here for completeness. Consider using the script calls listed in the Commands section instead.
			<pre>var self = interpreter; self._params = parameters; self.commandCode();</pre>	<pre>// In a Script command: invoke command "Jump to Label: win" this._params = ["win"]; this.command119();</pre>		
		Insert Event Command(s)	<pre>var self = interpreter; var pre = self._list.slice(0, self._index + 1); var post = self._list.slice(index + 1); var com = commands; self._list = pre.concat(com, post);</pre>	<pre>// In a Script command: insert Show Text, Wait, Show Text var pre = this._list.slice(0, this._index + 1); var post = this._list.slice(this._index + 1); var i = this._indent; var com = [{ code: 101, indent: i, parameters: ['Actor1', 1, 0, 0] }, { code: 401, indent: i, parameters: ['Mwahahaha! HAHAAHAHA!'] }, { code: 230, indent: i, parameters: [120] }, { code: 101, indent: i, parameters: ['Actor1', 1, 0, 0] }, { code: 401, indent: i, parameters: ['...well, this is awkward.'] }]; this._list = pre.concat(com, post);</pre>	Game Interpreter interpreter - Interpreter reference (this in a Script command). (command[]) commands - Array of objects, each with these properties: code: event command code. indent: command indent, used for branches. parameters: command parameters.	This deliberately creates a new command list. Simply inserting new commands into the current interpreter list will mutate the current event's database record for the rest of the play session. The RPG Maker MZ Script Call Reference has a list of event command codes and parameters. Most of these also apply to RPG Maker MV.
		Events	Some script calls for map events			
		Erase Event	\$gameMap.eraseEvent(eventId);	<pre>// Erase map event 8 \$gameMap.eraseEvent(8);</pre>	Number eventId ID of the event to erase.	Only effective for map events.
			event.erase();	<pre>// Erase map event 8 \$gameMap.event(8).erase();</pre>	Game Event event Event to erase.	
		Undo Erase Event	event._erased = false;	<pre>// Un-erase map event 1 \$gameMap.event(1)._erased = false;</pre>		An erased event is effectively on page -1 (inactive). Erased events will return after going to a different map.
		Is Event Erased?	event._erased	<pre>// Is map event 1 currently erased? \$gameMap.event(1)._erased</pre>	Game Event event Event to check.	
		Active Event Page	event.page()	<pre>// Get the page data for the active page of map event 2 \$gameMap.event(2).page()</pre>	Game Event event - Event to check. Returns (event.page) - Static event page data.	-
		Event Command List	event.list()	<pre>// Get the commands for the active page of map event 3 \$gameMap.event(3).list()</pre>	Game Event event - Event to check. Returns (event.command[]) - List of event command objects.	-
		Event Trigger	event_trigger	<pre>// Get the current trigger for map event 6 \$gameMap.event(6)._trigger</pre>	Game Event event - The event to check. Returns Number 0 (Action Button), 1 (Player Touch), 2 (Event Touch), 3 (Autorun), or 4 (Parallel).	These values reset when the event changes page.
		Event Priority	event._priorityType	<pre>// Get the current priority for map event 6 \$gameMap.event(6)._priorityType</pre>	Game Event event - The event to check. Returns Number 0 (below), 1 (same as characters), or 2 (above).	
		Set Event Priority	event.setPriorityType(priorityType);	<pre>// Set map event 6 to Above characters priority \$gameMap.event(6).setPriorityType(2);</pre>	Game Event event - The event to change. Number priorityType 0 (below), 1 (same as characters), or 2 (above).	
		Inputs	Check & simulate player inputs			
		Button	Keyboard/gamepad input checks			
		Button Triggered	Input.isTriggered(buttonName)	<pre>// True iff the "ok" button is down this frame but was up last frame Input.isTriggered("ok")</pre>	String buttonName - Name of the button input, e.g. "ok" Returns Boolean - true iff condition is met for this input.	Default button names: "down", "left", "right", "up", "ok", "cancel", "shift", "pageup", "pagedown", "menu", "escape", "control", "tab", "debug". "escape" is special: it can trigger "cancel" or "menu". <i>Input.keyMapper</i> maps key codes to names. <i>Input.gamepadMapper</i> maps gamepad inputs to names.
		Button Pressed	Input.isPressed(buttonName)	<pre>// True iff the "ok" button is down this frame Input.isPressed("ok")</pre>		
		Button Long-Pressed	Input.isLongPressed(buttonName)	<pre>// True iff the "ok" button been held for 24+ frames without release Input.isLongPressed("ok")</pre>		
		Button Repeat Signal	Input.isRepeated(buttonName)	<pre>// True every 6th frame while "ok" is long-pressed Input.isRepeated("ok")</pre>		

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Touch	Mouse/touch input checks			
✓	✓	Touch Triggered	TouchInput.isTriggered()		Returns Boolean • true iff condition is met.	True iff touch pressed this frame, but not last frame.
✓	✓	Touch Pressed	TouchInput.isPressed()			True iff touch pressed this frame.
✓	✓	Touch Long-Pressed	TouchInput.isLongPressed()			True iff touch has been held for 24+ frames.
✓	✓	Touch Repeat Signal	TouchInput.isRepeated()			True every 6th frame while long-pressed.
✓	✓	Touch Cancelled	TouchInput.isCancelled()			True iff touch cancelled this frame: RMB or multi-touch.
✓	✓	Touch Released	TouchInput.isReleased()			True iff touch released this frame.
☐	✓	Touch Clicked	TouchInput.isClicked()			True iff touch pressed; not moved; released this frame.
✓	✓	Touch Moved + Triggered	TouchInput.isMoved()	-		True iff touch pressed somewhere new this frame.
☐	✓	Touch Moved, No Trigger	TouchInput.isHovered()	-		True iff touch coordinates changed this frame, no press.
✓	✓	Touch Screen X	TouchInput.x	-	Returns Number • X/Y screen coordinate.	Screen coordinates (px) of touch/mouse input.
✓	✓	Touch Screen Y	TouchInput.y			
✓	✓	Wheel X	TouchInput.wheelX	// Was the wheel scrolled vertically in the + direction this frame? TouchInput.wheelY >= 20	Returns Number • Amount of wheel scroll distance this frame.	Wheel scroll increments are not a defined standard. The core scripts: • Use a threshold of ±20 (like the example); • Scroll down in response to a positive wheelY value; • Define, but do not use, wheelX.
✓	✓	Wheel Y	TouchInput.wheelY			
		Simulate input	Simulate button/touch inputs			
☐	✓	Virtual Input	Input.virtualClick(inputName);	// Simulate a press of the OK button Input.virtualClick('ok');	String inputName • Game label for the input to simulate.	Simulates button input for 1 frame.
✓	✓		Input._currentState[inputName] = true;	// Simulate a press of the OK button Input._currentState['ok'] = true;		
✓	✓	Set Player Move Destination	\$gameTemp.setDestination(x, y);	// Simulate a player click to move to tile (4, 5) \$gameTemp.setDestination(4, 5);	Number x, y • Target tile coordinates.	Marks the tile as the player's current move destination. Additional move input during motion will interrupt this.
☐	✓	Show Framerate Widget (F2)	Graphics._switchFPSCounter();	-	-	Same effect as pressing F2 during play.
✓	☐		Graphics._switchFPSMeter();			
✓	✓	Toggle Stretch (F3)	Graphics._switchStretchMode();	-	-	Same effect as pressing F3 during play.
✓	✓	Toggle Full-Screen (F4)	Graphics._switchFullScreen();	-	-	Same effect as pressing F4 during play.
☐	✓	Reload Game (F5)	SceneManager.reloadGame();	-	-	Same effect as pressing F5 during play.
✓	✓		window.location.reload();			
☐	✓	Show Console (F8)	SceneManager.showDevTools();	-	-	Same effect as pressing F8 during play.
✓	☐		if (Utils.isNwjs() && Utils.isOptionValid('test')) { require('nw.gui').Window.get().showDevTools(); }			
Characters			Get map characters: player, followers, events, vehicles			▲ Return to top ▲
✓	✓	Player	\$gamePlayer	-	Returns Game_Player • The player map character.	This is map character data, not actor data.
✓	✓	Follower	\$gamePlayer.followers().follower(index)	// Get follower #1 (index 0) \$gamePlayer.followers().follower(0)	Number index • Follower index: 0 = first follower. Returns Game_Follower • Corresponding follower.	
☐	✓	All Followers	\$gamePlayer.followers().data()	-	Returns Game_Follower[] • Array of all followers.	
✓	✓		\$gamePlayer.followers()._data			
☐	✓	Repeat for all Followers	for (const follower of \$gamePlayer.followers().data()) { operation }	// Make all followers face down (direction 2) for (const follower of \$gamePlayer.followers().data()) { follower.setDirection(2); }	operation • Script to run for each value of follower.	More information: for...of.
✓	☐		\$gamePlayer.followers().forEach(function(follower) { operation });	// Make all followers jump on the spot \$gamePlayer.followers().forEach(function(follower) { follower.jump(0, 0); });		This forEach is a method of Game_Followers, not Array.
✓	✓	Event	\$gameMap.event(eventId)	// Get map event ID 2 \$gameMap.event(2)	Number eventId • ID of the event. Returns Game_Event • Corresponding map event.	-
✓	✓	Vehicle	\$gameMap.vehicle(vehicleType)	// Get the boat vehicle, i.e. \$gameMap.boat() \$gameMap.vehicle('boat')	String Number vehicleType • "boat", "ship", or "airship"; OR	-
✓	✓	Boat	\$gameMap.boat()			-

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Ship	\$gameMap.ship()	-	• 0 (boat), 1 (ship), or 2 (airship). Returns <u>Game_Vehicle</u>	
✓	✓	Airship	\$gameMap.airship()	-	• Corresponding vehicle.	
✓	✓	Driven Vehicle	\$gamePlayer.vehicle()	-		Gets the current vehicle, or null if not in a vehicle.
✓	✓	Character / This Event	interpreter.character(characterId)	// In a Script command: get the player character this.character(-1)	<u>Number</u> characterId	Useful for getting "This Event".
✓	✓			// In a Script command: get "This Event" this.character(0)	• -1 (player), 0 (This Event), or 1+ (map event ID) Returns <u>Game_Character</u>	
✓	✓			// In a Script command: get map event ID 7 this.character(7)	• The corresponding character.	
✓	✓	Move Route Character (in a Move Route)	this	-	Returns <u>Game_Character</u> • The character being moved.	For use in a <i>Game_Character</i> context, e.g. a Script move route command.
✓	✓	Events at Position	\$gameMap.eventsXy(x, y)	// Get an array of all events at (7, 8) \$gameMap.eventsXy(7, 8)	<u>Number</u> x, y • Map coordinates of tile to check. Returns <u>Game_Event[]</u> • Array of events on that tile.	-
				// Get an array of names for all events at (7, 8) \$gameMap.eventsXy(7, 8).map(function(ev) { return ev.event().name; })		This example uses the Array#map method.
✓	✓	Non-Through Events at Position	\$gameMap.eventsXyNt(x, y)	// Get an array of all events at (7, 8) with Through OFF \$gameMap.eventsXyNt(7, 8)		-
✓	✓	Tile Events at Position	\$gameMap.tileEventsXy(x, y)	// Get an array of all tile events at (9, 10) \$gameMap.tileEventsXy(9, 10)		An event is a tile event if it has a tile image and is set to Below Characters. They are considered for passability.
✓	✓	Event Editor Reference	event.event()	// Does map event ID 5 have an <npc> tag in its Note field? \$gameMap.event(5).event().meta.npc	<u>Game_Event event</u> • The one to check. Returns <u>\$dataEvent</u> • Editor record for this event.	Event record structure can be explored via the console, e.g. • <i>\$gameMap.event(1).event()</i> The console tab is found on the dev tools window, opened by pressing F8 (or F12 if F8 fails) during playtest.
Character Properties			Get/set properties (location, opacity, etc) for map characters			▲ Return to top ▲
✓	✓	Character Position	character.x	// Get the player's x tile coordinate \$gamePlayer.x	<u>Game_Character character</u> • The one to check. Returns <u>Number</u> • The appropriate map coordinate.	-
✓	✓		character.y	// Get the y tile coordinate of follower #1 (index 0) \$gamePlayer.followers().follower(0).y		
✓	✓	Character Sprite Position	character._realX	// Get the x tile coordinate of the player's map sprite \$gamePlayer._realX		These coordinates will not necessarily be integers.
✓	✓		character._realY	// Get the y tile coordinate of the sprite of map event 2 \$gameMap.event(2)._realY		
✓	✓	Is Character Here?	character.pos(x, y)	// Is the player at tile (5, 6)? \$gamePlayer.pos(5, 6)	<u>Game_Character character</u> • The one to check. <u>Number</u> x, y • Tile coordinates to check. Returns <u>Boolean</u> • true iff they are at this location.	-
✓	✓	Direction	character.direction()	// Get the player's current facing direction \$gamePlayer.direction()	<u>Game_Character character</u> • The one to check. Returns <u>Number</u> • 2 (down), 4 (left), 6 (right), or 8 (up)	-
✓	✓	Region ID	character.regionId()	// Get the player's current region ID \$gamePlayer.regionId()	<u>Game_Character character</u> • The one to check.	-
✓	✓	Terrain tag	character.terrainTag()	// Get the player's current terrain tag \$gamePlayer.terrainTag()	Returns <u>Number</u> • The region ID (map) / terrain tag (tileset).	-
✓	✓	Is on a Ladder Tile?	character.isOnLadder()	// Is the player currently on a ladder tile? \$gamePlayer.isOnLadder()	<u>Game_Character character</u> • The one to check.	-
✓	✓	Is on a Bush Tile?	character.isOnBush()	// Is the player currently on a bush tile? \$gamePlayer.isOnBush()	Returns <u>Boolean</u> • true iff the character is on that tile type.	-
✓	✓	Is a Tile Character?	character.isTile()	// Is map event 3 being treated as a tile? \$gameMap.event(3).isTile()	<u>Game_Character character</u> • The one to check. Returns <u>Boolean</u> • true iff the character is a tile character.	Tile characters are ones that have Below Characters priority and use a tile image. They are considered for passability.
✓	✓	Opacity	character.opacity()	// Get the opacity of map event 15 \$gameMap.event(15).opacity()	<u>Game_Character character</u> • The one to check. Returns <u>Number</u> • 0 (fully transparent) to 255 (fully opaque).	Returns a value from 0 (fully transparent) to 255 (fully opaque).
✓	✓	Change Opacity	character.setOpacity(opacity)	// Set the player to full opacity (0% transparent) \$gamePlayer.setOpacity(255);	<u>Game_Character character</u> • The one to affect.	Note that by default map followers have many properties

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Change Opacity	<code>character.setOpacity(opacity);</code>	<code>// Set event 5 to 128 opacity (~50% transparent)</code> <code>\$gameMap.event(5).setOpacity(128);</code>	Number opacity • 0 to 255, where 0 is fully transparent.	updated to match the player (leader), including opacity.
✓	✓	Pattern (a.k.a. Stepping Frame)	<code>character.pattern()</code>	<code>// Get the current pattern for map event 5</code> <code>\$gameMap.event(5).pattern()</code>	Game_Character character • The one to check. Returns Number • 0 (left), 1 (middle), 2 (right), or 3 (middle).	<code>_pattern</code> ranges from 0~3: when <code>_pattern</code> is 3, <code>pattern()</code> is 1, due to ping-pong animation.
✓	✓	Raw Pattern	<code>character._pattern</code>	<code>// Get the current raw pattern value for map event 5</code> <code>\$gameMap.event(5)._pattern</code>		
✓	✓	Set Pattern	<code>character.setPattern(pattern);</code>	<code>// Set the player's current pattern to 2 (step right)</code> <code>\$gamePlayer.setPattern(2);</code>	Game_Character character • The one to change. Number pattern • New pattern value.	This call is of limited use because pattern resets when: • stopped for sufficient time; or • stepping/walking animation frame changes. For long event animations, see Set Original Pattern .
✓	✓	Original Pattern (for Map Events)	<code>event._originalPattern</code>	<code>// Get the starting pattern for the current page of map event 5</code> <code>\$gameMap.event(5)._originalPattern</code>	Game_Event event • The one to check. Returns Number • 0 (left), 1 (middle), or 2 (right).	The starting pattern for the event's current page. Whenever an event is stopped and its stepping animation is disabled, it resets its <code>pattern</code> to this value. Other characters use a value of 1 for this.
✓	✓	Set Original Pattern (for Map Events)	<code>var ev = event;</code> <code>ev._originalPattern = pattern;</code> <code>ev.resetPattern();</code>	<code>// Set the original pattern of map event 10 to 0 (step left)</code> <code>var ev = \$gameMap.event(10);</code> <code>ev._originalPattern = 0; ev.resetPattern();</code>	Game_Event event • The one to change. Number pattern • New original pattern value.	<code>resetPattern</code> resets the current pattern to the new original pattern value. It can be omitted if you like.
✓	✓	Move Speed	<code>character.realMoveSpeed()</code>	<code>// Get the current move speed of map event 2</code> <code>\$gameMap.event(2).realMoveSpeed()</code>	Game_Character character • The one to check. Returns Number • The move speed; typically an integer 1~6.	Includes +1 speed modifier when dashing. By default only the player (and followers) can dash. Current walking move speed. Default player walk speed is 4.
✓	✓	Base Move Speed	<code>character.moveSpeed()</code>			
✓	✓	Change Speed	<code>character.setMoveSpeed(speed);</code>	<code>// Set the player's walking move speed to 3.5</code> <code>\$gamePlayer.setMoveSpeed(3.5);</code>	Game_Character character • The one to affect. Number speed • New move speed; can be a decimal, e.g. 3.5.	Sets a character's walking move speed.
✓	✓	Move Frequency	<code>character.moveFrequency()</code>	<code>// Get the current move frequency for map event 4</code> <code>\$gameMap.event(4).moveFrequency()</code>	Game_Character character • The one to check. Returns Number • The move frequency; typically an integer 1~5.	Frequency affects time between moves for map events.
✓	✓	Change Frequency	<code>character.setMoveFrequency(frequency);</code>	<code>// Set the move frequency of map event 4 to Highest (5)</code> <code>\$gameMap.event(4).setMoveFrequency(5);</code>	Game_Character character • The one to affect. Number frequency • New frequency value; typically an integer 1~5.	
✓	✓	Walking Animation ON	<code>character.setWalkAnime(true);</code>	<code>// Disable walking animation for the player</code> <code>\$gamePlayer.setWalkAnime(false);</code>	Game_Character character • The one to affect.	Walk/step animation rate is based on move speed. Stepping animation happens when not moving. Walking animates +50% faster than stepping.
✓	✓	Walking Animation OFF	<code>character.setWalkAnime(false);</code>			
✓	✓	Stepping Animation ON	<code>character.setStepAnime(true);</code>	<code>// Enable stepping animation for the player</code> <code>\$gamePlayer.setStepAnime(true);</code>		
✓	✓	Stepping Animation OFF	<code>character.setStepAnime(false);</code>			
✓	✓	Direction Fix ON	<code>character.setDirectionFix(true);</code>	<code>// Lock the player's direction until further notice</code> <code>\$gamePlayer.setDirectionFix(true);</code>		-
✓	✓	Direction Fix OFF	<code>character.setDirectionFix(false);</code>	<code>// Unlock the player's direction</code> <code>\$gamePlayer.setDirectionFix(false);</code>		-
✓	✓	Through ON	<code>character.setThrough(true);</code>	<code>// Enable through for map event 10, i.e. ignore collisions</code> <code>\$gameMap.event(10).setThrough(true);</code>		-
✓	✓	Through OFF	<code>character.setThrough(false);</code>	<code>// Disable through for map event 10</code> <code>\$gameMap.event(10).setThrough(false);</code>		-
✓	✓	Transparent ON	<code>character.setTransparent(true);</code>	<code>// Make map event 4 invisible until further notice</code> <code>\$gameMap.event(4).setTransparent(true);</code>		This will not change the character's opacity value.
✓	✓	Transparent OFF	<code>character.setTransparent(false);</code>	<code>// Make map event 4 visible again</code> <code>\$gameMap.event(4).setTransparent(false);</code>		
✓	✓	Is Walking Animation On?	<code>character.hasWalkAnime()</code>	<code>// Is the player direction-fixed?</code> <code>\$gamePlayer.isDirectionFixed()</code>	Game_Character character • The one to check. Returns Boolean • true iff that property is active for that character.	-
✓	✓	Is Stepping Animation On?	<code>character.hasStepAnime()</code>			
✓	✓	Is Direction Fix On?	<code>character.isDirectionFixed()</code>			
✓	✓	Is Through On?	<code>character.isThrough()</code>			
✓	✓	Is Transparent On?	<code>character.isTransparent()</code>			
✓	✓	Show Map Followers	<code>\$gamePlayer.showFollowers();</code> <code>\$gamePlayer.refresh();</code>			makes party follower characters invisible on the map. vworks independently of <code>setTransparent</code> . Each frame, followers update to match <code>\$gamePlayer</code> : • move speed

Script Calls			Last updated 2023-12-14 (01:50 UTC)			
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Hide Map Followers	<code>\$gamePlayer.hideFollowers();</code> <code>\$gamePlayer.refresh();</code>			• opacity • blend mode • walk/step animation on/off • direction fix on/off • transparent on/off
✓	✓	Are Followers Shown?	<code>\$gamePlayer.followers().isVisible()</code>	-	Returns Boolean • true iff map followers are currently shown.	-
✓	✓	Gather Followers (Wait)	<code>\$gamePlayer.gatherFollowers();</code> interpreter .setWaitMode('gather');	// In a Script command <code>\$gamePlayer.gatherFollowers();</code> <code>this.setWaitMode('gather');</code>	Game Interpreter interpreter • Interpreter to pause (this in a Script command).	Gathering will stop when all followers are on the same tile as the player.
✓	✓	Gather Followers (No Wait)	<code>\$gamePlayer.gatherFollowers();</code>	-	-	
✓	✓	Are Followers Gathering?	<code>\$gamePlayer.followers().areGathering()</code>	-	Returns Boolean • true iff map followers are currently gathering.	
✓	✓	Change Character Image	character .setImage(name , index);	// Change map event 7 to use index 0 of sheet <i>Actor1_1.png</i> <code>\$gameMap.event(7).setImage('Actor1_1', 0);</code>	Game Character character • The one to affect. String name • Name of the spritesheet to use. Number index • Index to use (0 for top-left).	-
✓	✓	Character Image Name	character .characterName()	// Get the name of the spritesheet used by the player <code>\$gamePlayer.characterName()</code>	Game Character character • The one to check.	This is for map characters. To check this data for a specific actor, see Actor Map Character Name .
✓	✓	Character Image Index	character .characterIndex()	// Get the index of the player's sprite in their spritesheet <code>\$gamePlayer.characterIndex()</code>	Returns String Number • The filename (no extension) or sprite index.	
✓	✓	Change Vehicle Images	vehicle .setImage(filename , index);	// Set the airship to use character #3 (i.e. index 2) from <i>Bird.png</i> <code>\$gameMap.airship().setImage('Bird', 2);</code>	Game Vehicle vehicle • The vehicle to affect. String filename • Spritesheet filename, without extension. Number index • Position of the character in its file, 0–7.	-
✓	✓	Vehicle Image Name	vehicle .characterName()	// Get the name of the spritesheet the boat is using <code>\$gameMap.boat().characterName()</code>	Game Vehicle vehicle • The one to check.	-
✓	✓	Vehicle Image Index	vehicle .characterIndex()	// Get the sprite index used by the airship <code>\$gameMap.airship().characterIndex()</code>	Returns String Number • The filename (no extension) or sprite index.	-
✓	✓	Set Character Tile Image	character .setTileImage(tileId);	// Set map event 3 to use tile ID 1 <code>\$gameMap.event(3).setTileImage(1);</code>	Game Character character • The one to change. Number tileId • ID of the tile to use: 0 = first (blank) B tile.	-
✓	✓	Change Blend Mode	character .setBlendMode(blendMode);	// Set map event 11 to use Additive blend mode <code>\$gameMap.event(11).setBlendMode(1);</code>	Game Character character • The one to affect. Number blendMode • 0 (Normal), 1 (Additive), 2 (Multiply), or 3 (Screen).	-
✓	✓	Is in Specific Vehicle?	<code>\$gamePlayer.vehicle() === vehicle</code>			
✓	✓	Is Boat Driven?	<code>\$gamePlayer.isInBoat()</code>	// Two different ways to check if the player is in the ship vehicle <code>\$gamePlayer.isInShip()</code>	Game Vehicle vehicle • The vehicle to check.	-
✓	✓	Is Ship Driven?	<code>\$gamePlayer.isInShip()</code>	<code>\$gamePlayer.vehicle() === \$gameMap.ship()</code>	Returns Boolean • true iff the player is in that vehicle.	
✓	✓	Is Airship Driven?	<code>\$gamePlayer.isInAirship()</code>			
Character Movement			Map character movement and passability			▲ Return to top ▲
✓	✓	Move Down	character .moveStraight(direction);	// Make the player move 1 tile to the left <code>\$gamePlayer.moveStraight(4);</code>	Game Character character • The one to move. Number direction • 2 (down), 4 (left), 6 (right), or 8 (up).	-
✓	✓	Move Left				
✓	✓	Move Right				
✓	✓	Move Up				
✓	✓	Move Lower Left	character .moveDiagonally(horz , vert);	// Make map event 3 move 1 tile to the upper-left <code>\$gameMap.event(3).moveDiagonally(4, 8);</code>	Game Character character • The one to move. Number horz • Horizontal move direction: 4 (left) or 6 (right). Number vert • Vertical move direction: 2 (down) or 8 (up).	-
✓	✓	Move Lower Right				
✓	✓	Move Upper Left				
✓	✓	Move Upper Right				
✓	✓	Move at Random	character .moveRandom();	// Make the player move 1 tile in a random available direction <code>\$gamePlayer.moveRandom();</code>	Game Character character • The one to move.	-
✓	✓	Move Toward Player	character .moveTowardPlayer();	// Make map event 5 move 1 tile toward the player <code>\$gameMap.event(5).moveTowardPlayer();</code>	Game Character character • The one to move.	
✓	✓	Move Away From Player	character .moveAwayFromPlayer();	// Make map event 5 move 1 tile away from the player <code>\$gameMap.event(5).moveAwayFromPlayer();</code>		These commands do not use pathfinding.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Move Toward Character	<code>character1.moveTowardCharacter(character2);</code>	<code>// Make map event 2 move 1 tile towards map event 3 \$gameMap.event(2).moveTowardCharacter(\$gameMap.event(3));</code>	<u>Game_Character</u> character1 • The one to move.	These commands do not use pathfinding.
✓	✓	Move Away From Character	<code>character1.moveAwayFromCharacter(character2);</code>	<code>// Make the player move 1 tile away from event 5 \$gamePlayer.moveAwayFromCharacter(\$gameMap.event(5));</code>	<u>Game_Character</u> character2 • Target character.	
✓	✓	Move Forwards	<code>character.moveForward();</code>	<code>// Make map event 12 move 1 tile in the direction it is facing \$gameMap.event(12).moveForward();</code>	<u>Game_Character</u> character • The one to move.	-
✓	✓	Move Backwards	<code>character.moveBackward();</code>	<code>// Make the player move 1 tile backwards \$gamePlayer.moveBackward();</code>		Locks the character's facing direction for the move.
✓	✓	Jump	<code>character.jump(xPlus, yPlus);</code>	<code>// Make the player jump 5 tiles right \$gamePlayer.jump(5, 0);</code>	<u>Game_Character</u> character • The one to move.	Jumping ignores all passability checks.
✓	✓			<code>// Make map event 1 jump 2 tiles up \$gameMap.event(1).jump(0, -2);</code>	<u>Number</u> xPlus • Horizontal jump distance, use negative to go left. <u>Number</u> yPlus • Vertical jump distance, use negative to go up.	
✓	✓	Wait (Movement)	<code>character._waitCount = frames - 1;</code>	<code>// Make map event 20 wait for 60 frames \$gameMap.event(20)._waitCount = 59;</code>	<u>Game_Character</u> character • The one to affect. <u>Number</u> frames • Wait duration.	This is for move route waiting. For waiting between event commands, see Wait (Interpreter) .
✓	✓	Turn Down	<code>character.setDirection(direction);</code>	<code>// Make the player face right \$gamePlayer.setDirection(6);</code>	<u>Game_Character</u> character • The one to turn.	-
✓	✓	Turn Left			<u>Number</u> direction • 2 (down), 4 (left), 6 (right), or 8 (up).	
✓	✓	Turn Right				
✓	✓	Turn Up				
✓	✓	Turn 90° Right	<code>character.turnRight90();</code>	<code>// Make the player turn 90° left \$gamePlayer.turnLeft90();</code>		
✓	✓	Turn 90° Left	<code>character.turnLeft90();</code>		<u>Game_Character</u> character • The one to turn.	-
✓	✓	Turn 180°	<code>character.turn180();</code>	<code>// Make map event 2 turn 90° left or right (random) \$gameMap.event(2).turnRightOrLeft90();</code>		
✓	✓	Turn 90° Right or Left	<code>character.turnRightOrLeft90();</code>			
✓	✓	Turn at Random	<code>character.turnRandom();</code>	<code>// Make the player face a random direction \$gamePlayer.turnRandom();</code>	<u>Game_Character</u> character • The one to turn.	-
✓	✓	Turn Toward Player	<code>character.turnTowardPlayer();</code>	<code>// Make map event 15 face the player \$gameMap.event(15).turnTowardPlayer();</code>	<u>Game_Character</u> character • The one to turn.	These commands do not use pathfinding.
✓	✓	Turn Away From Player	<code>character.turnAwayFromPlayer();</code>	<code>// Make map event 15 face away from the player \$gameMap.event(15).turnAwayFromPlayer();</code>		
✓	✓	Turn Toward Character	<code>character1.turnTowardCharacter(character2);</code>	<code>// Make the player look at map event 1 \$gamePlayer.turnTowardCharacter(\$gameMap.event(1));</code>	<u>Game_Character</u> character1 • The one to turn.	
✓	✓	Turn Away From Character	<code>character1.turnAwayFromCharacter(character2);</code>	<code>// Make the player look away from map event 1 \$gamePlayer.turnAwayFromCharacter(\$gameMap.event(1));</code>	<u>Game_Character</u> character2 • Target character.	
✓	✓	Relocate Player	<code>\$gamePlayer.reserveTransfer(mapId, x, y, direction, fadeType);</code>	<code>// Transfer to (5, 10) on map 4, no direction change, fade Black \$gamePlayer.reserveTransfer(4, 5, 10, 0, 0);</code>	<u>Number</u> mapId • Destination map ID.	-
✓	✓			<code>// Transfer to (4, 15) on this map, face up, fade type None \$gamePlayer.reserveTransfer(\$gameMap.mapId(), 4, 15, 8, 2);</code>	<u>Number</u> x, y • Destination map coordinates.	
✓	✓			<code>// Transfer to [map, x, y] = variables [1, 2, 3], fade White \$gamePlayer.reserveTransfer(\$gameVariables.value(1), \$gameVariables.value(2), \$gameVariables.value(3), 0, 1);</code>	<u>Number</u> direction • 0 (no change), 2 (down), 4 (left), 6 (right), or 8 (up) <u>Number</u> fadeType • 0 (Black), 1 (White), or 2 (No Fade)	
✓	✓	Relocate Vehicle	<code>vehicle.setLocation(mapId, x, y);</code>	<code>// Place boat at (7, 5) on this map \$gameMap.boat().setLocation(\$gameMap.mapId(), 7, 5);</code>	<u>Game_Vehicle</u> vehicle • The vehicle to relocate.	There is only one copy of each vehicle.
✓	✓			<code>// Place ship at (20, 8) on map 2 \$gameMap.ship().setLocation(2, 20, 8);</code>	<u>Number</u> mapId • Destination map ID.	
✓	✓			<code>// Place airship at coordinates [map, x, y] = variables [1, 2, 3] var mapId = \$gameVariables.value(1); var x = \$gameVariables.value(2); var y = \$gameVariables.value(3); \$gameMap.airship().setLocation(mapId, x, y);</code>	<u>Number</u> x, y • Destination map coordinates.	
✓	✓	Relocate Character	<code>character.locate(x, y);</code>	<code>// Place map event 2 at (3, 4) \$gameMap.event(2).locate(3, 4);</code>	<u>Game_Character</u> character • The one to relocate.	-
✓	✓			<code>// In a Script command: place "This Event" at (5, 6) this.character(0).locate(5, 6);</code>	<u>Number</u> x, y • Destination map coordinates.	
✓	✓			<code>// In a move route: place "this character" at (7, 8) this.locate(7, 8);</code>		
✓	✓	Swap Character Locations	<code>character1.swap(character2);</code>	<code>// Swap the locations of the player and map event 7 \$gamePlayer.swap(\$gameMap.event(7));</code>	<u>Game_Character</u> character1, character2 • The ones to swap locations.	
✓	✓	Get On/Off Vehicle	<code>\$gamePlayer.getOnOffVehicle();</code>	-	-	-

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Set Movement Route (No Wait)	<code>character.forceMoveRoute(moveRoute);</code>	<code>// MV, in a Script command // Fade out "This Event" by -10 opacity/frame, wait for completion var list = []; var char = this._character = this.character(0); var cOpA = Game_Character.ROUTE_CHANGE_OPACITY; for (var p = char.opacity(); p > 0; p -= 10) { list.push({ code: cOpA, parameters: [p] }); }</code>	Game_Interpreter interpreter - Interpreter to pause (this in a Script command). Number characterId -1 (player), 0 (This Event), or 1+ (map event ID). Game_Character character - The one to move. Object moveRoute - An object with the following properties: list: an array of move command objects (required). repeat: if true, the route will repeat. skippable: if true, skip failed move commands. - A move command has these properties: code: a unique move command code (required). parameters: an array of values for this command. E.g. ROUTE_SWITCH_ON expects 1 parameter.	Use this one if you don't need to wait for the movement to finish before continuing. The last command in a move route should always have code ROUTE_END.
□	✓	Set Movement Route (Wait)	<code>// MZ const self = interpreter; const char = self.character(self._characterId = characterId); char.forceMoveRoute(moveRoute); self.setWaitMode("route");</code>	<code>list.push({ code: cOpA, parameters: [0] }); list.push({ code: Game_Character.ROUTE_END }); var route = { list: list, repeat: false, skippable: false }; char.forceMoveRoute(route); this.setWaitMode("route");</code>	Object moveRoute - An object with the following properties: list: an array of move command objects (required). repeat: if true, the route will repeat. skippable: if true, skip failed move commands. - A move command has these properties: code: a unique move command code (required). parameters: an array of values for this command. E.g. ROUTE_SWITCH_ON expects 1 parameter.	A character can only have one active move route at a time. Starting a new route will immediately end the current one. Events may have an auto move route to resume when their active route ends.
✓	□		<code>// MV var self = interpreter; var char = self._character = character; char.forceMoveRoute(moveRoute); self.setWaitMode("route");</code>			Move routes process up to one command per frame. A Script move command can be used to perform multiple actions in a single command.
✓	✓	Pathfinding (Move Route Script)	<code>this.moveStraight(this.findDirectionTo(X, Y));</code>	<code>// Move 1 tile towards (3, 5) using the built-in pathfinding this.moveStraight(this.findDirectionTo(3, 5));</code>	Number X, Y Target tile coordinates.	See the Move Commands section for details. Looks for an open path to the goal and takes one step along that path. The default pathfinding (12-tile A*) is only reliable over short distances.
✓	✓	Path to Tile (Move Route Script)	<code>var x = X, y = Y; var d = this.findDirectionTo(x, y); if (!d !this.canPass(this.x, this.y, d)) { this.forceMoveRoute({ list: [{ code: 0 }] }); this.setMovementSuccess(true); this._moveRouteIndex--; } else this.moveStraight(d);</code>	<code>// Path to (2, 6) using the built-in pathfinding var x = 2, y = 6; var d = this.findDirectionTo(x, y); if (!d !this.canPass(this.x, this.y, d)) { this.forceMoveRoute({ list: [{ code: 0 }] }); this.setMovementSuccess(true); this._moveRouteIndex--; } else this.moveStraight(d);</code>	Number X, Y Target tile coordinates.	The route should have Repeat enabled. It will stop when the target tile is reached or movement is blocked.
✓	✓	Is Character Moving?	<code>character.isMoving()</code>	<code>// Is the player moving at the moment? \$gamePlayer.isMoving()</code>	Game_Character character - The one to check. Returns Boolean - true iff the sprite is not at its destination.	When moving, the tile coordinates are immediately set the those of the next tile. The sprite coordinates then smoothly update based on the character's move speed.
✓	✓	Is Player Moving?	<code>var p = \$gamePlayer, d = p.getInputDirection(); p.isMoving() (d > 0 && p.canPass(p.x, p.y, d)) \$gameTemp.isDestinationValid()</code>	-	Returns Boolean true iff player is moving.	Useful for (event-based) stamina systems! This includes input checks to avoid giving the player 1 "free" standing frame per tile.
✓	✓	Player Move Input	<code>\$gamePlayer.getInputDirection()</code>	-	Returns Number 0 (none), 2 (down), 4 (left), 6 (right), or 8 (up).	Get player move button input for this frame.
✓	✓	Is Move Destination Active? (Did player move via mouse?)	<code>\$gameTemp.isDestinationValid()</code>	-	Returns Boolean true iff the touch-move destination is active.	-
✓	✓	Is Character Dashing?	<code>character.isDashing()</code>	<code>// Is the player dashing right now? \$gamePlayer.isDashing()</code>	Game_Character character - The one to check. Returns Boolean - true while moving to another tile at dashing speed.	Only the player can dash (+1 move speed).
✓	✓	Is Character Jumping?	<code>character.isJumping()</code>	<code>// Is map event 1 mid-jump right now? \$gameMap.event(1).isJumping()</code>	Game_Character character - The one to check. Returns Boolean - true iff the character is mid-jump.	-
✓	✓	Passability	<code>character.canPass(x, y, direction)</code>	<code>// Can the player move 1 tile left from where they are? \$gamePlayer.canPass(\$gamePlayer.x, \$gamePlayer.y, 4)</code>	Game_Character character The one to check.	Check all passability factors, including tile passability and character collisions.
✓	✓	Diagonal Passability	<code>character.canPassDiagonally(x, y, horz, vert)</code>	<code>// Can the player move 1 tile down + right from where they are? \$gamePlayer.canPassDiagonally(\$gamePlayer.x, \$gamePlayer.y, 6, 2);</code>	Number x, y - Starting tile coordinates. Number direction, horz, vert - Move direction: 2 (down), 4 (left), 6 (right), 8 (up). horz is horizontal (4, 6); vert is vertical (2, 8). Returns Boolean - true iff the character can move there unobstructed.	
✓	✓	Tile Passability	<code>character.isMapPassable(x, y, direction)</code>	<code>// Does the map allow event 5 to move 1 tile up from where it is? \$gameMap.event(5).isMapPassable(\$gameMap.event(5).x, \$gameMap.event(5).y, 2);</code>	Returns Boolean true iff the character can move there unobstructed.	Check tile passability (and map bounds).
✓	✓	Character Collisions	<code>character.isCollidedWithCharacters(x, y)</code>	<code>// Is there a character collision if the player moves to (3, 4)? \$gamePlayer.isCollidedWithCharacters(3, 4)</code>	Game_Character character - The one to check. Number x, y - Tile coordinates to check. Returns Boolean - true iff they would collide with someone there.	Check for character collisions only.
✓	✓	Map Bounds	<code>\$gameMap.isValid(x, y)</code>	<code>// Is tile (15, 18) within map bounds? \$gameMap.isValid(15, 18)</code>	Number x, y - Tile coordinates to check. Returns Boolean - true iff the tile is within map bounds.	Check map bounds only.
✓	✓	Event Move Type	<code>event._moveType</code>	<code>// Get the current auto-move type for map event 3 \$gameMap.event(3)._moveType</code>	Game_Event event - The event to check. Returns Number 0 (Fixed), 1 (Random), 2 (Approach), or 3 (Custom).	Resets when the event changes page.
Character Effects			Map character animations and balloons			▲ Return to top ▲

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
☐	☒	Show Animation (Character) (No Wait)	<code>\$gameTemp.requestAnimation([character], animationId, mirror);</code>	<code>// Show animation 15 on map event 20, no waiting \$gameTemp.requestAnimation([\$gameMap.event(20)], 15);</code>	<u>Game_Interpreter</u> interpreter • Interpreter to pause (this in a Script command)	MZ's <code>\$gameTemp.requestAnimation</code> can accept multiple target characters. This is typically for use with animations set to <i>Display Type: Center of all targets</i> .
☒	☐		<code>character.requestAnimation(animationId);</code>	<code>// Show animation 15 on map event 20, no waiting \$gameMap.event(20).requestAnimation(15);</code>	<u>Game_Character</u> character • Animation target. <u>Number</u> characterId • -1 (player), 0 (This Event), or 1+ (map event ID). <u>Number</u> animationId • ID of the animation to show. <u>Boolean</u> mirror • If true, mirror the animation.	
☐	☒	Show Animation (Character) (Wait)	<code>const self = interpreter; const char = self.character(self_characterId = characterId); \$gameTemp.requestAnimation([char], animationId, mirror); self.setWaitMode('animation');</code>	<code>// In a Script command: show animation 5 on the player and wait const char = this.character(this_characterId = -1); \$gameTemp.requestAnimation([char], 5); this.setWaitMode('animation');</code>		
☒	☐		<code>var self = interpreter; self._character = character; self._character.requestAnimation(animationId); self.setWaitMode('animation');</code>	<code>// In a Script command: show animation 5 on the player and wait this._character = \$gamePlayer; this._character.requestAnimation(5); this.setWaitMode('animation');</code>		
☐	☒	Is Any Animation Playing?	<code>SceneManager._scene._spriteset?.isAnimationPlaying?()</code>	-	-	
☒	☒	Is an Animation Playing on Character?	<code>character.isAnimationPlaying()</code>	<code>// Is an animation playing on the player? \$gamePlayer.isAnimationPlaying()</code>	<u>Game_Character</u> character • The one to check. Returns Boolean • true iff an animation is playing on that character.	-
☐	☒	Show Balloon (No Wait)	<code>\$gameTemp.requestBalloon(character, balloonId);</code>	<code>// Show Music Note balloon on the player, no waiting \$gameTemp.requestBalloon(\$gamePlayer, 3);</code>	<u>Game_Interpreter</u> interpreter • Interpreter to pause (this in a Script command)	Balloon IDs: • 1 = Exclamation • 2 = Question • 3 = Music Note • 4 = Heart • 5 = Anger • 6 = Sweat • 7 = Frustration • 8 = Silence • 9 = Light Bulb • 10 = Zzz • 11~15 = User Defined
☒	☐		<code>character.requestBalloon(balloonId);</code>	<code>// Show Music Note balloon on the player, no waiting \$gamePlayer.requestBalloon(3);</code>		
☐	☒	Show Balloon (Wait)	<code>const self = interpreter; const char = self.character(self_characterId = characterId); \$gameTemp.requestBalloon(char, balloonId); self.setWaitMode('balloon');</code>	<code>// In a Script command: show Zzz balloon on map event 1 and wait const char = this.character(this_characterId = 1); \$gameTemp.requestBalloon(char, 10); this.setWaitMode('balloon');</code>	<u>Game_Character</u> character • Balloon target. <u>Number</u> characterId • -1 (player), 0 (This Event), or 1+ (map event ID) <u>Number</u> balloonId • ID of balloon to show, see notes ->	
☒	☐		<code>var self = interpreter; self._character = character; self._character.requestBalloon(balloonId); self.setWaitMode('balloon');</code>	<code>// In a Script command: show Zzz balloon on map event 1 and wait this._character = \$gameMap.event(1); this._character.requestBalloon(10); this.setWaitMode('balloon');</code>		
☐	☒	Is Any Balloon Playing?	<code>SceneManager._scene._spriteset?._balloonSprites?.length > 0</code>	-	Returns Boolean • true iff at least one balloon sprite is playing.	-
☒	☒	Is a Balloon Playing on Character?	<code>character.isBalloonPlaying()</code>	<code>// Is a balloon playing on the player? \$gamePlayer.isBalloonPlaying()</code>	<u>Game_Character</u> character • The one to check. Returns Boolean • true iff a balloon is playing on that character.	-
☐	☒	Clear All Animations	<code>SceneManager._scene._spriteset?.removeAllAnimations();</code>	-	-	Immediately clears all animations on screen.
☐	☒	Clear Character Animation	<code>const spriteset = SceneManager._scene._spriteset; if (spriteset) { spriteset._animationSprites.filter(s => s.targetObjects.includes(character) }).forEach(s => spriteset.removeAnimation(s)); };</code>	<code>// MZ, stop all ongoing animations on the player const spriteset = SceneManager._scene._spriteset; if (spriteset) { spriteset._animationSprites.filter(s => s.targetObjects.includes(\$gamePlayer) }).forEach(s => spriteset.removeAnimation(s)); };</code>	<u>Game_Character</u> character • The one to affect.	Clears all ongoing animations on that character. (<i>Game_Character</i> has a method called <i>endAnimation</i> , but that only ends the wait time and has no effect on the animation itself.)
☒	☐		<code>var spriteset = SceneManager._scene._spriteset; if (spriteset && spriteset._characterSprites) { var sprite = null; spriteset._characterSprites.some(function(s) { if (s._character === character) sprite = s; return !!sprite; }); if (sprite) { sprite._animationSprites.forEach(function(s) { s.remove(); }); } };</code>	<code>// MV, stop all ongoing animations on the player var spriteset = SceneManager._scene._spriteset; if (spriteset && spriteset._characterSprites) { var sprite = null; spriteset._characterSprites.some(function(s) { if (s._character === \$gamePlayer) sprite = s; return !!sprite; }); if (sprite) { sprite._animationSprites.forEach(function(s) { s.remove(); }); } };</code>		
☐	☒	Clear All Balloons	<code>SceneManager._scene._spriteset?.removeAllBalloons?();</code>	-	-	Immediately clears all balloons on screen.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Clear Character Balloon	const spriteset = SceneManager._scene._spriteset; if (spriteset) { spriteset._balloonSprites.filter(s => s.targetObject === character).forEach(s => spriteset.removeBalloon(s)); }	// MZ, stop all ongoing balloons on the player const spriteset = SceneManager._scene._spriteset; if (spriteset) { spriteset._balloonSprites.filter(s => s.targetObject === \$gamePlayer).forEach(s => spriteset.removeBalloon(s)); }	Game_Character character • The one to affect.	Clear currently-playing balloon(s) for that character. (Game_Character has a method called <i>endBalloon</i> , but that only ends the wait time and has no effect on the balloon itself.)
			var spriteset = SceneManager._scene._spriteset; if (spriteset && spriteset._characterSprites) { var sprite = null; spriteset._characterSprites.some(function(s) { if (s._character === character) sprite = s; return !!sprite; }); if (sprite) sprite.endBalloon(); }	// MV, stop current balloon on the player var spriteset = SceneManager._scene._spriteset; if (spriteset && spriteset._characterSprites) { var sprite = null; spriteset._characterSprites.some(function(s) { if (s._character === \$gamePlayer) sprite = s; return !!sprite; }); if (sprite) sprite.endBalloon(); }		
Map Properties			Get/set map information			▲ Return to top ▲
		Map ID	\$gameMap.mapId()	-	Returns Number • ID of the current map.	-
		Map X Distance	\$gameMap.deltaX(x1 , x2)	// Get the horizontal separation from the player to map event 1 \$gameMap.deltaX(\$gamePlayer.x, \$gameMap.event(1).x)	Number x1, x2, y1, y2 • Tile coordinates to compare.	Accounts for looped maps. Separation will be negative if: • x2 is left of x1 , or
		Map Y Distance	\$gameMap.deltaY(y1 , y2)	// Get the vertical separation from the player to Y = 10 \$gameMap.deltaY(\$gamePlayer.y, 10);	Returns Number • Separation, in tiles.	• y2 is above y1 .
		Map Distance (Rectilinear)	\$gameMap.distance(x1 , y1 , x2 , y2)	// Get the rectilinear distance between the player and map event 1 \$gameMap.distance(\$gamePlayer.x, \$gamePlayer.y \$gameMap.event(1).x, \$gameMap.event(1).y)	Number x1, y1, x2, y2 • X and Y coordinates of each tile. Returns Number • X distance + Y distance.	Accounts for looped maps. Distance cannot be less than 0.
		Map Distance (Cartesian)	Math.hypot(\$gameMap.deltaX(x1 , x2), \$gameMap.deltaY(y1 , y2))	// Get straight-line distance between player and map event 1 Math.hypot(\$gameMap.deltaX(\$gamePlayer.x, \$gameMap.event(1).x), \$gameMap.deltaY(\$gamePlayer.y, \$gameMap.event(1).y))	Number x1, y1, x2, y2 • X and Y coordinates of each tile. Returns Number • Length (tiles) of straight line from tile 1 to tile 2.	Uses Math.hypot (Pythagoras). The result will generally not be an integer. Accounts for looped maps. Distance cannot be less than 0.
		Change Tileset	\$gameMap.changeTileset(tilesetId);	// Change to tileset ID 2 \$gameMap.changeTileset(2);	Number tilesetId • ID of the new tileset.	This will not wait for the new tileset images to load before continuing.
		Change Battle Background	\$gameMap.changeBattleback(back1 , back2);	// Change this map's battle BG to <i>Wood2.png</i> and <i>Bridge.png</i> \$gameMap.changeBattleback('Wood2', 'Bridge'); // Remove this map's specific battle background \$gameMap.changeBackground(null, null);	String back1, back2 • Battleback image filenames, without extension.	This change will be forgotten when the player transfers to a different map.
		Change Parallax	\$gameMap.changeParallax(name , loopX , loopY , speedX , speedY);	// Set the parallax to <i>Stars.png</i> , X & Y loop, auto-scroll X speed 1 \$gameMap.changeParallax('Stars', true, true, 1, 0);	String name • Parallax image filename, without extension. Boolean loopX, loopY • If true, the parallax will loop in that direction. Number speedX, speedY • Auto-scroll speeds.	This change will be forgotten when the player transfers to a different map. At 48 px tile size, auto-scroll speed 1 takes 96 frames to move 1 tile.
		Get Location Info (Terrain Tag)	\$gameMap.terrainTag(x, y)	// Store the terrain tag for tile (2, 3) in variable ID 1 \$gameVariables.setValue(1, \$gameMap.terrainTag(2, 3));	Number x, y • Tile coordinates to check. Number z • Tile layer to check, 0–3.	There are shortcuts for getting the region ID or terrain tag at a character's current position: • character.regionId() • character.terrainTag()
		Get Location Info (Event ID)	\$gameMap.eventIdXy(x, y)	// Get the lowest event ID of events at x = variable 3, y = variable 4 \$gameMap.eventIdXy(\$gameVariables.value(3), \$gameVariables.value(4))		
		Get Location Info (Tile ID)	\$gameMap.tileId(x, y, z)	// Get the ID of the layer #1 (z = 0) tile at the player's coordinates \$gameMap.tileId(\$gamePlayer.x, \$gamePlayer.y, 0)		
		Get Location Info (Region ID)	\$gameMap.regionId(x, y)	// Store the region ID of the tile in front of the player in variable 5 var d = \$gamePlayer.direction(); var x = \$gameMap.roundXWithDirection(\$gamePlayer.x, d); var y = \$gameMap.roundYWithDirection(\$gamePlayer.y, d); \$gameVariables.setValue(5, \$gameMap.regionId(x, y));		
		Map Camera Coordinates	\$gameMap.displayX() \$gameMap.displayY()	-	Returns Number • Map tile coordinate for the top-left of the screen.	-

Script Calls			Last updated 2023-12-14 (01:50 UTC)			
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Relocate Map Camera	\$gameMap.setDisplayPos(x, y);	// Move the camera so tile (10, 5) is in the top-left of the screen \$gameMap.setDisplayPos(10, 5);	Number x, y - Target tile coordinates. - Can be non-integer, e.g. 8.5.	The camera will adjust as needed to stay within the map bounds. Like with the Scroll Map effect, this is not a permanent offset.
✓	✓		\$gamePlayer.center(x, y);	// Center the game screen on tile (10, 5) \$gamePlayer.center(10, 5);		
✓	✓	Is Map Scrolling?	\$gameMap.isScrolling()	-	Returns Boolean true iff the map is currently scrolling.	-
✓	✓	Scroll Map	\$gameMap.startScroll(direction, distance, speed);	// Scroll down by 3 tiles at speed 4 \$gameMap.startScroll(2, 3, 4);	Number direction 2 (down), 4 (left), 6 (right), or 8 (up). Number distance - Distance to scroll, in tiles. Number speed - A character move speed.	The default player walking speed is 4.
✓	✓	Stop Scroll Map	\$gameMap.setupScroll();	Example Use: ◆Script: \$gameMap.startScroll(2, 125, 4); ◆Text: None, None, Window, Bottom : :HI PLAYER. ◆Script: \$gameMap.setupScroll(); ◆Text: None, None, Window, Bottom : :owo scrolling stop.	-	Stops Scroll Map before it's finished. Useful if you are playing a cutscene that has a long backstory and if players want to speedrun, the scroll map won't stop them. (RUBY: \$game_map.setup_scroll)
Screen Effects & Pictures			Fade, tint, flash, shake, weather, zoom, pictures			▲ Return to top ▲
✓	✓	Fade Out	\$gameScreen.startFadeOut(frames);	// In a Script command: fade out over 60 frames \$gameScreen.startFadeOut(60);	Number frames How long it takes to fade in/out, in frames.	Default fade time is 24 frames.
✓	✓	Fade In	\$gameScreen.startFadeIn(frames);	// Make the screen fade in over 60 frames \$gameScreen.startFadeIn(60);		
✓	✓	Screen Fade	\$gameScreen.brightness()	-	Returns Number 0 (fully obscured) to 255 (fully visible).	This relates to the Fade Out and Fade In effects.
✓	✓	Tint Screen	\$gameScreen.startTint([red, green, blue, grey], frames);	// Tint the screen magenta (red + blue) over 200 frames \$gameScreen.startTint([90, 0, 90, 0], 180);	Number red, green, blue - Values from -255 to 255. Number grey 0 (no change) to 255 (full greyscale). Number frames - Tint will gradually be applied over this duration.	-
✓	✓	Screen Tint Colour	\$gameScreen.tone()	// Get the red component of the current screen tint \$gameScreen.tone()[0]	Returns Number[] An array: [red, green, blue, grey].	-
✓	✓	Flash Screen	\$gameScreen.startFlash([red, green, blue, intensity], frames);	// Apply a 120-frame black screen flash at full intensity \$gameScreen.startFlash([0, 0, 0, 255], 120);	Number red, green, blue - Values from 0 to 255. Number intensity - Starting flash opacity: 0 (invisible) to 255 (opaque). Number frames - Flash duration, in frames.	-
✓	✓	Screen Flash Colour	\$gameScreen.flashColor()	// Get the current screen flash intensity \$gameScreen.flashColor()[3]	Returns Number[] An array: [red, green, blue, intensity].	-
✓	✓	Shake Screen	\$gameScreen.startShake(power, speed, frames);	// Apply a 60-frame shake at speed 3, power 5 \$gameScreen.startShake(5, 3, 60);	Number power - Shake power, typically 1~9. Number speed - Shake speed, typically 1~9. Number frames - Shake duration, in frames.	-
✓	✓	Screen Shake Offset	\$gameScreen.shake()	-	Returns Number Current screen shake offset.	Used as a horizontal display offset for map and battle sprites.
✓	✓	Change Weather	\$gameScreen.changeWeather(type, power, frames);	// Change the weather to rain, power 4, over 300 frames \$gameScreen.changeWeather('rain', 4, 300); // Instantly disable weather effects \$gameScreen.changeWeather('none', 0, 1);	String type "none", "rain", "storm", or "snow". Number power - Weather intensity, typically 1~9. Number frames - Weather will gradually change over this duration.	-
✓	✓	Weather Type	\$gameScreen.weatherType()	// Is the current game weather "rain"? \$gameScreen.weatherType() === 'rain'	Returns String "none", "rain", "storm", or "snow".	-
✓	✓	Weather Power	\$gameScreen.weatherPower()	// Is the current weather power more than 5? \$gameScreen.weatherPower() > 5	Returns Number Current weather power.	
✓	✓	Weather Transition Time	\$gameScreen._weatherDuration	// Is the game weather changing? \$gameScreen._weatherDuration > 0	Returns Number Remaining frames for transition to new weather.	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
☒	☒	Screen Zoom (Zoom In, Magnify)	\$gameScreen.startZoom(x, y, scale, duration);	// Zoom in on the centre of the screen by 200%, over 5 seconds var x = Graphics.width / 2; var y = Graphics.height / 2; var scale = 2; var duration = 5 * 60; \$gameScreen.startZoom(x, y, scale, duration);	<u>Number</u> x, y • Screen coordinates (px) for zoom focus. <u>Number</u> scale • Target scale, e.g. 2 = 200%. <u>Number</u> duration • Smoothly change scale over this many frames.	The map will scroll as usual when zoomed. This can result in the player moving off-screen. For scrollable maps, consider adding borders and applying the zoom to the screen center.
☒	☒		\$gameScreen.setZoom(x, y, scale);	// Zoom in on the player by 150%, instantly var x = \$gamePlayer.screenX(); var y = \$gamePlayer.screenY(); var scale = 1.5; \$gameScreen.setZoom(x, y, scale);		To get the screen dimensions, you can use: • Graphics.width • Graphics.height
☒	☒		\$gameScreen.zoomX()	-	<u>Returns</u> <u>Number</u> • X/Y screen coordinate (px) of the zoom focus.	Scale values < 1 can be used (zoom out), but may expose unrendered map areas.
☒	☒		\$gameScreen.zoomY()	-		
☒	☒	Zoom X	\$gameScreen.zoomX()	-	<u>Returns</u> <u>Number</u> • Current zoom scale/magnification factor.	-
☒	☒	Zoom Y	\$gameScreen.zoomY()	-		-
☒	☒	Zoom Scale	\$gameScreen.zoomScale()	-		-
		Pictures	Picture-related scripting			
☒	☒	Picture	\$gameScreen.picture(pictureId)	// Get picture ID 1 \$gameScreen.picture(1)	<u>Number</u> pictureId • ID of the picture: 1~100. <u>Returns</u> <u>Game_Picture</u> • The corresponding picture object.	
☒	☒	Show Picture	\$gameScreen.showPicture(pictureId, name, origin, x, y, scaleX, scaleY, opacity, blendMode);	// Show picture 1, pic.png, at 100% scale centred on (50, 50) \$gameScreen.showPicture(1, 'pic', 1, 50, 50, 100, 100, 255, 0);	<u>Number</u> pictureId • ID of the picture 1~100. <u>String</u> name • Filename, without extension. <u>Number</u> origin • Anchor type: 0 (Upper Left) or 1 (Center). <u>Number</u> x, y • Anchor screen coordinates (px). <u>Number</u> scaleX, scaleY • Scale multipliers: use 100 for 100% (original size). <u>Number</u> opacity • 0 to 255, where 0 is 100% transparent. <u>Number</u> blendMode • 0 (Normal), 1 (Additive), 2 (Multiply), or 3 (Screen) <u>Game_Picture</u> picture • Picture object to change	Map and battle pictures are stored separately: picture ID 1 is one picture on the map, and a different picture in battle.
☒	☒		picture.show(name, origin, x, y, scaleX, scaleY, opacity, blendMode);	// Show picture 2, dog.png, 50% scale in the top-left of the screen \$gameScreen.picture(2).show('dog', 0, 0, 50, 50, 255, 0);		All battle pictures are erased at the start of each battle.
☒	☒	Move Picture	\$gameScreen.movePicture(pictureId, origin, x, y, scaleX, scaleY, opacity, blendMode, frames, easing);	// Move pic 1 to center on (200, 150) at 0 opacity, over 60 frames \$gameScreen.movePicture(1, 1, 200, 150, 100, 100, 0, 0, 60);	<u>Number</u> pictureId • ID of the picture 1~100. <u>Number</u> origin • New anchor type: 0 (Upper Left) or 1 (Center). <u>Number</u> x, y • New anchor screen coordinates (px). <u>Number</u> scaleX, scaleY • New scale multipliers: use 100 for 100%. <u>Number</u> opacity • 0 to 255, where 0 is 100% transparent. <u>Number</u> blendMode • 0 (Normal), 1 (Additive), 2 (Multiply), or 3 (Screen). <u>Number</u> frames • Movement duration, in frames. <u>Number</u> easing (MZ only) (optional) • 1 (Slow start), 2 (Slow end), 3 (Slow start & end).	origin, blendMode, and easing will apply as soon as the movement starts. Other changes will be applied smoothly over the duration.
☒	☒		picture.move(origin, x, y, scaleX, scaleY, opacity, blendMode, frames, easing);	// Move pic 2 to top-left of screen at 100% scale, instantly \$gameScreen.picture(2).move(0, 0, 0, 100, 100, 255, 0, 1);	<u>Game_Picture</u> picture • Picture object to move.	Cross-fade can be achieved with multiple pictures on top of one another.
☒	☒		var pic = picture; pic._targetX = x; pic._targetY = y; pic._targetScaleX = scaleX; pic._targetScaleY = scaleY; pic._targetOpacity = opacity;	// [MZ] Scale picture 5 vertically to 200% over 60 frames var pic = \$gameScreen.picture(5); pic._targetScaleY = 200; pic._duration = pic._wholeDuration = 60;		Can be used instead of a standard Move Picture effect to avoid setting certain properties, e.g. to only update the picture's X scale. Omit any lines that you want to remain unchanged.
☐	☒		picture._duration = picture._wholeDuration = frames;	// [MV] Scale picture 1 horizontally to 50% and +6 frames duration var pic = \$gameScreen.picture(1); pic._targetScaleX = 50; pic._duration += 6;		Movement can only occur if _duration > 0.
☒	☐		picture._duration = frames;			Note that MZ requires setting the _wholeDuration value as well, due to how its easing is calculated.
☒	☒	Set Picture Name	picture._name = name;	// Change picture 5 to use Cloud.png \$gameScreen.picture(5)._name = 'Cloud';		These properties will apply immediately.
☒	☒	Set Picture Origin	picture._origin = origin;			
☒	☒	Set Picture Blend Mode	picture._blendMode = blendMode;	// Change picture 1's origin to Center, effective immediately \$gameScreen.picture(1)._origin = 1;		A name change may take a few frames to load the new image.
☐	☒	Set Picture Easing Type	picture._easingType = easing;			
☒	☒	Picture Name	picture.name()	// Is picture ID 1 showing cat.png right now? \$gameScreen.picture(1).name() === 'cat'	<u>Game_Picture</u> picture • The one to check. <u>Returns</u> <u>String</u> • Name of this picture's assigned file (no extension).	
☒	☒	Picture Origin	picture.origin()			
☒	☒	Picture X	picture.x()	// Get the current x coordinate of picture 1's origin \$gameScreen.picture(1).x()		
☒	☒	Picture Y	picture.y()			
☒	☒	Picture Scale X	picture.scaleX()	// Get the X-scale of picture 1	<u>Game_Picture</u> picture • The one to check.	These properties are set by the Show Picture and/or Move Picture effects.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Picture Scale Y	<code>picture.scaleY()</code>	<code>\$gameScreen.picture(1).scaleX()</code>	Returns <u>Number</u> • Value of that property for this picture.	
✓	✓	Picture Opacity	<code>picture.opacity()</code>	// Get the opacity of picture 1 <code>\$gameScreen.picture(1).opacity()</code>		
✓	✓	Picture Blend Mode	<code>picture.blendMode()</code>			
✓	✓	Picture Move Duration	<code>picture._duration</code>	// Get the remaining frames for picture 1's current movement <code>\$gameScreen.picture(1)._duration</code>		
❑	✓	Picture Easing Type	<code>picture._easingType</code>			
✓	✓	Erase Picture	<code>\$gameScreen.erasePicture(pictureId);</code>	// Erase picture 1 <code>\$gameScreen.erasePicture(1);</code>	<u>Number pictureId</u> • ID of the picture 1~100.	-
✓	✓	Rotate Picture	<code>\$gameScreen.rotatePicture(pictureId, speed);</code>	// Make picture 1 start rotating at speed 3 <code>\$gameScreen.rotatePicture(1, 3);</code>	<u>Number pictureId</u> • ID of the picture 1~100. <u>Number speed</u> • Rotation speed 4 is +2° clockwise per frame.	Rotation occurs around the picture's origin/anchor.
✓	✓		<code>picture.rotate(speed);</code>	// Make picture 2 start rotating at speed -4.5 <code>\$gameScreen.picture(2).rotate(-4.5);</code>	<u>Game_Picture picture</u> • Picture object to rotate.	
✓	✓	Picture Rotation Speed	<code>picture._rotationSpeed</code>	// Get the rotation speed of picture 1 <code>\$gameScreen.picture(1)._rotationSpeed</code>	<u>Game_Picture picture</u> • The one to check.	Set by the Rotate Picture effect.
✓	✓	Picture Angle	<code>picture.angle()</code>	// Get the current angle of picture 1 <code>\$gameScreen.picture(1).angle()</code>	Returns <u>Number</u> • The rotation speed or current angle of that picture.	Updates each frame based on the picture's rotation speed.
✓	✓	Set Picture Angle	<code>picture._angle = angle;</code>	// Turn picture 1 to 45° clockwise from its original state <code>\$gameScreen.picture(1)._angle = 45;</code>	<u>Game_Picture picture</u> • The one to move. <u>Number angle</u> • Angle in degrees.	Immediately sets the picture to that angle.
✓	✓	Tint Picture	<code>\$gameScreen.tintPicture(pictureId, [red, green, blue, grey], frames);</code>	// Tint picture 1 fully grey over 300 frames <code>\$gameScreen.tintPicture(1, [0, 0, 0, 255], 300);</code>	<u>Number pictureId</u> • ID of the picture 1~100. <u>Number red, green, blue</u> • Values from -255 to 255. <u>Number grey</u> • 0 (no change) to 255 (greyscale).	-
✓	✓		<code>picture.tint([red, green, blue, grey], frames);</code>	// Tint picture 2 yellow (red + green) over 60 frames <code>\$gameScreen.picture(2).tint([80, 80, 0, 0], 60);</code>	<u>Number frames</u> • Tint will gradually be applied over this duration. <u>Game_Picture picture</u> • Picture object to tint.	
✓	✓	Picture Tint Colour	<code>picture.tone()</code>	// Get the blue component of picture 1's current tint <code>\$gameScreen.picture(1).tone()[2]</code>	<u>Game_Picture picture</u> • The one to check. Returns <u>Number[]</u> • An array: [red, green, blue, grey].	Set by the Tint Picture effect.
		Clearance	<i>Clearing the screen</i>			
✓	✓	Clear Screen	<code>\$gameScreen.clear();</code>	-	-	Clears/resets all screen effects (see below).
✓	✓	Clear Screen Fade	<code>\$gameScreen.clearFade();</code>			Clears all fade in/out effects.
✓	✓	Clear Screen Tint	<code>\$gameScreen.clearTone();</code>			Resets the screen tint to default.
✓	✓	Clear Screen Flash	<code>\$gameScreen.clearFlash();</code>			Ends any screen flash effects.
✓	✓	Clear Screen Shake	<code>\$gameScreen.clearShake();</code>	-	-	Ends any screen shake effects.
✓	✓	Clear Screen Zoom	<code>\$gameScreen.clearZoom();</code>			Resets zoom to default.
✓	✓	Clear Weather	<code>\$gameScreen.clearWeather();</code>			Ends any weather effects.
✓	✓	Clear Pictures	<code>\$gameScreen.clearPictures();</code>			Clears all map & battle pictures.
✓	✓	Erase All Battle Pictures	<code>\$gameScreen.eraseBattlePictures();</code>	-	-	Clears all battle pictures. This happens at the start of each battle.
		Scenes	<i>Change/check scenes</i>			▲ Return to top ▲
✓	✓	(battle note)	The core scripts do not support scene changes mid-battle.			
✓	✓	Start Battle (No Result Handlers)	<code>BattleManager.setup(troopId, canEscape, canLose);</code> <code>\$gamePlayer.makeEncounterCount();</code> <code>SceneManager.push(Scene_Battle);</code>	// Start battle with troop ID 20, can escape, lose = game over <code>BattleManager.setup(20, true, false);</code> <code>\$gamePlayer.makeEncounterCount();</code> <code>SceneManager.push(Scene_Battle);</code>	<u>Number troopId</u> • ID of the troop to fight. <u>Boolean canEscape</u> • If true, the party can escape. <u>Boolean canLose</u> • If true, the party can continue after losing.	If canLose is false, a defeat will result in a game over. A result of 1 is: • Escape if <code>BattleManager._escaped</code> is true; • Abort otherwise.
✓	✓	Start Battle (Result Handlers)	<code>BattleManager.setup(troopId, canEscape, canLose);</code> <code>BattleManager.setEventCallback(callback);</code> <code>\$gamePlayer.makeEncounterCount();</code> <code>SceneManager.push(Scene_Battle);</code>	// Start battle with troop 3, no escape; on lose, turn on switch 5 <code>BattleManager.setup(3, false, true);</code> <code>BattleManager.setEventCallback(function(result) {</code> if (result === 2) <code>\$gameSwitches.setValue(5, true);</code> }); <code>\$gamePlayer.makeEncounterCount();</code> <code>SceneManager.push(Scene_Battle);</code>	<u>Function callback</u> • Runs when the battle ends. 1 argument: result . <u>Number result</u> • 0 (victory), 1 (escape/abort), or 2 (defeat).	

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
☒	☒	Open Shop Screen	SceneManager.push(Scene_Shop); SceneManager.prepareNextScene(goods , purchaseOnly);	// Open a shop selling armors 1, 2, 3 at standard price (buy only) SceneManager.push(Scene_Shop); SceneManager.prepareNextScene([2, 1, 0], [2, 2, 0], [2, 3, 0]), true);	Number[] goods • An array of [itemType, itemId, priceType, price]: ◦ itemType : 0 (Item), 1 (Weapon), or 2 (Armor). ◦ itemId : ID of the item. ◦ priceType : 0 (Standard) or 1 (Specific). ◦ price : item price; ignored if priceType = 0. Boolean purchaseOnly • If true, items cannot be sold to this shop.	-
☒	☒	Open Name Input Screen	SceneManager.push(Scene_Name); SceneManager.prepareNextScene(actorId , maxLength);	// Get the player to rename the party leader, up to 12 characters SceneManager.push(Scene_Name); SceneManager.prepareNextScene(\$gameParty.members()[0].actorId(), 12);	Number actorId • ID of the actor to rename. Number maxLength • Maximum character count for the new name.	-
☒	☒	Open Pause Menu	SceneManager.push(Scene_Menu); Window_MenuCommand.initCommandPosition();	-	-	Open the pause menu.
☒	☒	Open Item Screen	SceneManager.push(Scene_Item);	-	-	Open the party inventory.
☒	☒	Open Equip Screen	\$gameParty.setMenuActor(actor); SceneManager.push(Scene_Equip);	// Open the equip screen for party member #2 (index 1) \$gameParty.setMenuActor(\$gameParty.members()[1]); SceneManager.push(Scene_Equip);	Game Actor actor • The one to inspect.	Open the equip screen for the specified actor.
☒	☒	Open Skill Screen	\$gameParty.setMenuActor(actor); SceneManager.push(Scene_Skill);	// Open the skill screen for the party leader \$gameParty.setMenuActor(\$gameParty.members()[0]); SceneManager.push(Scene_Skill);		Open the skill screen for the specified actor.
☒	☒	Open Status Screen	\$gameParty.setMenuActor(actor); SceneManager.push(Scene_Status);	// Open the status screen for the party leader (index 0) \$gameParty.setMenuActor(\$gameParty.members()[0]); SceneManager.push(Scene_Status);		Open the status screen for the specified actor.
☒	☒	Open Save Screen	SceneManager.push(Scene_Save);	-	-	Open the save screen.
☒	☒	Open Continue Screen	SceneManager.push(Scene_Load);	-	-	Open the continue/load screen.
☒	☒	Open Options Screen	SceneManager.push(Scene_Options);	-	-	Open the options menu.
☒	☒	Open "Return to Title?"	SceneManager.push(Scene_GameEnd);	-	-	Same as choosing "Game End" from the pause menu.
☒	☒	Return to Title Screen	SceneManager.goto(Scene_Title);	-	-	Return to the title screen.
☒	☒	Game Over	SceneManager.goto(Scene_Gameover);	-	-	Cause a game over.
☒	☒	Exit Game	SceneManager.exit();	-	-	Will not close the window in Web-deployed games.
☒	☒	Current Scene	SceneManager._scene	-	Returns Scene_Base • Current scene instance.	-
☒	☒	Compare Scene	SceneManager._scene instanceof scene	// Is the current scene an item/skill menu? SceneManager._scene instanceof Scene_ItemBase	Scene_Base scene • Scene to compare against.	This checks for inheritance, e.g. <i>instanceof Scene_ItemBase</i> will return true for <i>Scene_Item</i> and <i>Scene_Skill</i> . Details: instanceof .
☒	☒		SceneManager._scene.constructor === scene	// Is the current scene the pause menu? SceneManager._scene.constructor === Scene_Menu	Returns Boolean • true iff the current scene is of that type.	This does not check for inheritance.
☒	☒	Go to Scene	SceneManager.goto(scene);	(See above for examples.)	Scene_Base scene	Forget the current scene and go to the new scene.
☒	☒		SceneManager.push(scene);		• Next scene class.	Add the current scene to the stack, then go to the new scene.
☒	☒	End Current Scene	SceneManager.pop();	-	-	Forget the current scene and return to the scene on the top of the stack. If the stack is empty, exit the game.
☒	☒	Clear Scene Stack	SceneManager.clearStack();	-	-	Empty the stack. This happens automatically when the map and title scenes start.
		Save & Continue	Save file management		▲ Return to top ▲	
☒	☒	Does Any Save File Exist?	DataManager.isAnySavefileExists()	-	Returns Boolean • true iff there is at least one save file.	May be inaccurate if saves have been added manually.
☐	☒	Does Save File Exist?	DataManager.savefileExists(slotId)	// MZ: is save slot 1 occupied? DataManager.savefileExists(1)	Number slotId • ID of save slot.	-
☒	☐		StorageManager.exists(slotId)	// MV: is save slot 1 occupied? StorageManager.exists(1)	Returns Boolean • true iff that save slot is filled.	
☒	☐	Last Accessed Save	DataManager.lastAccessedSavefileId();	-	Returns Number • ID of save file last accessed this session.	This will not persist from one play session to the next. Default: 1.
☒	☒	Most Recent Save (Global)	DataManager.latestSavefileId();	-	Returns Number • ID of save file with most recent timestamp.	MZ: 0 if no saves are present. MV: 1 if no saves are present.
☐	☒	Most Recent Save (Playthrough)	\$gameSystem.savefileId();	-	Returns Number • ID of the latest save slot used this playthrough.	On new game this is set to the first empty slot, or the least recent save slot if all slots are filled.
☐	☒	Least Recent Save	DataManager.earliestSavefileId();	-	Returns Number • ID of save file with least recent timestamp.	0 if no saves are present.
☐	☒	Next Empty Save Slot	DataManager.emptySavefileId();	-	Returns Number • ID of first save slot with no save data.	-1 if all save slots are filled.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
☐	☒	Delete Save	StorageManager.remove(DataManager.makeSavename(slotid)); DataManager.removeInvalidGlobalInfo();	// MZ: delete save ID 1 StorageManager.remove(DataManager.makeSavename(1)); DataManager.removeInvalidGlobalInfo();	Number slotid - ID of save slot. - MZ's autosave is slot ID 0.	Deletes specified save file.
☒	☐		StorageManager.remove(slotid);	// MV: delete save ID 1 StorageManager.remove(1);		
☐	☒	Delete All Saves	for (let id = DataManager.maxSavefiles(); id >= 0; id--) { StorageManager.remove(DataManager.makeSavename(id)); } DataManager.removeInvalidGlobalInfo();	-	-	Deletes all save files.
☒	☐		for (var id = DataManager.maxSavefiles(); id > 0; id--) { StorageManager.remove(id); }			
☐	☒	Autosave	SceneManager._scene.requestAutosave();	-	-	Requests an autosave, obeying all normal rules.
☐	☒	Force Autosave	SceneManager._scene.executeAutosave();	-	-	Forces an autosave, ignoring save menu access etc.
☐	☒	Save	\$gameSystem.setSavefileId(slotid); \$gameSystem.onBeforeSave(); DataManager.saveGame(slotid) .then(() => { saveSuccess }) .catch(() => { saveFailure });	// Save to slot 4 \$gameSystem.setSavefileId(4); \$gameSystem.onBeforeSave(); DataManager.saveGame(4); // Save to slot 4; on success/failure, play suitable sounds \$gameSystem.setSavefileId(4); \$gameSystem.onBeforeSave(); DataManager.saveGame(4) .then(() => SoundManager.playSave()) .catch(() => SoundManager.playBuzzer());	Number slotid - ID of save slot. saveSuccess - Code to run when save is successful. saveFailure - Code to run when save fails.	setSavefileId updates the ID of the last-used save slot. You can omit this for a "silent" save. Save success/failure parts are optional, see examples. By default, save files do not include: \$gameTemp (e.g. common event queue) \$gameMessage (e.g. remaining text to display) \$gameTroop (e.g. enemy data from previous battle)
☒	☐		\$gameSystem.onBeforeSave(); if (DataManager.saveGame(slotid)) { StorageManager.cleanBackup(slotid); saveSuccess } else { saveFailure }	// Save to slot 4 \$gameSystem.onBeforeSave(); if (DataManager.saveGame(4)) StorageManager.cleanBackup(4); // Save to slot 4; on success/failure, play suitable sounds \$gameSystem.onBeforeSave(); if (DataManager.saveGame(4)) { StorageManager.cleanBackup(4); SoundManager.playSave(); } else { SoundManager.playBuzzer(); }		
☐	☒	Continue/Load	DataManager.loadGame(slotid) .then(() => { loadSuccess SceneManager._scene.fadeOutAll(); if (\$gameSystem.versionId() !== \$dataSystem.versionId()) { const p = \$gamePlayer; p.reserveTransfer(\$gameMap.mapId(), p.x, p.y, p.direction(), 0); p.requestMapReload(); } SceneManager.goto(Scene_Map); \$gameSystem.onAfterLoad(); }).catch(() => { loadFailure }).finally(() => { afterLoad });	// Continue from save 4; play success/fail SE; turn on switch 100 DataManager.loadGame(4) .then(() => { SoundManager.playLoad(); SceneManager._scene.fadeOutAll(); if (\$gameSystem.versionId() !== \$dataSystem.versionId()) { const p = \$gamePlayer; p.reserveTransfer(\$gameMap.mapId(), p.x, p.y, p.direction(), 0); p.requestMapReload(); } SceneManager.goto(Scene_Map); \$gameSystem.onAfterLoad(); }).catch(() => { SoundManager.playBuzzer(); }).finally(() => { \$gameSwitches.setValue(100, true); });	Number slotid - ID of save slot. loadSuccess - Code to run when load is successful. loadFailure - Code to run when load fails. afterLoad - Code to run after load has succeeded or failed.	The map will reload if the game version does not match that of the save file. loadSuccess , loadFailure , and afterLoad are all optional. For MZ: this is an async script call. The event will continue processing commands while loading from the save file. This is what the switch is for in the example: once the Load operation finishes (success or fail) the switch is turned on. The switch could be used to break a Loop, allowing a failed Load operation to continue the event. If using that approach, remember to turn the switch OFF before this script call.
☒	☐		if (DataManager.loadGame(slotid)) { loadSuccess SceneManager._scene.fadeOutAll(); if (\$gameSystem.versionId() !== \$dataSystem.versionId()) { var p = \$gamePlayer; p.reserveTransfer(\$gameMap.mapId(), p.x, p.y, p.direction(), 0); p.requestMapReload(); } SceneManager.goto(Scene_Map); \$gameSystem.onAfterLoad(); } else { loadFailure }	// Continue from save slot 4 if (DataManager.loadGame(4)) { SceneManager._scene.fadeOutAll(); if (\$gameSystem.versionId() !== \$dataSystem.versionId()) { var p = \$gamePlayer; p.reserveTransfer(\$gameMap.mapId(), p.x, p.y, p.direction(), 0); p.requestMapReload(); } SceneManager.goto(Scene_Map); \$gameSystem.onAfterLoad(); }		
Timer & System		Get/set system & timer properties				▲ Return to top ▲
☒	☒	Start Timer	\$gameTimer.start(frames);	// Start the timer with a duration of 10 seconds	Number frames	-
☒	☒	Stop Timer	\$gameTimer.stop();	\$gameTimer.start(10 * 60);	Starting timer duration, in frames.	-

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Timer: Remaining Time	\$gameTimer.seconds()		Returns Number • Seconds/frames remaining on the game timer.	Rounds down to an integer.
✓	✓		\$gameTimer._frames		-	-
✓	✓	Play Time	\$gameSystem.playtime()		Returns Number • Seconds/frames so far this playthrough.	Seconds are rounded down to an integer. Counts time spent in menus/paused.
✓	✓		Graphics.frameCount		-	-
✓	✓	Save Count	\$gameSystem.saveCount()		Returns Number • Relevant system stat for this playthrough.	-
✓	✓	Battle Count	\$gameSystem.battleCount()			-
✓	✓	Win Count	\$gameSystem.winCount()			-
✓	✓	Escape Count	\$gameSystem.escapeCount()			Does not count aborted battles.
✓	✓	Step Count	\$gameParty.steps()		Returns Number • Map step count for this playthrough.	Does not count forced movement, or movement while in vehicles.
✓	✓	Currency Unit	\$dataSystem.currencyUnit	-	Returns String • Currency unit as defined on System database tab.	-
✓	✓	Enable Map Name Display	\$gameMap.enableNameDisplay();	-	-	Changes whether the map's display name is shown when arriving on a new map.
✓	✓	Disable Map Name Display	\$gameMap.disableNameDisplay();			
✓	✓	Is Map Name Display Enabled?	\$gameMap.isNameDisplayEnabled()	-	Returns Boolean • true iff the map name is set to display on arrival.	-
✓	✓	Enable Save Access	\$gameSystem.enableSave();	-	-	When disabled, the Save option on the pause menu will be locked. It also disables MZ's autosave.
✓	✓	Disable Save Access	\$gameSystem.disableSave();			
✓	✓	Is Save Access Enabled?	\$gameSystem.isSaveEnabled()	-	Returns Boolean • true iff save access is currently enabled.	-
✓	✓	Enable Menu Access	\$gameSystem.enableMenu();	-	-	When disabled, pressing the menu button on the map will not open the pause menu.
✓	✓	Disable Menu Access	\$gameSystem.disableMenu();			
✓	✓	Is Menu Access Enabled?	\$gameSystem.isMenuEnabled()	-	Returns Boolean • true iff menu access is currently enabled.	-
✓	✓	Enable Formation Access	\$gameSystem.enableFormation();	-	-	When disabled, the Formation option on the pause menu will be locked.
✓	✓	Change Formation Access	\$gameSystem.disableFormation();			
✓	✓	Is Formation Access Enabled?	\$gameSystem.isFormationEnabled()	-	Returns Boolean • true iff formation access is currently enabled.	-
✓	✓	Enable Encounters	\$gameSystem.enableEncounter(); \$gamePlayer.makeEncounterCount();	-	-	When disabled, random encounters will not occur. <i>makeEncounterCount</i> resets the step counter for random encounters.
✓	✓	Disable Encounters	\$gameSystem.disableEncounter(); \$gamePlayer.makeEncounterCount();			
✓	✓	Are Random Encounters Enabled?	\$gameSystem.isEncounterEnabled()	-	Returns Boolean • true iff random encounters are enabled.	-
✓	✓	Can a Random Encounter Happen?	\$gamePlayer.canEncounter()	-	Returns Boolean • true iff all of the following are true: ◦ Random encounters are enabled; ◦ The party does not have an Encounter None trait; ◦ The player is not in the airship; ◦ The player is not undergoing a move route; ◦ "Debug through" (Ctrl/Cmd in playtest) is off.	This does not check for the existence of troops in the random encounter list.
✓	✓	Change Window Tone	\$gameSystem.setWindowTone([red, green, blue]);	// Set the game window tone to a green/blue colour \$gameSystem.setWindowTone([-50, 120, 120]);	Number red, green, blue • Values from -255 to 255.	This corresponds to the Window Color setting on the System tab of the database. It applies to all in-game windows.
✓	✓	Window Tone	\$gameSystem.windowTone()	// Get the red component of the current window colour \$gameSystem.windowTone()[0]	Returns Number[] • An array: [red, green, blue].	
Audio & Video			Get/set audio & video			▲ Return to top ▲
✓	✓	(Audio Object)	{ name: filename , volume: volume , pitch: pitch , pan: pan }	(see below)	String filename • Audio filename, without extension. Number volume • Volume multiplier, use 100 for 100%. Number pitch • Pitch multiplier, use 100 for 100%. Number pan (optional) • Left/right panning, from -100 to 100.	pan is optional; if omitted, it will default to 0.
✓	✓	Play BGM	AudioManager.playBgm(bgm , pos);	// Play BGM <i>bgm1.ogg</i> with 20% volume, 100% pitch, 0 pan AudioManager.playBgm({ name: 'bgm1', volume: 20, pitch: 100 });	(Audio Object) bgm • Background music object. Number pos (optional) • Start time, in seconds from the start of the track.	pos is optional; if omitted, it will default to 0.
✓	✓	Fade Out BGM	AudioManager.fadeOutBgm(seconds);	// Fade out the current BGM over 5 seconds AudioManager.fadeOutBgm(5);	Number seconds • Fadeout duration, in seconds.	-

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Is this the Current BGM?	AudioManager.isCurrentBgm(bgm);	// Is the current BGM named bgm1? AudioManager.isCurrentBgm({ name: 'bgm1' });	(Audio Object) bgm • Background music object. Returns Boolean • true iff the current BGM matches the provided one.	By default the bgm object here only needs one property: name.
✓	✓	Current BGM object	AudioManager._currentBgm	-	Returns (Audio Object) • The currently-playing BGM.	-
✓	✓	Save BGM	\$gameSystem.saveBgm();	-	-	Saves the current BGM. Pairs with Replay BGM .
✓	✓	Replay BGM	\$gameSystem.replayBgm();	-	-	Replays the saved BGM. Pairs with Save BGM .
✓	✓	Change BGM Volume	var c = AudioManager._currentBgm; c.volume = newVolume ; AudioManager.updateBgmParameters(c);	// Change the current BGM's volume to 20%. var c = AudioManager._currentBgm; c.volume = 20; AudioManager.updateBgmParameters(c);	Number newVolume • New BGM volume, see Audio Object for details.	These change the BGM volume/pan without restarting the track.
✓	✓	Change BGM Pan	var c = AudioManager._currentBgm; c.pan = newPan ; AudioManager.updateBgmParameters(c);	// Change the current BGM's panning to 50% left. var c = AudioManager._currentBgm; c.pan = -50; AudioManager.updateBgmParameters(c);	Number newPan • New BGM panning, see Audio Object for details.	
☐	✓	Change BGM Pitch	if (AudioManager._currentBgm) { var b = AudioManager._bgmBuffer, p = newPitch ; // Set new pitch directly to avoid restarting from 0 b._pitch = (AudioManager._currentBgm.pitch = p) / 100; // Directly adjust source playback rate (perceived pitch) for (const s of b._sourceNodes) s.playbackRate.setValueAtTime(b._pitch, WebAudio_context.currentTime); // Recreate end timer structure b._startTime = WebAudio_context.currentTime - b.seek() / b._pitch; b._removeEndTimer(); b._createEndTimer(); }	// MZ: change the current BGM's pitch to 150%. if (AudioManager._currentBgm) { var b = AudioManager._bgmBuffer, p = 150; // Set new pitch directly to avoid restarting from 0 b._pitch = (AudioManager._currentBgm.pitch = p) / 100; // Directly adjust source playback rate (perceived pitch) for (const s of b._sourceNodes) s.playbackRate.setValueAtTime(b._pitch, WebAudio_context.currentTime); // Recreate end timer structure b._startTime = WebAudio_context.currentTime - b.seek() / b._pitch; b._removeEndTimer(); b._createEndTimer(); }	Number newPitch • New BGM pitch, see Audio Object for details.	These change the BGM pitch without restarting the track. They're much bigger script calls because they have to work around the core scripts, which normally restart the track when the pitch changes. The difference between MV and MZ is that MZ implements multiple source buffers per track.
✓	☐		if (AudioManager._currentBgm) { var b = AudioManager._bgmBuffer, p = newPitch ; // Set new pitch directly to avoid restarting from 0 b._pitch = (AudioManager._currentBgm.pitch = p) / 100; // Directly adjust source playback rate (perceived pitch) b._sourceNode.playbackRate.setValueAtTime(b._pitch, WebAudio_context.currentTime); // Recreate end timer structure b._startTime = WebAudio_context.currentTime - b.seek() / b._pitch; b._removeEndTimer(); b._createEndTimer(); }			
✓	✓	Play BGS	AudioManager.playBgs(bgs , pos);	// Play BGS raindrops.ogg with 25% volume, 80% pitch, 0 pan AudioManager.playBgs({ name: 'raindrops', volume: 25, pitch: 100 });	(Audio Object) bgs • Background sound object. Number pos (optional) • Start time, in seconds from the start of the track.	pos is optional; if omitted, it will default to 0.
✓	✓	Fadeout BGS	AudioManager.fadeOutBgs(seconds);	// Fade out the current BGS over 3 seconds AudioManager.fadeOutBgs(3);	Number seconds • Fadeout duration, in seconds.	-
✓	✓	Play ME	AudioManager.playMe(me);	// Play ME fanfare.ogg with 20% volume, 120% pitch, 0 pan AudioManager.playMe({ name: 'fanfare', volume: 20, pitch: 120 });	(Audio Object) me • Music effect sound object.	-
✓	✓	Play SE	AudioManager.playSe(se);	// Play SE beep.ogg at 5% volume, 100% pitch, panned 50% left AudioManager.playSe({ name: 'beep', volume: 5, pitch: 100, pan: -50 });	(Audio Object) se • Sound effect object.	-
✓	✓	Stop SE	AudioManager.stopSe();	-	-	Stops all currently-playing SE audio channels.
☐	✓	Play Movie	interpreter.command261([movieName]);	// In a Script command: play sample.webm this.command261(['sample']);	Game Interpreter interpreter • Interpreter to pause (<i>this</i> in a Script command). String movieName • Video filename, without extension.	-
✓	☐		interpreter._params = [movieName]; interpreter.command261();	// In a Script command: play sample.webm this._params = ['sample']; this.command261();		

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Change Battle BGM	<code>\$gameSystem.setBattleBgm(bgm);</code>	// Set the battle BGM to <i>Battle1.ogg</i> at 20% volume, 100% pitch <code>\$gameSystem.setBattleBgm({ name: 'Battle1', volume: 20, pitch: 100 });</code>	(Audio Object) bgm Background music object.	Sets the battle BGM.
✓	✓	Battle BGM	<code>\$gameSystem.battleBgm()</code>	-	Returns (Audio Object) The current battle BGM.	-
✓	✓	Change Victory ME	<code>\$gameSystem.setVictoryMe(me);</code>	// Set the victory ME to <i>Victory1</i> at 20% volume, 100% pitch <code>\$gameSystem.setVictoryMe({ name: 'Victory1', volume: 20, pitch: 100 });</code>	(Audio Object) me Music effect object.	Sets the victory/defeat ME.
✓	✓	Change Defeat ME	<code>\$gameSystem.setDefeatMe(me);</code>	// Set the defeat ME to <i>Defeat1</i> at 20% volume, 100% pitch <code>\$gameSystem.setDefeatMe({ name: 'Defeat1', volume: 20, pitch: 100 });</code>		
✓	✓	Victory ME	<code>\$gameSystem.victoryMe()</code>	-	Returns (Audio Object) Current victory/defeat music effect object.	-
✓	✓	Defeat ME	<code>\$gameSystem.defeatMe()</code>	-		
✓	✓	Change Vehicle BGM	<code>vehicle.setBgm(bgm);</code>	// Set the boat BGM to <i>Boat1</i> at 20% volume, 90% pitch, 0 pan <code>\$gameMap.boat().setBgm({ name: 'Boat1', volume: 20, pitch: 90 });</code>	Game_Vehicle vehicle - The vehicle to affect. (Audio Object) bgm - Background music object.	Sets a vehicle's BGM.
✓	✓	Vehicle BGM	<code>vehicle._bgm vehicle.vehicle().bgm</code>	// Get the current BGM for the ship vehicle <code>\$gameMap.ship()._bgm \$gameMap.ship().vehicle().bgm</code>	Game_Vehicle vehicle - The vehicle to check. Returns (Audio Object) - That vehicle's current BGM.	-
✓	✓	Play System Sound	<code>SoundManager.playSystemSound(soundId);</code>	// Play system sound 16 (Recovery) <code>AudioManager.playSystemSound(16);</code>	Number soundId ID of the system SE to play.	Alternatively, use the specific method (see below).
✓	✓		<code>SoundManager.playCursor();</code>			
✓	✓		<code>SoundManager.playOk();</code>			
✓	✓		<code>SoundManager.playCancel();</code>			
✓	✓		<code>SoundManager.playBuzzer();</code>			
✓	✓		<code>SoundManager.playEquip();</code>			
✓	✓		<code>SoundManager.playSave();</code>			
✓	✓		<code>SoundManager.playLoad();</code>			
✓	✓		<code>SoundManager.playBattleStart();</code>			
✓	✓		<code>SoundManager.playEscape();</code>			
✓	✓		<code>SoundManager.playEnemyAttack();</code>			
✓	✓		<code>SoundManager.playEnemyDamage();</code>			
✓	✓		<code>SoundManager.playEnemyCollapse();</code>			
✓	✓		<code>SoundManager.playBossCollapse1();</code>	-	-	-
✓	✓		<code>SoundManager.playBossCollapse2();</code>			
✓	✓		<code>SoundManager.playActorDamage();</code>			
✓	✓		<code>SoundManager.playActorCollapse();</code>			
✓	✓		<code>SoundManager.playRecovery();</code>			
✓	✓		<code>SoundManager.playMiss();</code>			
✓	✓		<code>SoundManager.playEvasion();</code>			
✓	✓		<code>SoundManager.playMagicEvasion();</code>			
✓	✓		<code>SoundManager.playReflection();</code>			
✓	✓		<code>SoundManager.playShop();</code>			
✓	✓		<code>SoundManager.playUseItem();</code>			
✓	✓		<code>SoundManager.playUseSkill();</code>			
Battle		Battle-specific scripting				▲ Return to top ▲

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Is the Party in Battle?	\$gameParty.inBattle()	-	Returns Boolean • true iff the party is currently in battle.	-
☐	✓	Show Animation (Battler)	\$gameTemp.requestAnimation([battler], animationId, mirror);	// MZ, show animation 9 on enemy #2 (index 1), not mirrored \$gameTemp.requestAnimation([\$gameTroop.members()[1]], 9);	Game_Battler battler • Animation target. Number animationId • ID of animation to show.	Can have multiple targets. This is typically for animations with <i>Display Type: Center of all targets</i> .
✓	☐		battler.startAnimation(animationId, mirror, delay);	// MV, show animation 9 on enemy #2 (index 1) \$gameTroop.members()[1].startAnimation(9);	Boolean mirror • If true, mirror the animation. Number delay (optional) • Frames before the animation starts.	-
✓	✓	Force Action (Wait)	var subject = battler; subject.forceAction(skillId, targetIndex); BattleManager.forceAction(subject); interpreter.setWaitMode('action');	// Force the party leader to use skill ID 5 on a random target var subject = \$gameParty.members()[0]; subject.forceAction(5, -1); BattleManager.forceAction(subject);	Game_Battler battler • The action user. Number skillId • ID of the skill to use. Number targetIndex • Determines the action target: ◦ -2: user's previous target; ◦ -1: random from target unit; ◦ 0+: specific index from target unit.	Force action will clear the user's current action list.
✓	✓	Force Action (No Wait)	var subject = battler; subject.forceAction(skillId, targetIndex); BattleManager.forceAction(subject);	// In a Script command... // Force enemy #1 (index 0) to use skill 18 on target index 0 var subject = \$gameTroop.members()[0]; subject.forceAction(18, 0); BattleManager.forceAction(subject); this.setWaitMode('action');	Game_Interpreter interpreter • Interpreter to pause (this in a Script command)	
✓	✓	Abort Battle	BattleManager.abort();	-	-	Aborts the battle. Mechanically similar to escape, but will not be counted in the <i>Battles escaped</i> system statistic.
✓	✓	Escape (Battler)	battler.escape();	// Make enemy #2 (index 1) escape \$gameTroop.members()[1].escape();	Game_Battler battler • The one to escape.	Hides them, clears their actions/states, plays the escape SE, and triggers any relevant visual effects.
✓	✓	Appear	battler.appear(); \$gameTroop.makeUniqueNames();	// Make enemy #2 (index 1) appear \$gameTroop.members()[1].appear(); \$gameTroop.makeUniqueNames();	Game_Battler battler • The one to appear.	This is typically used for enemies marked <i>Appear Mid-Battle</i> in their troop. It can also be used to bring back a battler who escaped earlier this battle.
✓	✓	Enemy Transform	enemy.transform(newEnemyId); \$gameTroop.makeUniqueNames();	// Transform enemy #1 (index 0) into enemy ID 8 \$gameTroop.members()[0].transform(8); \$gameTroop.makeUniqueNames();	Game_Enemy enemy • The one to transform. Number newEnemyId • Database ID of the transformed enemy.	HP/MP values will be lowered to the new maximum values after transforming, if appropriate.
✓	✓	Inputting Action	BattleManager.inputtingAction()	-	Returns Game_Action • The action being selected right now.	Input phase: gets the action currently being input, i.e. choosing skill/item and target(s).
✓	✓	Inputting Actor	BattleManager.actor()	-	Returns Game_Actor • The actor who is selecting actions right now.	Input phase: gets the actor who is currently choosing their action (s).
✓	✓	Active Battler	BattleManager._subject	-	Returns Game_Battler • The battler who is currently acting.	Turn phase: gets the battler who is currently performing their action(s).
✓	✓	Current Target Index	action._targetIndex	// In a damage formula: do double damage if targetting index 0 (a.atk * 4 - b.def * 2) * (this._targetIndex === 0 ? 2 : 1)	Game_Action action • The action to check. Returns Number • The target index of that action.	This indicates the selected/primary target. The action may also affect other targets.
✓	✓	Action Targets	BattleManager._targets	// In a damage formula: count remaining targets (see note! -->) BattleManager._targets.length	Returns Game_BattlerBase[] • Array of targets for this action.	The <i>BattleManager</i> shifts out the next target immediately before invoking the action on that target. Therefore this will not include the current target if checked mid-action, e.g. in the damage formula.
✓	✓	Check if Battle is Escape-able [Can Escape from Battle?]	BattleManager.canEscape();	-	Returns Boolean • true iff the party can escape from this battle.	-
✓	✓	Show Battle Log Text	BattleManager._logWindow.addText(text);	// Show Hello world! in the battle log BattleManager._logWindow.addText('Hello world!');	String text • Text to show in the battle log.	-
☐	✓	Refresh Battle Status	SceneManager._scene._statusWindow.preparePartyRefresh();	-	-	Loads relevant face images then refreshes the status window. Useful for if the party changes mid-battle!
✓	✓		SceneManager._scene._statusWindow.refresh();	-	-	Refreshes the status window. For MZ: this is OK if you're certain all face images are already loaded in memory.
Last Action Data		Check recent item/skill usage info.		▲ Return to top ▲		
✓	✓	Last Selected Item	\$gameParty.lastItem()	// Get ID of last selected Item, or 0 if no Item was selected yet \$gameParty.lastItem() ? \$gameParty.lastItem().id : 0	Returns \$dataItem • Representation of most recently-selected Item.	This retrieves the most recently-selected Item in menu or battle. There is no guarantee the Item was used, e.g. if it was selected, then cancelled before choosing a target. Also, it may have been removed from the inventory since then.
☐	✓	Last Used Skill ID	\$gameTemp.lastActionData(0)	-	-	Updated when an action starts (turn phase). [Game_Action#applyGlobal.]
☐	✓	Last Used Item ID	\$gameTemp.lastActionData(1)			
☐	✓	Last Actor ID to Act	\$gameTemp.lastActionData(2)			
☐	✓	Last Enemy Index to Act	\$gameTemp.lastActionData(3)			
☐	✓	Last Target Actor ID	\$gameTemp.lastActionData(4)			Updated immediately after applying an action's damage/effects to a target. [Game_Action#apply.]
☐	✓	Last Target Enemy Index	\$gameTemp.lastActionData(5)			

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Damage Formula	Some damage formula stuff~			▲ Return to top ▲
✓	✓	(eval condition)	The damage formula does not get evaluated if the action misses.			
✓	✓	(auto-battle eval)	Actors with an Auto Battle trait will evaluate the damage formula for each of their available actions on a clone of each available target to decide which skill to use.	Plugins might also extend this to enemy AI!		
✓	✓	Action	this	-	Returns <u>Game_Action</u> • The action object.	I.e. the damage formula is evaluated in a <i>Game_Action</i> context.
✓	✓	Subject	a	-	Returns <u>Game_Battler</u> • The action user.	-
✓	✓	Target	b	-	Returns <u>Game_Battler</u> • The action target.	Actions with multiple targets are evaluated on each target separately.
✓	✓	Item	item	-	Returns <u>\$dataItem</u> <u>\$dataSkill</u> • Database reference for this action's item.	Gets the <i>\$dataItems</i> or <i>\$dataSkills</i> reference for this action's item. See the Database sheet of the MZ Script Call Reference for details on available properties.
✓	✓	Get Game Variable	v[variableId] 0	// Get the value of game variable 5 v[5] 0	Number variableId • ID of game variable to check/change. Returns <u>Any</u> • That variable's value, or 0 if it hasn't been set yet.	Because this is a raw reference, variables that have not been assigned a value will be <i>undefined</i> . v[1] 0 returns 0 if v[1] is "falsy". More information here: Logical OR .
✓	✓	Set Game Variable	v[variableId] = value;	// Set game variable 5 equal to 20 v[5] = 20;	Number variableId • ID of game variable to check/change. Any value • New value.	Alternatively you can use the usual methods: • <i>\$gameVariables.value(variableId)</i> • <i>\$gameVariables.setValue(variableId, value);</i>
✓	✓			// Add 10 to game variable 1 v[1] = (v[1] 0) + 10;		
✓	✓			// Multiply game variable 1 by 2 v[1] = (v[1] 0) * 2;		
✓	✓	Set Temporary Variable	var name = value;	// Normal formula a.atk * 2 - b.def; 20% chance to multiply by 5 var r = a.atk * 2 - b.def; Math.random() < 0.2 ? r * 5 : r	name • Temporary variable identifier. Any value • Value to store.	The variable will be deleted after the damage formula is evaluated. This can be handy to avoid calculating the same thing more than once. You can use ES6 <i>let/const</i> instead, if supported.
✓	✓	Action Result	b.result()	-	Returns <u>Game_ActionResult</u> • The current result of the action on its target.	-
✓	✓	Check Result	b.result().critical	// If this action is a critical hit then ignore the target's DEF a.atk * 2 - (b.result().critical ? 0 : b.def)	Returns <u>Boolean</u> • true iff the action is a critical hit.	This is one of the few properties worth checking here because the formula only evaluates when the action hits.
✓	✓	Change Visual Result	b.result().used = false;	// 50% extra chance to miss & do 0 damage; otherwise hit for 200 if (Math.random() < 0.5) { b.result().missed = true; 0 } else { 200 }	-	Hide all post-apply action messages/popups.
✓	✓		b.result().missed = true;			Show "Miss" popup + miss message.
✓	✓		b.result().evaded = true;			Show "Miss" popup + evade message.
✓	✓		b.result().critical = value;			If true, do crit effects (red flash + message).
✓	✓		b.result().physical = value;	b.result().critical = true; 10 * a.atk	Boolean value • true (enable) or false (disable).	Determines the miss/evade message (magic/physical).
		Cache Management	Advanced (the cache can generally handle itself)			▲ Return to top ▲
✓	✓	Are all Requested Images Loaded?	ImageManager.isReady()	-	Returns <u>Boolean</u> • true iff all cache entries are loaded into memory.	Automatically checked before any scene starts.
☐	✓	Are a Tileset's Images Loaded?	// Check (repeatedly) while loading \$dataTilesets[tilesetId].tilesetName .map(pageName => ImageManager.loadTileset(pageName)) .every bmp => bmp.isReady()	// MZ: are tileset 5's images all loaded? \$dataTilesets[5].tilesetName .map(pageName => ImageManager.loadTileset(pageName)) .every bmp => bmp.isReady()	Number tilesetId • ID of tileset to check. Game_Interpreter interpreter • Interpreter reference (this in a Script command). Returns <u>Boolean</u> • true iff that tileset's images are all in memory.	Can be used to check if the new tileset images are done loading before calling <i>\$gameMap.changeTileset</i> . MV requires additional before and after stages to interface with its tileset image reservation system.
✓	☐		// Initialise tileset reservation ID var self = interpreter ; if (self._imageReservationId) { ImageManager.releaseReservation(self._imageReservationId); } self._imageReservationId = Utils.generateRuntimeId();	// MV, in a Script command: setup reservation if (this._imageReservationId) { ImageManager.releaseReservation(this._imageReservationId); } this._imageReservationId = Utils.generateRuntimeId();		
✓	☐		// Check (repeatedly) while loading var reservationId = interpreter ._imageReservationId; \$dataTilesets[tilesetId].tilesetName .map(function(p) { return ImageManager.reserveTileset(p, 0, reservationId); }).every(function bmp) { return bmp.isReady(); })	// MV, in a Script command: are tileset 5's images loaded? var reservationId = this._imageReservationId; \$dataTilesets[5].tilesetName .map(function(p) { return ImageManager.reserveTileset(p, 0, reservationId); }).every(function bmp) { return bmp.isReady(); })		
✓	☐		// Remove reservation when loaded var self = interpreter ; ImageManager.releaseReservation(self._imageReservationId); self._imageReservationId = null;	// MV, in a Script command: remove reservation, change to tileset 5 ImageManager.releaseReservation(this._imageReservationId); this._imageReservationId = null; \$gameMap.changeTileset(5);		

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
		Clear Image Cache (1 Entry)	const bmp = ImageManager._cache[path]; if (bmp) bmp.destroy();	// MZ: remove the chicken map character sheet from image cache const bmp = ImageManager._cache['img/characters/chicken.png']; if (bmp) bmp.destroy();	<u>String path</u> • File path of image, e.g. "img/pictures/test.png". <u>Number hue</u> • Hue shift of the image.	Clears data from the game's cache.
			var key = ImageManager._generateCacheKey(path, hue); delete ImageManager._imageCache._items[key];	// MV: remove the chicken map character sheet from image cache var path = 'img/characters/chicken.png'; var key = ImageManager._generateCacheKey(path, 0); delete ImageManager._imageCache._items[key];		
		Clear Image Cache (All Entries)	ImageManager.clear();	-	-	This clears the game engine's cache reference. RAM occupied by the object data will only be available for release if there are no remaining references to it. For more details, see: Memory Management .
		Clear Effectseer Cache (1 Entry)	const path = EffectManager.makeUrl(filename); const effect = EffectManager._cache[path]; if (effect) Graphics.effectseer.releaseEffect(effect);	// MZ: remove the Absorb effect from the effect cache const path = EffectManager.makeUrl('Absorb'); const effect = EffectManager._cache[path]; if (effect) Graphics.effectseer.releaseEffect(effect);	<u>String filename</u> • Name of the .efkefc file, without extension.	
		Clear Effectseer Cache (All Entries)	EffectManager.clear();	-	-	
Plugins						
Plugin Header		Plugin header details				▲ Return to top ▲
		Locale	/*:locale // etc */	/*: * @plugindesc Non-Japanese plugin description. */ /*:ja * @plugindesc プラグインのタイトルです。 */	locale (optional) • Locale code for this header. • Default header should have no locale code.	Allows multiple headers for localisation purposes. Pairs with the @text directive for parameters. Can also be specified for struct definitions (see below).
		Target	@target MZ	-	-	Suppresses this warning in MZ's Plugin Manager: <i>The plugin "name" may not support RPG Maker MZ.</i>
		Description	@plugindesc description	@plugindesc v1.0 - this is a plugin description~	description • Summary text to show in the Plugin Manager.	Including a version number at the start is recommended: that makes it much easier to check the plugin's version.
		Author	@author author	@author Me	author • Plugin author(s), shown in the Plugin Manager.	-
		URL	@url url	@url https://forums.rpgmakerweb.com/	url • Link displayed in the Plugin Manager.	Useful to link back to your plugin page, e.g. for support.
		Required Plugin	@base requiredPlugin	@base CorePlugin1 @base CorePlugin2	requiredPlugin • Name of required plugin, without extension.	If the specified plugin is not enabled in the load order, the Plugin Manager will show a warning. Use multiple directives to specify multiple required plugins.
		Load Order	@orderAfter beforePlugin	@orderAfter CorePlugin1 @orderAfter CorePlugin2	beforePlugin • Name of plugin that should load before this one.	If the specified plugin is present and is in the wrong order relative to this one, the Plugin Manager will show a warning. Use multiple directives to specify multiple plugins.
			@orderBefore afterPlugin	@orderBefore AddOnPlugin1 @orderBefore AddOnPlugin2	afterPlugin • Name of plugin that should load after this one.	
		Help	@help text	@help This is some text explaining how the plugin works. More text here!	text • Text shown in the big area in the Plugin Manager.	This can be used for instructions, examples, terms of use, changelog, etc.
		Plugin Parameters	@param name @text displayName @parent parent @type type @desc description @default default	@param myParam @text My Param @type string @desc This is a description of my param! @default Default text	name • Internal identifier for this parameter. displayName (optional) • Parameter name shown in the Plugin Manager. parent (optional) • The name of another parameter, for nesting. type (optional) • Parameter type; see below for definitions. description • Help text shown for this parameter. default • Starting value for this parameter.	Handy for values that you want users to be able to configure for themselves! You can define as many parameters as you like. type is only for ease of use: it can be bypassed by switching to the Text tab in the Plugin Manager. Parameters are stored in JSON/string format regardless of their type.
		Required File (Parameter)	@require 1	// Mark the file selected for this parameter as required @param ME @desc The ME name for sample music effect. @type file @dir audio/me/ @require 1 @default Organ	-	Optional extra for plugin parameters. Includes a resource specified in a plugin parameter when deploying with the <i>Exclude unused files</i> option.
		Required File (Specific)	@requiredAssets path	// Mark the file img/tutorial/img1.png as required @requiredAssets img/tutorial/img1	path • Relative path to a required resource file.	Includes specific resources when deploying with the <i>Exclude unused files</i> option.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Required File (Notetag)	@noteParam name @noteRequire 1 @noteDir path @noteType type @noteData dataType	// The Actor notetag <portrait:filename> marks required pictures @noteParam portrait @noteRequire 1 @noteDir img/pictures @noteType file @noteData actors	name • Notetag name. path • Path to resource directory. type • Parameter type (see below). dataType • Note fields to scan for this tag; see notes ->	Includes resources specified in notetags when deploying with the <i>Exclude unused files</i> option. Values for dataType : • maps • classes • weapons • states • events • skills • armors • tilesets • actors • items • enemies
✗	✓	Plugin Command	@command name @text displayName @desc description	@command My Command @desc This command does a thing!	name • Internal identifier for this command. displayName (optional) • Name shown in Plugin Command window. description • Help text shown for this command.	Should be followed by 0+ command argument blocks, as detailed below.
✗	✓	Plugin Command Argument	@arg name @text displayName @type type @desc description @default default	@arg Gain Item @type item @desc The item to give to the party. @default 0	name • Internal identifier for this argument. displayName (optional) • Name shown in the Plugin Command window. type (optional) • Argument type (a parameter type, see below). description • Help text shown for this argument. default • Starting value for this argument.	-
		Plugin Header: Types	Plugin parameter/argument types		▲ Return to top ▲	
✓	✓	Type: String	@type text @type string	@param Some Text	-	The default type, used if you do not include a @type directive or if the specified type is unrecognised.
✗	✓	Type: Multi-Line String	@type multiline_string	@param Multiline Text @type multiline_string @desc Enter multiple lines of text. @default	-	-
✓	✓	Type: Note	@type note	@param A Note @type note @desc Enter multiple lines of text. @default	-	Apply JSON.parse before use.
✓	✓	Type: Number	@type number @min min @max max @decimals decimals	@param numberParam @type number @min 1 @max 100 @decimals 2 @desc A value from 1.00 to 100.00 @default 1.00	Number min • Minimum input value. Number max • Maximum input value. Number decimals (optional) • Available decimal places.	Stored as a string. See below for parsing suggestions.
✓	✓	Type: File	@type file @dir path	@param Overlay @type file @dir img/parallax @desc Pick a parallax image! @default	path • The directory where the file is located.	Stores only the remaining path, without extension. E.g. for the example, if you select img/parallax/Stars.png, it will store "Stars".
✓	✓	Type: Boolean (true/false)	@type boolean @on onText @off offText	@param Enabled @on Yes @off No @desc Use this feature? @default false	onText (optional) • Text to show instead of ON offText (optional) • Text to show instead of OFF	Stored as "true" or "false". See below for parsing suggestions.
✓	✓	Type: Select	@type select @option option1 @value value1 @option option2 @value value2 etc	@param A number @type select @option 3 @option 2 @option 1 @desc Pick a number! @default 1	option1, option2, ... • Display names for each choice. value1, value2, ... (optional) • Values for each choice.	The selection is stored as its value, if specified; otherwise its option name.
✓	✓	Type: Combo	@type combo @option option1 @option option2 etc	@param Dog name @type combo @option Arfthur @option Barkley @option Charles @desc Pick a dog name, or enter your own! @default Arfthur	option1, option2, ... • Preset values.	Allows the user to enter their own value directly, without using the Text tab.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	Type: Game Object	@type object	@param Switch ID @type switch @desc Pick a switch! @default 0	object • One of the following: ◦ actor, class, skill, item, weapon, armor, enemy troop, state, animation, tileset, common_event, switch, variable.	Stored as the object's database ID. "0" = (None).
✓	✓	Type: Structure (Custom Object)	@type struct<structName>	@param BGM @type struct<bgm> @desc Some BGM settings.	structName • A valid identifier for the structure. locale (optional) • Locale code for this header section (see above).	Each /*~struct~etc definition should be its own "header" section, either before or after the main one. Structs allow nesting parameters inside other parameters. Apply JSON.parse before use.
✓	✓		/*~struct~structName:locale * params */	/*~struct~bgm: * @param name * @param volume * @type number * @min 0 * @max 100 * @default 20 */		
✓	✓	Type: List (Array)	@type type[]	@param Region IDs @type number[] @min 1 @max 255 @desc Enter a list of region IDs! @default []	type • Any of the above parameter types.	Allows the user to enter an array of values, of any length. Apply JSON.parse before use.
✓	✓	Type: Icon Index	-	-	-	Not implemented as a type, but users can right-click a Text parameter field and choose Iconset Viewer / Insert Icon Index to bring up the icon selector.
		Plugin Code	Code to interface with the core scripts			▲ Return to top ▲
✓	✓	Get Plugin Parameters	PluginManager.parameters(pluginName)	// Get the parameters for myPlugin.js PluginManager.parameters('myPlugin')	String pluginName • Name of the plugin file, without extension. Returns Object • One property per parameter. • If no parameters are found: ◦ MZ returns an empty object; ◦ MV returns undefined.	By default all parameters are in JSON/stringified format; see below for parsing suggestions. Auto-parsing is discouraged without extra hints (e.g. prefix number param names with <i>num</i>), because data type is not always decidable based on value alone. E.g. should "000" be a 3-character string or the number 0?
✓	✓			// 2 different ways to get the parameter named Switch PluginManager.parameters('myPlugin').Switch PluginManager.parameters('myPlugin')['Switch']		
✓	✓			// Get the parameter named Region IDs PluginManager.parameters('myPlugin')['Region IDs']		
✗	✓	Define Plugin Commands	PluginManager.registerCommand(pluginName , commandName , function);	// Register command Open Equips for plugin Shortcuts.js PluginManager.registerCommand('Shortcuts', 'Open Equips', args => { const actorId = parseInt(args['Actor ID'], 10); if (actorId) { \$gameParty.setMenuActor(actorId); SceneManager.push(Scene_Equip); } });	String pluginName • Plugin filename, without extension. String commandName • Name of the command. Function function • Runs when the command is invoked.	The function will run in a <i>Game_Interpreter</i> context. It will receive a single argument: an object with one property per command argument. Property names match the argument names.
✓	✗		Game_Interpreter.prototype.pluginCommand = function(command, args) { if (command === commandName) { commandCode } alias.apply(this, arguments); };	// Add plugin command OpenEquips to open the equip scene (function(alias) { Game_Interpreter.prototype.pluginCommand = function (command, args) { if (command === 'OpenEquips') { var actorId = parseInt(args[0], 10); if (actorId) { \$gameParty.setMenuActor(actorId); SceneManager.push(Scene_Equip); } } } else alias.apply(this, arguments); }; })(Game_Interpreter.prototype.pluginCommand);	String commandName • Name of the command. commandCode • Code to invoke the command's effect. alias • Reference to the original <i>pluginCommand</i> method.	Plugin Command input is split on spaces into an array of strings. The first entry is the command; the following entries are the arguments. args is an array (possibly empty) of strings.
✓	✓	Notetags	dataObject .meta.tagName	// Get the value of the <portrait: value> tag on actor ID 1 \$dataActors[1].meta.portrait // Get the value of the <portrait: value> tag on actor ID 1 \$gameActors.actor(1).actor().meta.portrait	\$data dataObject • A database object reference. tagName, tag name • Name of the notetag to check. • Bracket notation is needed for names with spaces.	Tags without a value, e.g. <save location>, are stored as true. Other tags are stored as string values.
✓	✓					
✓	✓		dataObject .meta['tag name']	// Do any of the party leader's trait objects have a <repel> notetag? \$gameParty.members()[0].traitObjects().some(function(obj) { return obj && obj.meta.repel; });		Alternatively, if you want a different tag format, you can parse/store your notetags yourself. Regular Expressions are useful for parsing text.

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
✓	✓	String -> Boolean	json === 'true'	// Is boolean @param Enabled set to "true"? PluginManager.parameters('myPlugin').Enabled === 'true';	String json • Parameter value (JSON string). Returns Boolean • true iff the parameter is set to "true".	-
✓	✓	String -> Integer	parseInt(json, radix)	// Parse @param Decimal number and @param Integer number var params = PluginManager.parameters('myPlugin'); var fNum = Number(params["Decimal number"]); var iNum = parseInt(params["Integer number"], 10);	String json • Parameter value (JSON string). Number radix • Base of number to parse, usually 10. Returns Number • String converted to a number.	Truncates to an integer, details here: parseInt .
✓	✓	String -> Number	Number(json)			-
✓	✓	Note -> String		// Parse array of numbers @param Region IDs var params = PluginManager.parameters('myPlugin'); var regions = JSON.parse(params["Region IDs"]).map(function(n) { return parseInt(n, 10); });	String json • Parameter value (JSON string). Returns String Any[] Object • Appropriate representation of that data type.	An array/struct parameter may require multiple parses (e.g. array of structs).
✓	✓	String -> List/Array	JSON.parse(json)			
✓	✓	String -> Struct				
✓	✓	Method Override (ES5-Compatible)	Base_Object.prototype.methodName = function(...) { replacement };	// Override amount the volume changes per "click" in the options Window_Options.prototype.volumeOffset = function() { return 5; // originally 20 };	replacement • New method code.	Replaces the original method entirely. Sometimes necessary, but can easily cause conflicts with other plugins that alter the same method.
✓	✓	Method Patch (ES5-Compatible)	alias = Base_Object.prototype.methodName; Base_Object.prototype.methodName = function(...) { pre alias .apply(this, arguments); post };	// Turn on switch 1 after every player step (function(\$){ \$._Game_Player.increaseSteps = Game_Player.prototype.increaseSteps; Game_Player.prototype.increaseSteps = function(){ \$._Game_Player.increaseSteps.apply(this, arguments); \$gameSwitches.setValue(1, true); }; })(myPlugins.myFirstPlugin myPlugins.myFirstPlugin = {});	alias • A variable used to store the original method. pre • Code to run before the original method. post • Code to run after the original method.	This is an override, except the original method is retained and invoked as part of the new method. This greatly reduces the chance of compatibility problems with other plugins! If alias is global (e.g. a property of a global object, like the example) then other plugins can reference it. This can help improve compatibility further, e.g. by skipping incompatible feature(s) regardless of load order.
Code Style Suggestions			Suggestions for plugin coding style			
✓	□	Back-Compatibility	-	-	-	Earlier versions of RMMV (before v1.6.0) used ES5, which lacks support for some modern JavaScript features like arrow functions, class syntax, destructuring, etc. This affects playtest and any games not deployed for Web. If you want your plugin to support older RMMV versions, consider avoiding ES6+ features.
✓	✓	Scope	(function(){ // private scope })();	var cake = 'tasty'; (() => { var cake = 'a tie'; console.log(cake); // 'a tie' })();	-	IIFEs (immediately-invoked function expressions) can be used to provide an anonymous scope. This can help to avoid name clashes between plugins.
✓	✓		(() => { // private scope (newer syntax) })();	console.log(cake); // 'tasty'		
□	✓	Namespaces	// main namespace (global, newer syntax) window.MyPlugins = {}; // sub-namespaces (newer syntax) MyPlugins.myFirstPlugin = {};	window.MyPlugins = {}; (\$ => { // in here, \$ = MyPlugins.myFirstPlugin \$.cake = 'tasty'; // etc })(MyPlugins.myFirstPlugin = {}); console.log(MyPlugins.myFirstPlugin.cake); // 'tasty'	-	It is common practice to define a single global variable: a "namespace". You can then define all your public plugin data as properties (or sub-properties, etc) of this global variable. This helps to keep things organised and minimises the chance of a naming clash.
✓	✓		// main namespace (global) window.MyPlugins (window.MyPlugins = {}); // sub-namespaces MyPlugins.myFirstPlugin (MyPlugins.myFirstPlugin = {});			
✓	✓	Import & Version Declarations	MyPlugins. pluginName .version = value ;	MyPlugins.myFirstPlugin.version = 1.0;	pluginName • Internal name for the plugin space. Any value • A version number, or simply true/false.	Something like this can help people to write plugins compatible with yours. It makes it easy to check if a plugin is loaded (and what version it is).
✓	✓		// Using a global "import namespace" window.Imported (window.Imported = {}); Imported. pluginName = value ;	window.Imported = {};		
✓	✓		// Using a global "import namespace" (newer syntax) window.Imported = {}; Imported. pluginName = value ;	window.Imported.myFirstPlugin = 1.0;		
✓	✓	Eval vs Functions (Runtime Code Evaluation)	// Function constructor var fct = new Function(a , b , ..., body); // Invoke with values fct(1, 2, ...);	// Compile all <condition: code> State notetags into functions var stateConditions = {}; for (var n = \$dataStates.length; n > 0; n--) { var tag = \$dataStates[n].meta.condition; if (tag) stateConditions[n] = new Function('user', tag); } // Call later by passing in a value for the user argument, e.g. return stateConditions[id] ? stateConditions[id](user) : false;	String a , b , ... (optional) • Expression/function input names (arguments). String body • Expression/function code, as a string.	Functions are good, especially for things you want to run often. You can use the Function constructor to create functions at runtime based on user-defined text, e.g. plugin parameters.
✓	✓					

Script Calls		Last updated 2023-12-14 (01:50 UTC)				
MV	MZ	Effect	Script call(s)	Example(s)	Inputs/Outputs	Notes
☒	☒		// Eval var a = A, b = B, ...; eval(body)	// Eval the <i>Enable Debug</i> parameter of test.js if (eval(PluginManager.parameters('test')['Enable Debug'])) { console.log('debug active'); }		Eval is OK for things you do not need to evaluate often. It is slower because it compiles the string into code on the spot every time.
		WIP	scrap area			▲ Return to top ▲
☐	☐	(nothing here!)				
☐	☐					
☐	☐					
☐	☐					
		▲ Return to top ▲				